

```
//CSE  
110;
```

The Making of a Software Engineer

with Thomas Powell

Today's Agenda

- Last time in CSE 110 Explained
 - The memes
 - A few words on the advice on speed, unknown, etc.
- Short version of the logistics and first day announcements and some awareness of notes and slides
- Reviewing the Lab aka “Why should I care>” or “Why does this matter?”
- A lecture, class activity, and tech review

(Re)Warning about the topic & a bit about last time



Ethan Mollick @emollick · Aug 21

A bias I wish everyone on Twitter knew: the Illusion of Explanatory Depth: "People feel they understand complex phenomena with far greater precision, coherence & depth than they really do."

Try to explain how something works to yourself. It shatters the Illusion & lets you learn

This confusion of environment support with internal representation is related to a confusion that has been noted in the "change blindness" literature: people grossly overestimate their ability to remember what they have observed in a scene. This phenomenon, termed "change blindness blindness," presumably occurs because people are mistaken about how visual information is stored—they confuse their ability to acquire details by re-sampling a live scene with exhaustive, VCR-like storage of everything one sees (Levin, Momen, Drivdahl, & Simons, 2009).

The confusion of environmental support for detailed representation might be expected to be strongest for phenomena that have perceptually vivid mechanisms. If we can see many of the "working parts" of a system, we may assume that the mechanisms can be easily internalized. But there is far more complexity in the interactions of the parts than is immediately apparent. Furthermore, as suggested by the change blindness literature, we may assume we remember vividly things we have seen as vivid.

A second feature leading to the IOED may be a confusion of higher with lower levels of analysis. Most complex artificial and natural systems are hierarchical in terms of explanations of their natures. In explaining a car one might describe the function of a unit, such as the brakes, in general terms, and then turn to describing the functions of subcomponents, such as pistons and brake pads, which in turn can be broken down even further. The iterative nature of explanations of this sort (Miyake, 1986) may lead to an illusion of understanding when a person gains insight into a high level function and, with that rush of insight, falsely assumes an understanding of further levels down in the hierarchy of causal mechanisms. This effect can easily happen for many natural and artificial systems with complex causal structures, especially those that have "stable subassemblies." The concept of stable subassemblies was developed by Simon (1996) as a way of describing units in the hierarchical structure of complex systems that are sufficiently internally stable that they can be conceived of as an operational unit.

Confusion between higher and lower levels of analysis may be related to the confusion of environmental support with representation, especially for perceptually vivid mechanisms which may trigger a sense of understanding at higher levels. For example, functional sub-assemblies that are easy to visualize and mentally animate may lead to strong (but mistaken) feelings of understanding at a high level of analysis, and thereby induce inaccurate feelings of comprehension about the lower levels.

A third feature of explanations leading to the illusion is related to the second: because explanations have complex hierarchical structure they have indeterminate end states. Therefore, self-testing one's knowledge of explanations is difficult. In contrast, determining how well one knows, e.g., a fact, can be trivially simple. Do you know the capital of England? If you can produce "London," then "yes." Similarly, to assess whether one knows a procedure one can envision a clear end state (e.g., a baked cake, a successful log-on to the Internet) and then work backwards to see if one knows how to get to that state. Errors of omission are still possible but they are constrained by knowledge of the end state. But with explanatory understanding one usually has little idea of what the final explanation will look like and the end state is

Published in final edited form as:
Cogn Sci. 2002 September 1; 26(5): 521-562. doi:10.1207/s15516709cog2605_1.

The misunderstood limits of folk science: an illusion of explanatory depth

Leonid Rozenblit* and Frank Keil
Department of Psychology, Yale University, 2 Hillhouse Avenue, P.O. Box 208205, New Haven, CT 06520-8205, USA

Abstract

People feel they understand complex phenomena with far greater precision, coherence, and depth than they really do; they are subject to an illusion—an illusion of explanatory depth. The illusion is far stronger for explanatory knowledge than many other kinds of knowledge, such as that for facts, procedures or narratives. The illusion for explanatory knowledge is most robust where the environment supports real-time explanations with visible mechanisms. We demonstrate the illusion of depth with explanatory knowledge in Studies 1-6. Then we show differences in overconfidence about knowledge across different knowledge domains in Studies 7-10. Finally, we explore the mechanisms behind the initial confidence and behind overconfidence in Studies 11 and 12. Implications for the roles of intuitive theories in models of concepts and cognition are discussed.

Keywords

Concepts; Epistemology; Meta-cognition; Knowledge; Overconfidence

1. Introduction

Intuitive or lay theories are thought to influence almost every facet of everyday cognition. People appeal to explanatory relations to guide their inferences in categorization, diagnosis, induction, and many other cognitive tasks, and across such diverse areas as biology, physical mechanics, and psychology (Gopnik & Wellman, 1994; Keil, 1998; Murphy & Medin, 1985; Murphy, 2000). Individuals will, for example, discount high correlations that do not conform to an intuitive causal model but overemphasize weak correlations that do (Chapman & Chapman, 1969). Theories seem to tell us what features to emphasize in learning new concepts as well as highlighting the relevant dimensions of similarity (Murphy, 2002). Intuitive theories have also been heavily emphasized in accounts of the cognitive development of children (Gelman & Koenig, 2002) and even of infants (Spelke, Breinlinger, Macomber, & Jacobson, 1992).

25

295

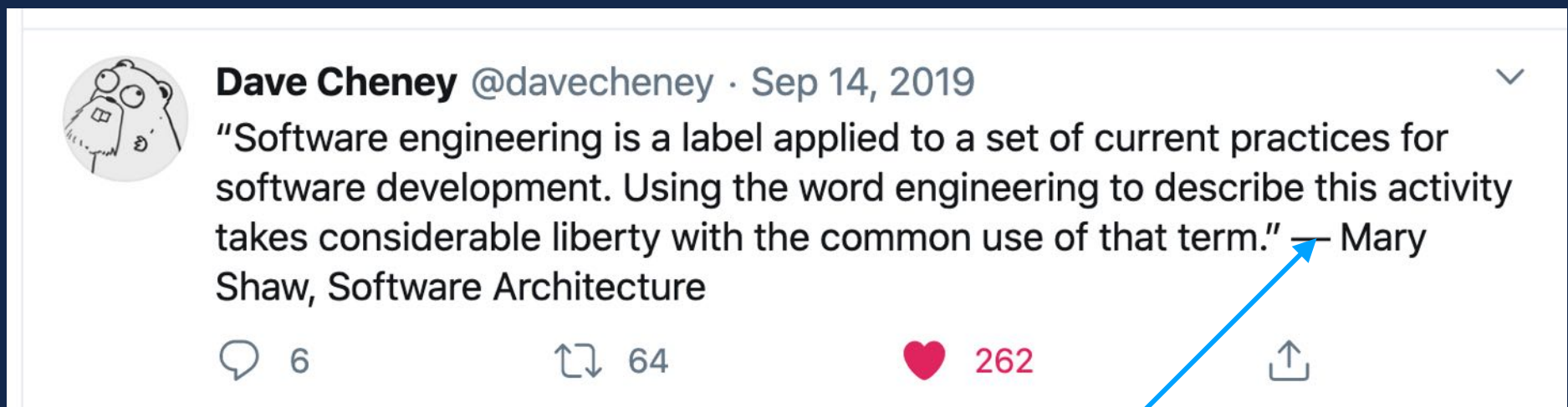
1,279



<https://thedecisionlab.com/biases/the-illusion-of-explanatory-depth>

<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC3062901/>

Also before you get all crazy about any of this



Before you go whatever rando

Oh Snap!

[https://en.wikipedia.org/wiki/Mary_Shaw_\(computer_scientist\)](https://en.wikipedia.org/wiki/Mary_Shaw_(computer_scientist))

Oh and a Meme “Thank You” and Decode



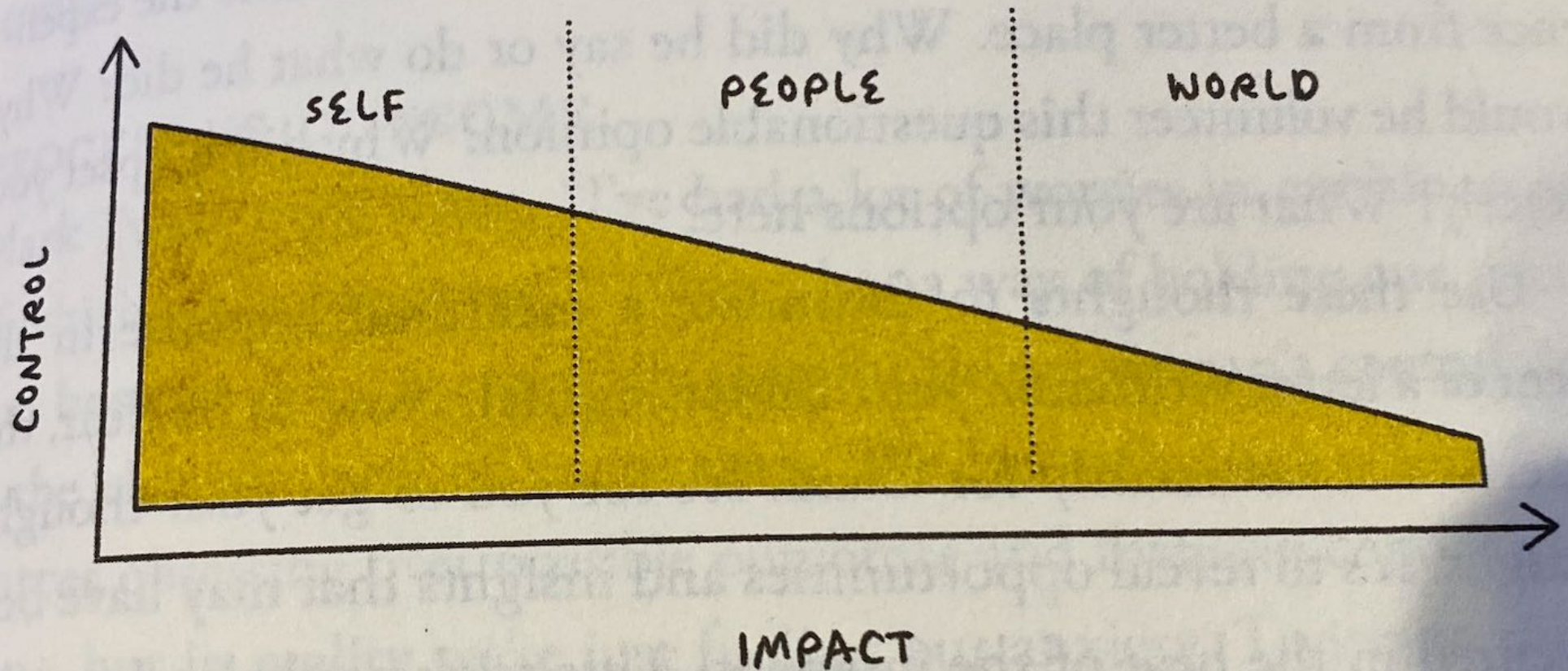
We break to a vibe coding session now in progress



Let's Start Topic #2

We are starting from the center and working out...from us, to groups, to play style (aka process), to strategy, tools, etc.

Why Start Here?



Simple! We have max control and impact!

Also we will find out it matters most for SE!

Pragmatic Professionals Wanted

One of the cornerstones of the pragmatic philosophy is the idea of taking responsibility for yourself and your actions in terms of your career advancement, your project, and your day-to-day work.

The Pragmatic Programmer

Andrew Hunt, David Thomas



Before We Present Info
Let's Take a Break and Talk
about Gordon Ramsay!?!

Class Activity Time

What does this all this Gordon stuff mean?

- It means that being a professional and getting to a certain level take a lot of time and work
- It means that your ability shouldn't give you blanket permission to be a jerk
- It means that many of the techniques and approaches when dealing with skills and teams are pretty simple and work in ANY field
- Ultimately I hope it means that we can depersonalize ourselves and our field to see our fails and our needs so we can actually grow



Gordon the 10x Dev Jerk?

Question #1 - Where are we when we start?

Realistically...where are we?



Coming Soon: The Return of the Apprentice?



Jason Gorman @jasongorman · Aug 19



An apprenticeship for a bricklayer takes roughly 2 years, and requires a big investment in training and supervision. Why any employer would think training software developers requires a smaller investment in time and money is one of life's little mysteries.

 14

 33

 115



Question #2 - What should we focus on?

According to a survey for
top 10 skills for 2025 how
many are technical?

:Let's put our guesses from 1 - 10 on the board

Skills Quiz

This article is part of:

The Jobs Reset Summit

- 50% of all employees will need reskilling by 2025, as adoption of technology increases, according the the World Economic Forum's Future of Jobs Report.
- Critical thinking and problem-solving top the list of skills employers believe will grow in prominence in the next five years.
- Newly emerging this year are skills in self-management such as active learning, resilience, stress tolerance and flexibility.
- Respondents to the Future of Jobs Survey estimate that around 40% of workers will require reskilling of six months or less.

Top 10 skills of 2025



Analytical thinking and innovation



Active learning and learning strategies



Complex problem-solving



Critical thinking and analysis



Creativity, originality and initiative



Leadership and social influence



Technology use, monitoring and control



Technology design and programming



Resilience, stress tolerance and flexibility



Reasoning, problem-solving and ideation

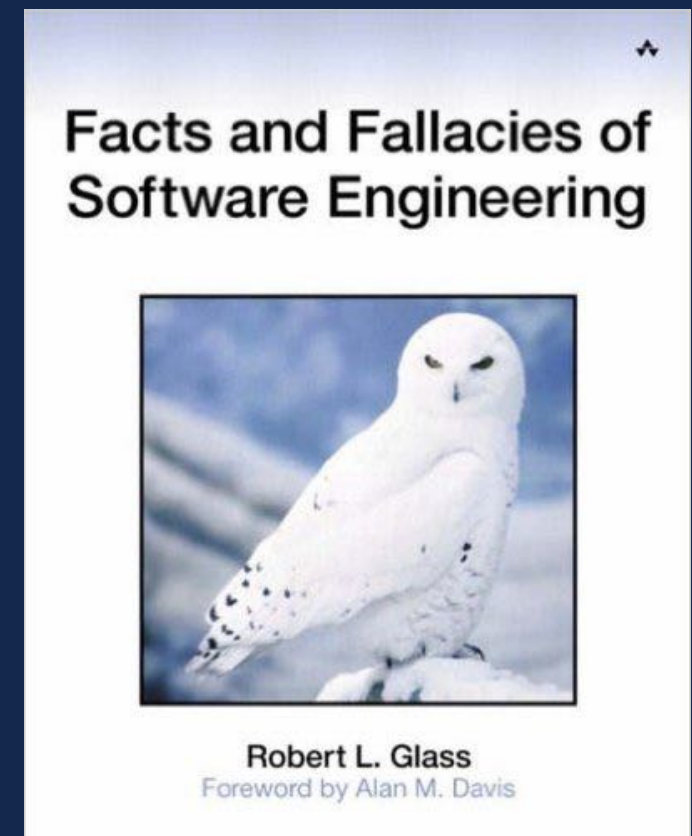
Source: Future of Jobs Report 2020, World Economic Forum.

The most important things to improve a software project outcome

- | | |
|--------------------------|---|
| Programming language | A |
| Schedule | B |
| Tools and Tech | C |
| People and their ability | D |
| None of the above | E |

Fact #1

The most important factor in improving software development is not the tools and techniques used by programmers, but the **quality of the programmers themselves**



Even Social Media, Reddit, etc. Seniors Agree



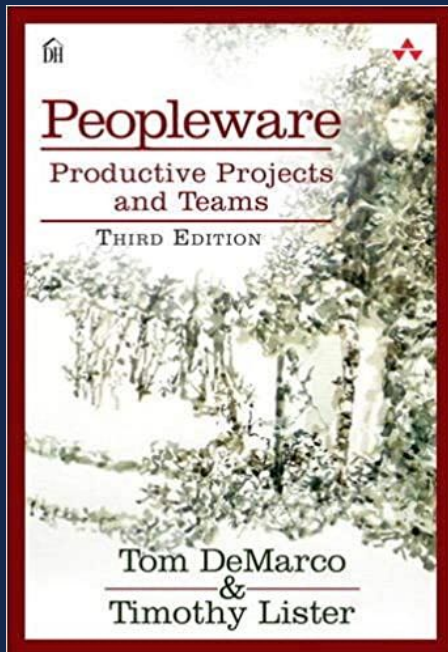
https://twitter.com/hot_town/status/1628487381052424193

NO MATTER WHAT THE PROBLEM IS



IT'S ALWAYS A PEOPLE PROBLEM

Fact #1 Is **NOT** controversial, but common



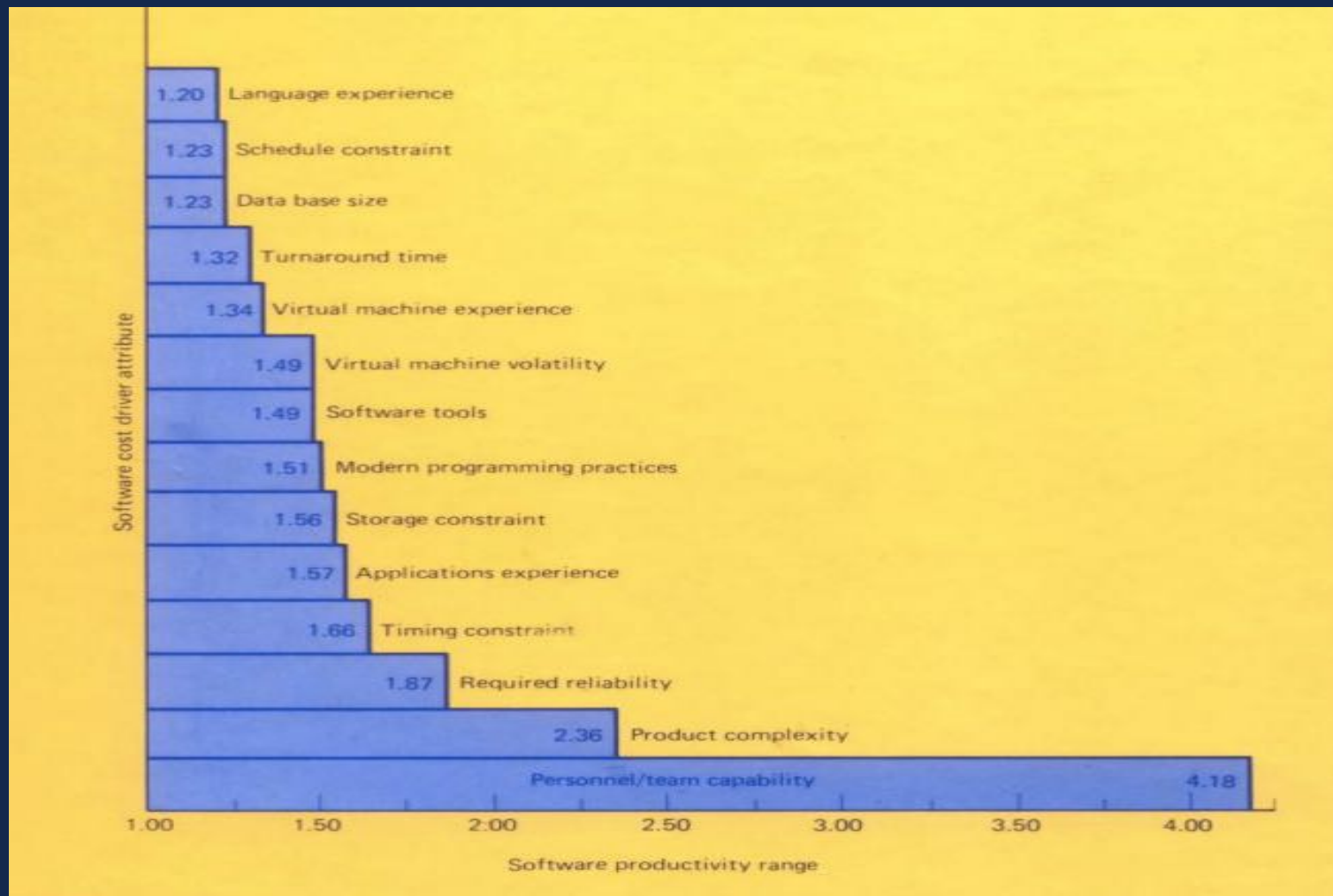
Few books in computing have had as profound an influence on software management as **Peopleware**. The unique insight of this longtime best seller is that the major issues of software development are human, not technical. They're not easy issues; but solve them, and you'll maximize your chances of success.

"**Peopleware** has long been one of my two favorite books on software engineering. Its underlying strength is its base of immense real experience, much of it quantified. Many, many varied projects have been reflected on and distilled; but what we are given is not just lifeless distillate, but vivid examples from which we share the authors' inductions. Their premise is right: most software project problems are sociological, not technological. The insights on team jelling and work environment have changed my thinking and teaching. The third edition adds strength to strength."

— Frederick P. Brooks, Jr., Kenan Professor of Computer Science, University of North Carolina at Chapel Hill, Author of The Mythical Man-Month and The Design of Design

"**Peopleware** is the one book that everyone who runs a software team needs to read and reread once a year. In the quarter century since the first edition appeared, it has become more important, not less, to think about the social and human issues in software development. This is the only way we're going to make more humane, productive

Even Economic Data Supports It



Ok then let's be better programmers

Solution - 10X Programmers!?

“...researchers have found 10-fold differences in productivity and quality between different programmers with the same levels of experience”

http://www.construx.com/10x_Software_Development/Productivity_Variations_Among_Software_Developers_and_Teams_The_Origin_of_10x/

The romantic image of an über-programmer is someone who fires up Emacs, types like a machine gun, and delivers a flawless final product from scratch. A more accurate image would be someone who stares quietly into space for a few minutes and then says “Hmm. I think I’ve seen something like this before.”

<http://www.johndcook.com/blog/2009/12/23/why-programmers-are-not-paid-in-proportion-to-their-productivity/>

Range of Soccer Players



1x

Rando Noob



3.5x

Graham



9.2x

Mia Hamm



10x

Messi

Range of Programmers



1x

Rando Noob



3.5x

Sam Gomez



8.8x

Janet Kennedy



10x

Rockstar Ninja

2x, 10x, or 28x?

- Best programmers are X times better than average/worst programmers what X is
 - X generally isn't small per the experts it seems to range from 2, 10, to 28x or more depending on pundit's degree of hyperbole.
 - Let's just agree it is better to have better programmers
- Prof: "I rather have a coding SEAL team than 50 'hacks' in nearly every situation"
 - What about a brute force job? Maybe not?
 - Think about how the coding "SEAL" would address this?
 - "Code generator, tools, outsource, build vs. buy, etc."

Tips If You Are Following Some “10x” Path

- There is nothing wrong with pursuing the rock star level of ability, just beware of the swagger without the ability.



- Strive for excellence and be self-aware of your current status, it will takes time and effort to get there is all

Avoid One Upping and Be Brave



Ali Spittel   @ASpittel · Jan 11



I hope that the tech industry moves to a point where people can admit to not being geniuses, ninjas, or rockstars and have that be a positive.

I hope that people can acknowledge that there is more to learn, and love that, rather than trying to one-up each other.

 110

 329

 1,976



Why Do Fight Time, Practice and The Truth of Being a “Rockstar” Dev

- We embrace the fallacy because what we do is hard as it is, and we want it easier and quicker
 - Non-crazy Idea: People are not computers with emotions, but instead are emotional beings who try to be logical or rational.
- Tools and languages are predictable, people often seem arbitrary and thus are much harder

Others on 10x Devs

- Watch Dave Farley's take on Rockstar Devs <https://www.youtube.com/watch?v=mVY2rFninp8> (start at 1:30 to skip the commercials)
 - Software development is a team sport. It's not about individual performance it is about team success or failure.
 - A group of middling devs with world class talents tends to beat the "rockstars"
 - "Programming by remote control" and "Please Can I Release?"
 - it's like a soccer coach "joy sticking" his players

The unit of delivery is the team

Why Messi can't win your games for you!

BELBIN

***“The lack of coherent teamwork,
nullified the gains of individual effort
or brilliance”***



Great Teams > Great Players?

- Players jump in to cover for others
 - Futbol: Think about a player that slides over or covers a position as someone attacks

***“Who is on the team matters less than
how the team members interact,
structure their work, and view
their contributions”***

- GOOGLE -

10x Inevitability?

Gresham's Law for Software Dev

- When bad behavior is rewarded the same as good behavior like Gresham's Law of money the bad will take root and push good out.
- In software such bad behavior (10x Rockstars, Bro Culture, Hack and Ship, we are the users, etc.) can in fact have a survival benefit particularly when intertwined with org culture or personal identity
- Example: You get paid & praised the same to write poor performing, insecure, or a11y hostile code so why would you want to change? If later the praise falls away and then I am shamed I might in fact even attack the very premise.

<https://itnext.io/greshams-law-why-bad-developers-push-out-good-developer-and-developers-create-low-quality-code-9b6a8d15bffb>

10x Devs Considered Harmful?



Nina Zakharenko ❤️🇺🇦 @nnja · Aug 11, 2020

every "10x developer" I worked with left behind a wake of 10x destruction behind them

💬 66

↻ 328

❤️ 2,423

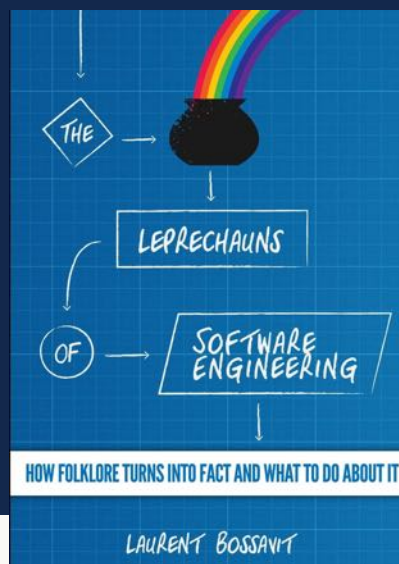


[Show this thread](#)

Also -10x Programmers (NNPPs) Exist

- Net Negative Producing Programmer (NNPP)
 - In short folks who cause harm more than good
- Why is someone an NNPP?
 - Lack of job satisfaction
 - Lack of identification/involvement with org (believe “not on team”)
 - Lack of belief that professional work results in reward (cynicism?)
 - Personal lack of professionalism
- Remediation
 - Ignore (x), Deny (x), Blame Others (x), Assume Responsibility (!) and Find Solution (!)
 - Usually the solution is consul, reassignment/readjust with may include team add, or dismissal
 - <https://wiki.c2.com/?NetNegativeProducingProgrammer#:~:text=%22There%20are%20net%20negative%20producing,than%20adding%20a%20good%20one.%22>
 - http://web.archive.org/web/20011023084845/http://pyxisinc.com/pyxis_artmain.html

10x-ers are Leprechauns!?!



Broader Implications

Bossavit's work extends beyond just debunking the 10x developer myth:

The value in Bossavit's approach lies not just in his conclusion that "the 10x programmer doesn't exist," but in the critical thinking methods he demonstrates to analyze claims in software engineering. Mark Greville

This case study illustrates a larger pattern where dubious claims become accepted "facts" through repetition rather than evidence, suggesting a need for greater scientific rigor in software engineering research and practice.



<https://markgreville.ie/2025/03/17/book-review-leprechauns-of-software-engineering/>

A Reasonable Conclusion of the Segment

- Yes there are people quite a better than others (in any space)
- Obviously you'd want better people to work with
 - ... unless those folks ruined the team with their nonsense behavior
- If you are a better one don't be a jerk
- Careful with factualizing much of what is going on tech without heavy skepticism. Be skeptical, check your priors, chase references, etc.
- Over time I'll bet your lived experience will run into beliefs. Trust yourself.

Fact #5 Supports Fact #1

Software tool and technique improvements account for at most 5-35% percent increase in productivity or quality, not the order of magnitude benefits promised

- Careful: Online grifters tend to have underlying reasons for their hype
 - Consulting, books, \$ attraction (VC), naiveté and wishful thinking
 - Sometimes it is a hyperbolic means to an end
 - Common marketing approach to fixing a market is to make a new market
 - Can't beat the competition or already lost to them
 - Rename/rebrand existing idea and claim it is all new
- Be careful there is a lot of magic fix solutions to productivity and coding out there

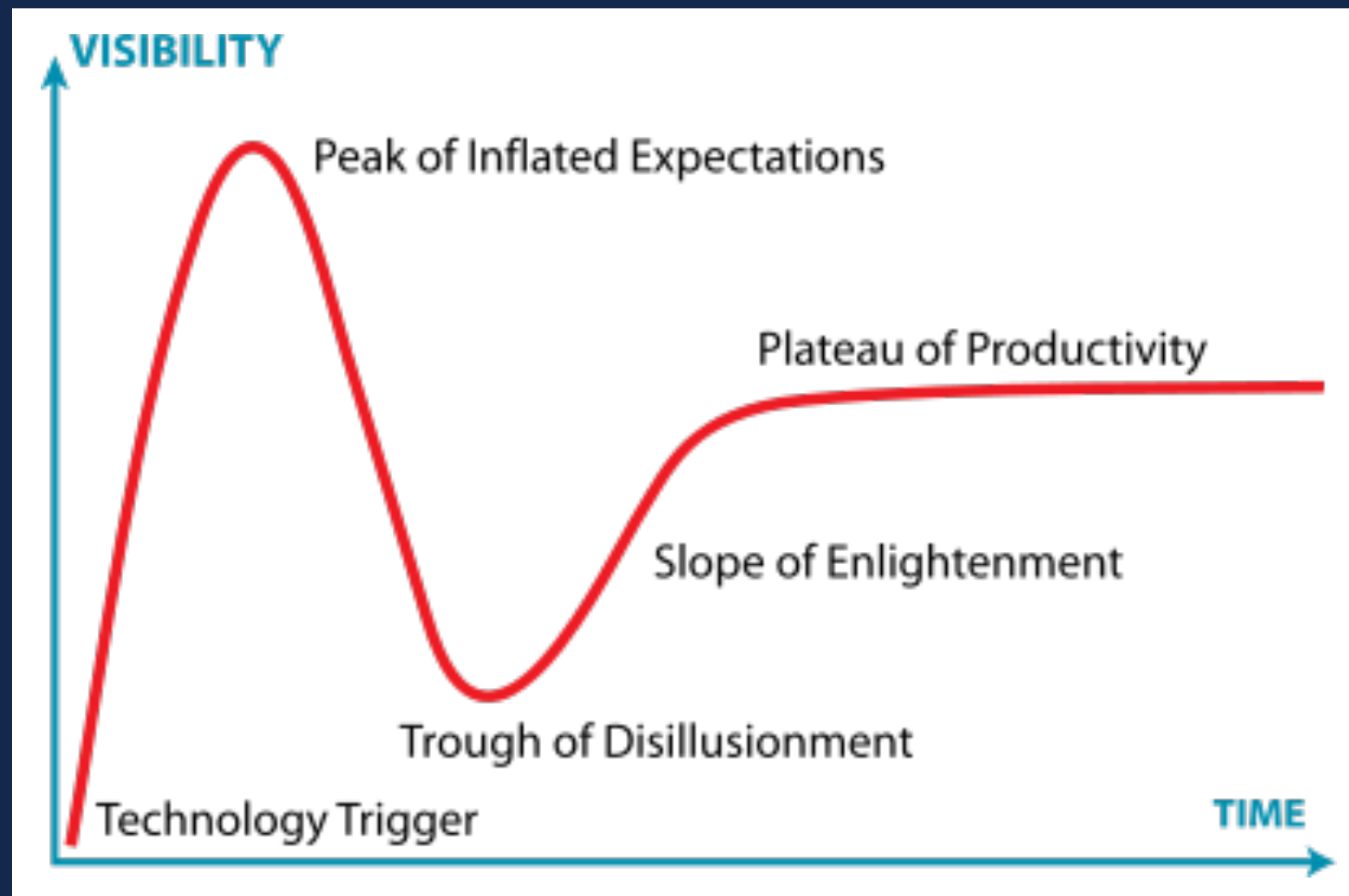
And we **want** to believe ... 3min Full Stack!!!!



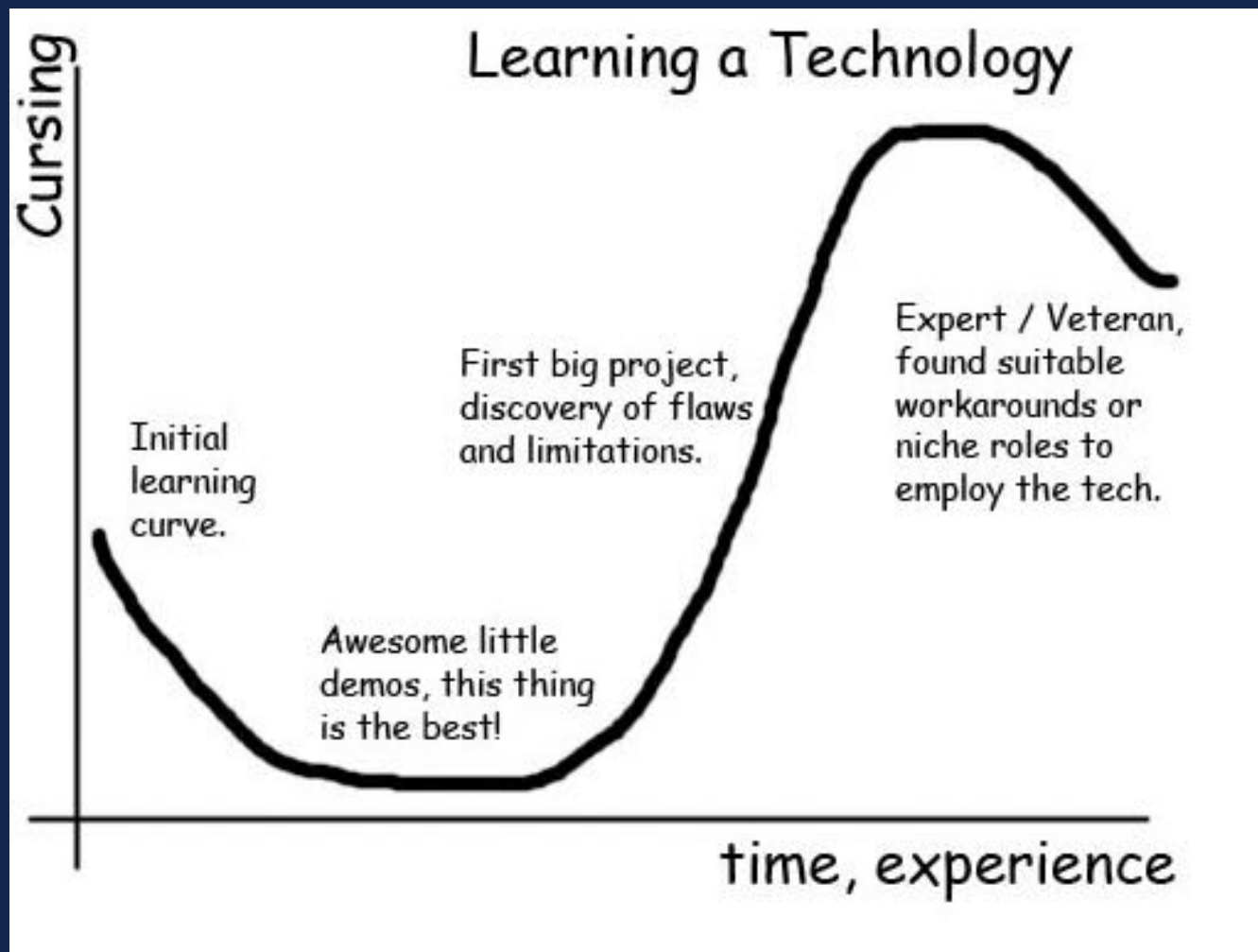
Fact #6 isn't acknowledged enough

- Learning a new tool, language, library, technique, etc. **actually lowers productivity and quality initially. The gains come only later**
 - Class Factoid - This reality means RDD tool/tech fetishing adds risk to our project, though to be fair we might get what we want?
- My Related Fact: There is always another tool! Because of this fact it is easy to always be chasing the next best thing!
- We talk a lot about tools, evaluate some, adopt/buy a few, initially use some, and very rarely adopt many long term!
 - Takeaway - Plan that cost and time appropriately

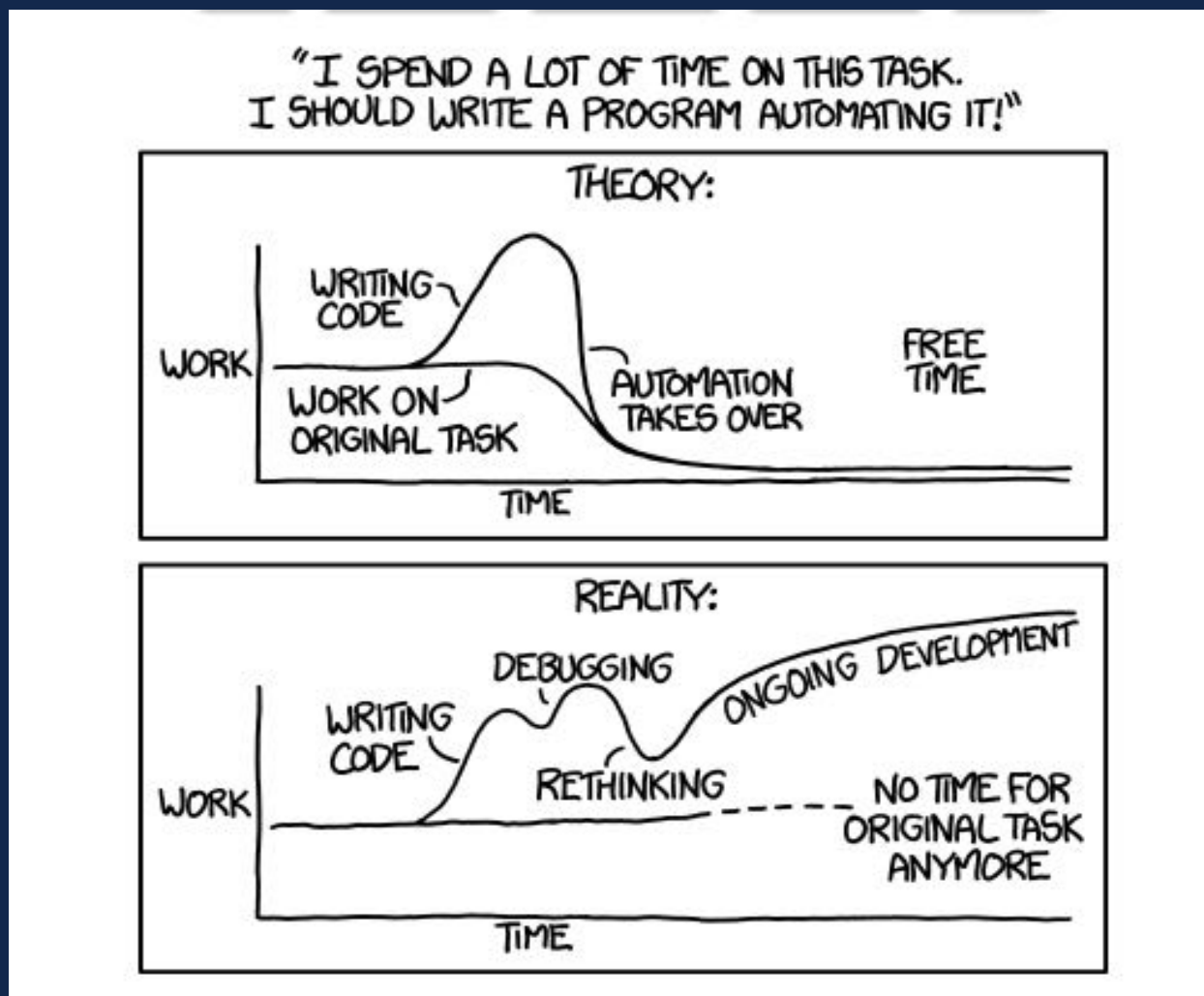
Again From the “Joke” Part of Lecture 1



A “Joke” for Lecture 2



Sadly, None of this is a Joke

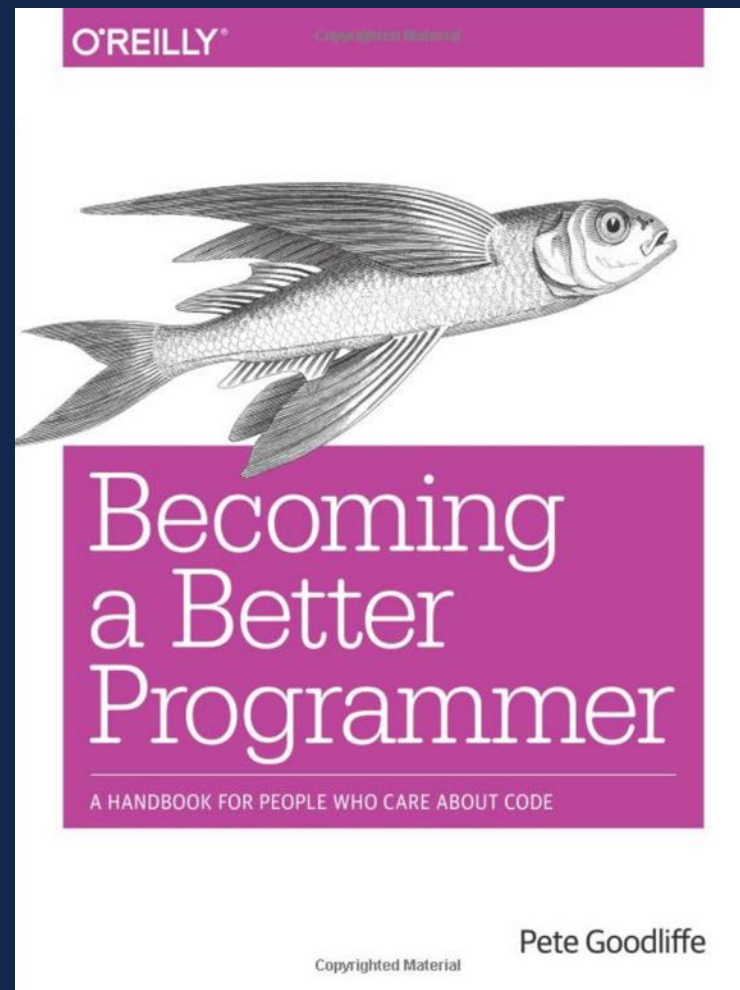


Programmer Master Thyself

- “When all is said and done, the ultimate factor in software productivity is the capability of the individual practitioner”
 - If this is true, then our goal is to make the individual practitioners better right!?
- Unfortunately as in life in general it is quite difficult to control or even influence the improvement others
- On a positive note we have a fighting chance with ourselves so we really should start there!

So if we are buying the idea now...

Just read this right? No live it...



Warning: This is NOT
meme this and many
other books are quite
real and since I've
taught SE many books
on this topic have been
written when few were
around early on

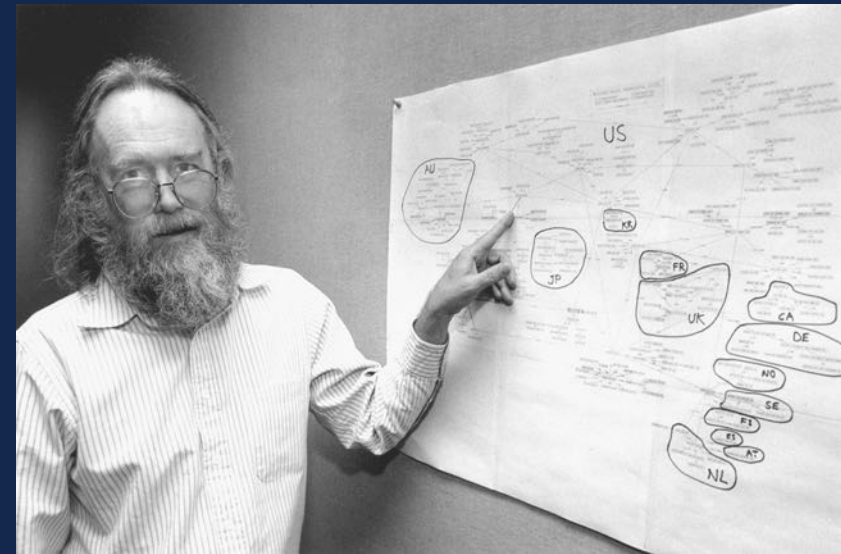
What I can control and what I can't

Data source: @mindfulenough | Infographic design by @agrassoblog for educational and motivational purposes



Postel's Law for Yourself

- Robustness principle
 - “*Be conservative in what you do, be liberal in what you accept from others* (often reworded as “Be conservative in what you send, be liberal in what you accept”).
- Roughly applied to our effort you can control what you do and shouldn't contribute to the “mess” but need to be flexible with what others do since they might give you “mess”
- This seems to be a golden rule far beyond computing!



Smarter People Than Me Believe This

A broken umbrella lies on its side in a field of tall grass under a cloudy sky. The umbrella is dark-colored and its frame is bent. The background shows a vast, open landscape with rolling hills or fields stretching towards the horizon.

“Watch your thoughts; they become words.

Watch your words; they become actions.

Watch your actions; they become habit.

Watch your habits; they become character.

Watch your character; it becomes your **destiny.**”

~Lao Tzu

Attitude First

You must want to be a better programmer

It will take time and effort to become a better programmer

Like any thing, to do good X you have to care about X

To write good code you have to care about it.

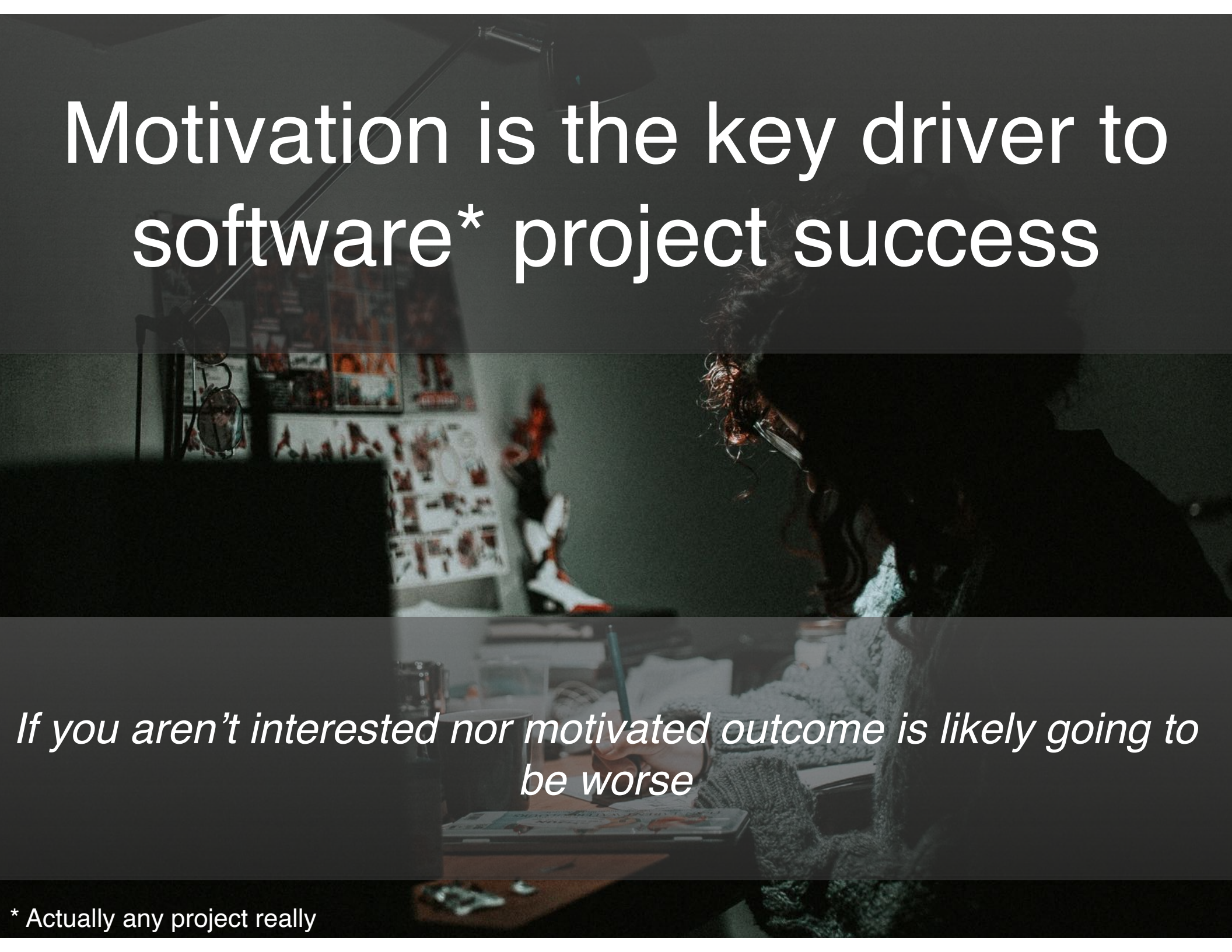
So being emotional about code is indeed valid

Disgust and pride are reasonable reactions to code

However, player $\neq$ game, do not make YOU $===$ CODE mistake

Spoiler Alert: Is the assignment “your purpose statement” becoming obvious ? It should be now!

Motivation is the key driver to software* project success

A person with curly hair and glasses is seen from the side, working at a desk. A desk lamp is positioned over their work area. In the background, a bulletin board is covered with various papers and photos. The scene is dimly lit, with the primary light source being the desk lamp.

If you aren't interested nor motivated outcome is likely going to be worse

* Actually any project really

Selfish Programmers

- Selfish programmers think of themselves more than others
- Selfish programmers take more pride in their tools, materials, processes, and other aspects of the endeavor personal to them as opposed to their users
- Selfishness as I describe it here boils down to why we are doing something, is it solely for ourselves or in service to others as well?

Note: If coding for ourselves we aren't being selfish at all. Though If you code commercially you are not in fact building for yourself. Selfish choices made in this context could be considered insubordinate if understood by your employer, be very cautious with your decisions.

Selfishness versus Our Responsibilities to Others

“What kind of world do we want to live in?” Princeton computer scientist Arvind Narayanan argues, “It’s not enough to ask if code executes correctly. We also need to ask if it makes society better or worse.”

Data Violence and How Bad Engineeri...

Anna Lauren Hoffmann

Don't Mistake This Last Point - You Need Pride!

- Appropriate pride of workmanship and result is required of true professionals, both for their own work and the appreciations of others.
- Healthy pride should come more from outcomes as opposed to materials, tools, or procedure.
- Excessive pride in effort or result like the ancient maxims suggest is dangerous and may lead to ruin.

Software as a Service Philosophy

- Software as a service is technically and economically a description of a way software is delivered to its users less as a product owned but a service hired.
- This definition is fine but we also should think of the idea of software as a service or **in service to others**.
- In some ways we might call adherents to this philosophy “selfless programmers,” but personally I think it is always a balance we should maintain and engineers respect balance and trade-offs inherently.



Q: How to make a better programmer?

A: Practice!

- The adage practice makes perfect is clearly not a programming specific idea
 - Example: I want to play the guitar
 - Requirement: Lots of time, need to practice, play chords, play songs badly many times, etc.

Q: How to make a better programmer?

A: Practice!

- If applied to the act of creating ideas like coding it suggests you need to write lots of bad code before you can write a little great code
 - Long held in writing - the “shitty first draft”
 - Don’t forget the rewrite (or in our case refactor!)

10,000 hours then!?

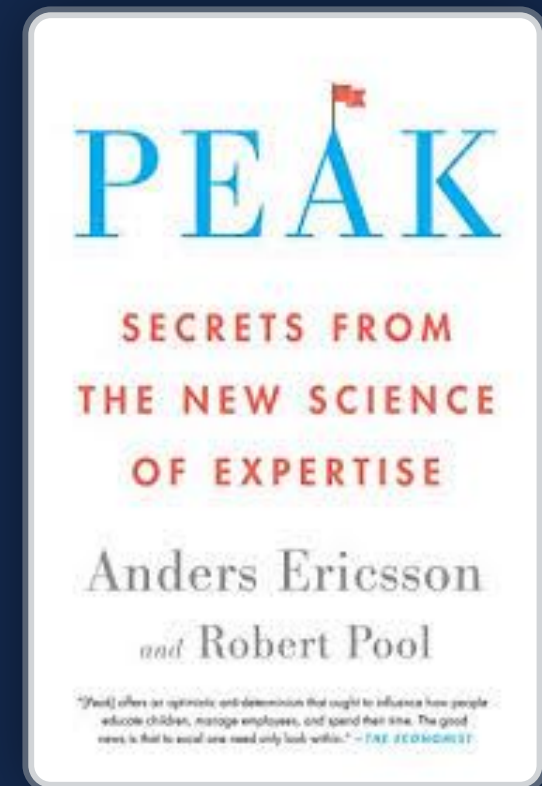
<http://www.wisdomgroup.com/blog/10000-hours-of-practice/>

Or not

<http://www.businessinsider.com/new-study-destroys-malcolm-gladwells-10000-rule-2014-7>

The Need for Practice

- ‘Generally once a person reaches that level of “acceptable” performance and automaticity, the additional years of “practice” don’t lead to improvement. If anything ... abilities generally deteriorate in the absence of deliberate efforts to improve
 - Think Driving
 - The UCSD infamous Rick Ord has an opinion on this!



Tested? Practice makes perfect better

In his book *Art&Fear*, David Bayles shares the story of a ceramics teacher experiment.

“The ceramics teacher announced on opening day that he was dividing the class into two groups. All those on the left side of the studio, he said, would be graded solely on the quantity of work they produces, all those on the right solely on its quality. His procedure was simple: on the final day of class he would bring in his bathroom scales and weigh the work of the “quantity” group (...) Those being graded on “quality”, however needed to produce only one pot — albeit a perfect one — to get an A.

The interesting part of the story comes in the assessment of the outcomes.

Well, came grading time and a curious fact emerged: the works of highest quality were all produced by the group being graded for quantity. It seems that while the “quantity” group was busily churning out piles of work-and learning from their mistakes- the “quality” group had sat theorizing about perfection, and in the end had little more to show for their efforts than grandiose theories and a pile of dead clay.”

“...the works of highest quality were all produced by the group being graded for quantity.”

Not any kind of practice

- Purposeful practice which has well-defined, specific goals
- 3Fs Required
 1. Focus
 2. Feedback - positive focused feedback
 3. Failure – just beyond one's comfort zone
- It won't be easy. You'll hit plateaus. And guess what there may be no limits?

POP: Practice Often and with Purpose

- Beyond SE we see that practice is paramount
- In SE: Big question do we actually practice?
- Practice Plan
 - Find what you are about broadly, figure out what you are needing to do, and then and only then dig in deeply
 - Repeat until goal state reached except when you need to change
 - For improvement persistence is great assuming you are doing the right thing. We need to know when to quit or change practice

So if we are following some training idea...

6 Brogrammer Insights!?! A man with short dark hair, wearing a bright red polo shirt and black sunglasses, is sitting at a desk in an office. He is looking towards the camera with a serious expression. His right hand is raised, showing a hand gesture with the index and middle fingers extended. Behind him are several computer monitors displaying code or data. The background is a plain office wall.

“Brogrammers” so far have gotten very very few things right, but caring and being aware of our bodies as an important part of a software engineer’s optimality equation is maybe the one thing they have positively added to our profession.

Train the Brain?

- We all believe in some sense that you can improve the body, be it diet, lifting weights, etc. What about the brain?
 - Of course! In fact we can see in studies of brains of those undergoing long term training changes in brain structures which appear to be relevant or related to the skill being developed
- By design the human body is tuned to achieve homeostasis
- But you can adjust what that homeostasis is, and the body will regain homeostasis over time

See a Memory Athlete!?!



<https://well.blogs.nytimes.com/2014/05/19/remembering-as-an-extreme-sport/>

Yeah but ours is not a memory game



Brad Traversy @traversymedia · Jan 5



You don't have to have a great memory to write code. Focus more on having an overall understanding of what you're doing and why it works (or doesn't). You can always look up syntax and specifics.

Remembering syntax without a clue of what is happening is useless.



Memory - sure it's important, but understanding and habits usually far outweigh it!

To Optimize Here Is...

- Somewhat like what the athletes or frankly realistic people would do
- You'd work on your abilities, you'd train, you'd worry about things like
 - Sleep
 - Energy
 - Focus
 - Health
 - Emotional state
- Right now that is tough, clearly! ☹
 - Yet rough situations can certainly be an effective teacher

Yes “Good Gear” Also Matters

We Avoid This and Often Focus on Gear

As a coder your “instrument” is made up of your coding tools

- Computer
- Monitor(s)
- Keyboard
- OS
- Editor/IDE
- Language
- Framework
- Misc. Tools – Debuggers, Build Tools, Syntax Analyzers, etc.*
 - * Hesitate to break out since they are often part or heavily dictated by the IDE, framework

We shouldn't, but if are so focused let's go there and get it right!

Meet “*Trigger*”...Do you have one?



[https://en.wikipedia.org/wiki/Trigger_\(guitar\)](https://en.wikipedia.org/wiki/Trigger_(guitar))

<https://www.youtube.com/watch?v=b6IB0trJoJU>

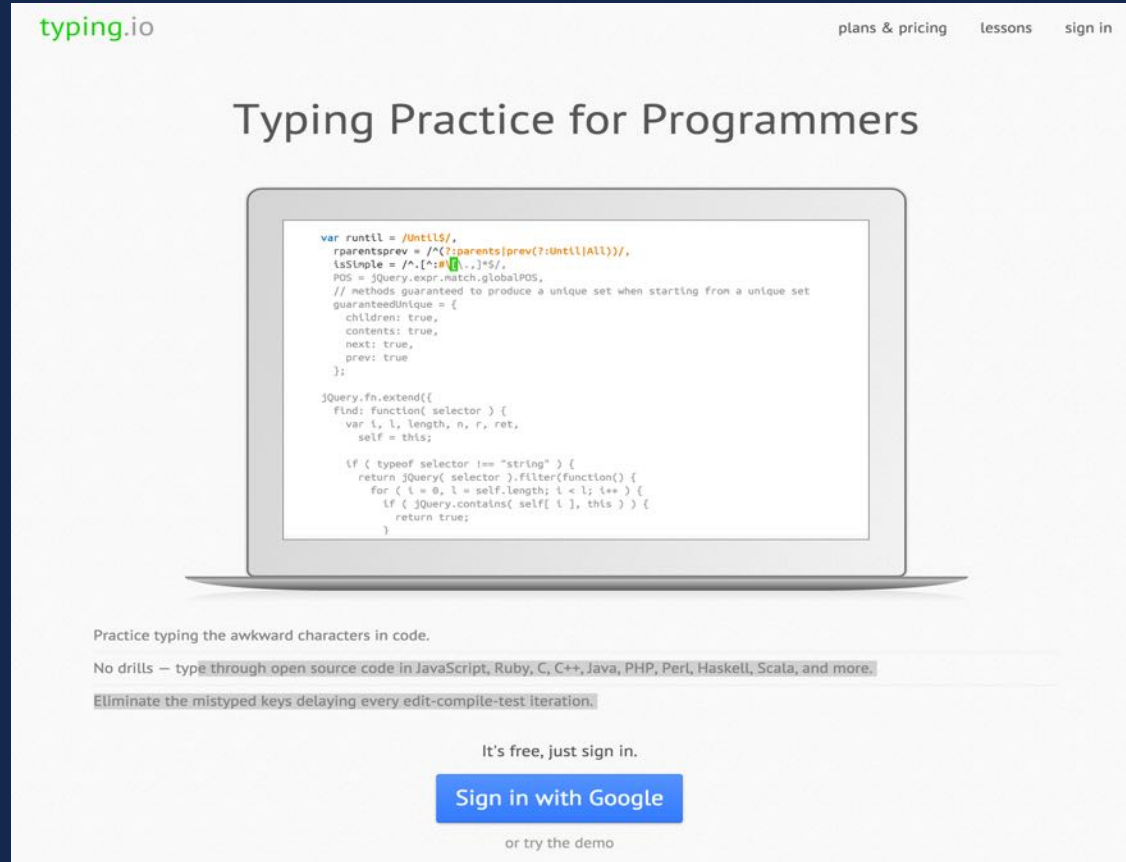
Gear Up

- Computer – buy the best system you can always
 - I know we are students but really this is like being a musician please buy a good instrument!
- Focus on I/O
 - Disk no, SSD yes
 - Multiple monitors or single wide monitor
 - Avoid cheap pointing devices
 - Keyboard (see dedicated slide)
- Connectivity
 - AirCards, WiFi access plans, VPN software if you must use public access points, fiber if you can get it

“Playing” aka Typing

- Being a code programmer requires you to be a good (fast and accurate) typist
<http://blog.codinghorror.com/we-are-typists-first-programmers-second/>
- Given the importance of typing
 - Get a good keyboard (see next slide)
 - Learn to type
 - Try to keep your hands on the keyboard at all times
 - Translation: Learn key commands, expanders, and build finger muscle memory
 - Window management, text expansion examples
 - Skeptical but strong opinions also surround possibility of changing to Dvorak keyboard
https://en.wikipedia.org/wiki/Dvorak_Simplified_Keyboard#Comparison_of_the_QWERTY_and_Dvorak_layouts

Interesting Service?



The screenshot shows the homepage of typing.io. At the top left is the logo 'typing.io'. At the top right are links for 'plans & pricing', 'lessons', and 'sign in'. The main heading is 'Typing Practice for Programmers'. Below this is a laptop graphic displaying a code editor with JavaScript code. The code includes regular expressions for finding parents and simple words, and a jQuery plugin for finding elements. Below the laptop, there is text describing the service: 'Practice typing the awkward characters in code.', 'No drills — type through open source code in JavaScript, Ruby, C, C++, Java, PHP, Perl, Haskell, Scala, and more.', and 'Eliminate the mistyped keys delaying every edit-compile-test iteration.' At the bottom, there is a call to action 'It's free, just sign in.' with a 'Sign in with Google' button and a link 'or try the demo'.

typing.io plans & pricing lessons sign in

Typing Practice for Programmers

```
var runtil = /Until$/,
    rparentsprev = /^(?:parents|prev(?:Until|All))/,
    isSimple = /^.[^:#\[\.,]*$/;
POS = jQuery.expr.match.globalPOS;
// methods guaranteed to produce a unique set when starting from a unique set
guaranteedUnique = {
  children: true,
  contents: true,
  next: true,
  prev: true
};

jQuery.fn.extend({
  find: function( selector ) {
    var i, l, length, n, r, ret,
        self = this;

    if ( typeof selector !== "string" ) {
      return jQuery( selector ).filter(function() {
        for ( i = 0, l = self.length; i < l; i++ ) {
          if ( jQuery.contains( self[ i ], this ) ) {
            return true;
          }
        }
      });
    }
  }
});
```

Practice typing the awkward characters in code.

No drills — type through open source code in JavaScript, Ruby, C, C++, Java, PHP, Perl, Haskell, Scala, and more.

Eliminate the mistyped keys delaying every edit-compile-test iteration.

It's free, just sign in.

[Sign in with Google](#)

[or try the demo](#)

<https://typing.io/>

Good Keyboards

- A mechanical keyboard is widely thought as the way to go
 - <http://blog.codinghorror.com/the-keyboard-cult/>
 - <http://www.tested.com/tech/accessories/460198-you-should-use-mechanical-keyboard/>
 - <https://codekeyboards.com/>
 - <https://www.daskeyboard.com/>
 - Check e-Bay to see how much vintage mechanicals still fetch

Or is it Mechanical Madness? :-)



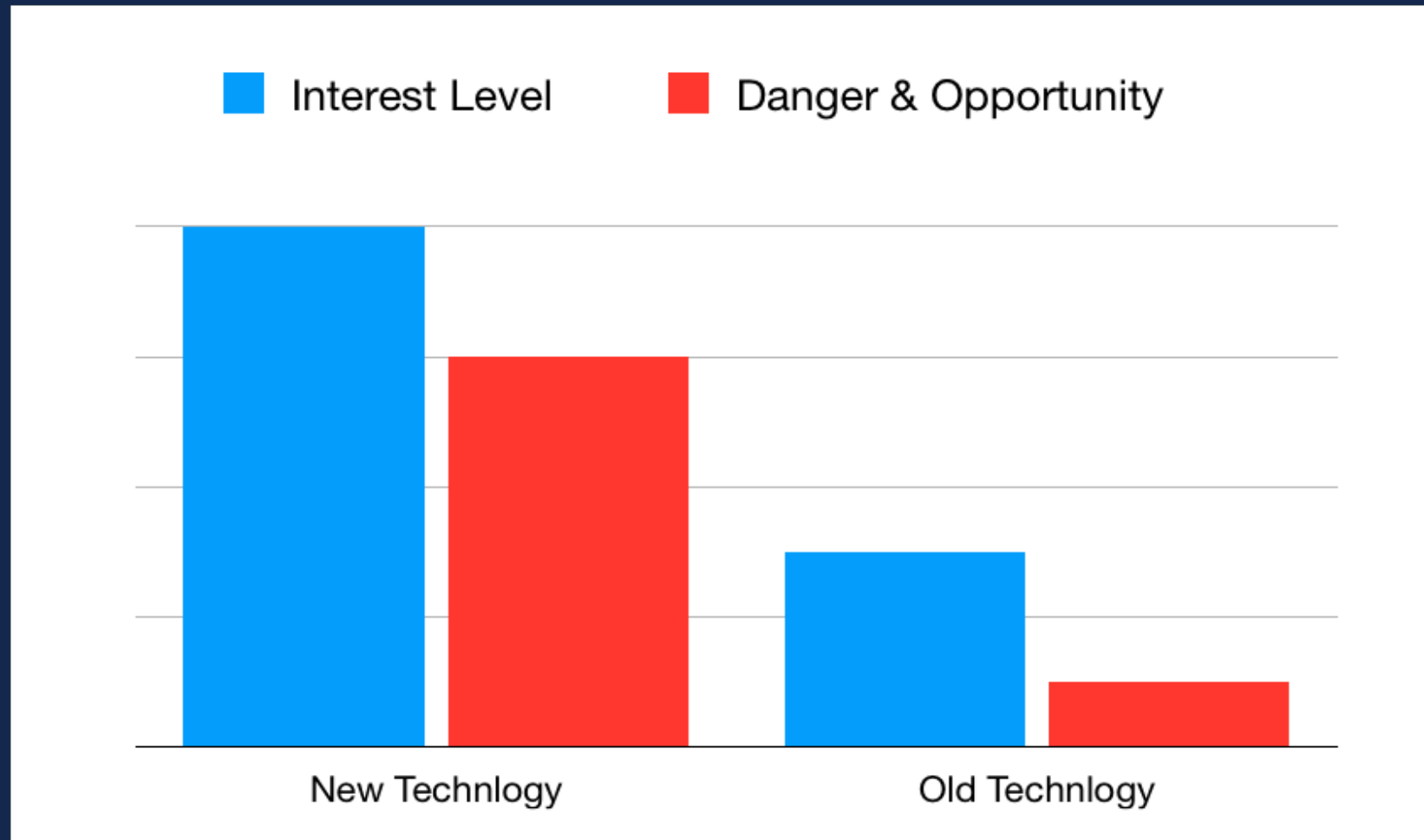
Ok let's not get carried away here...



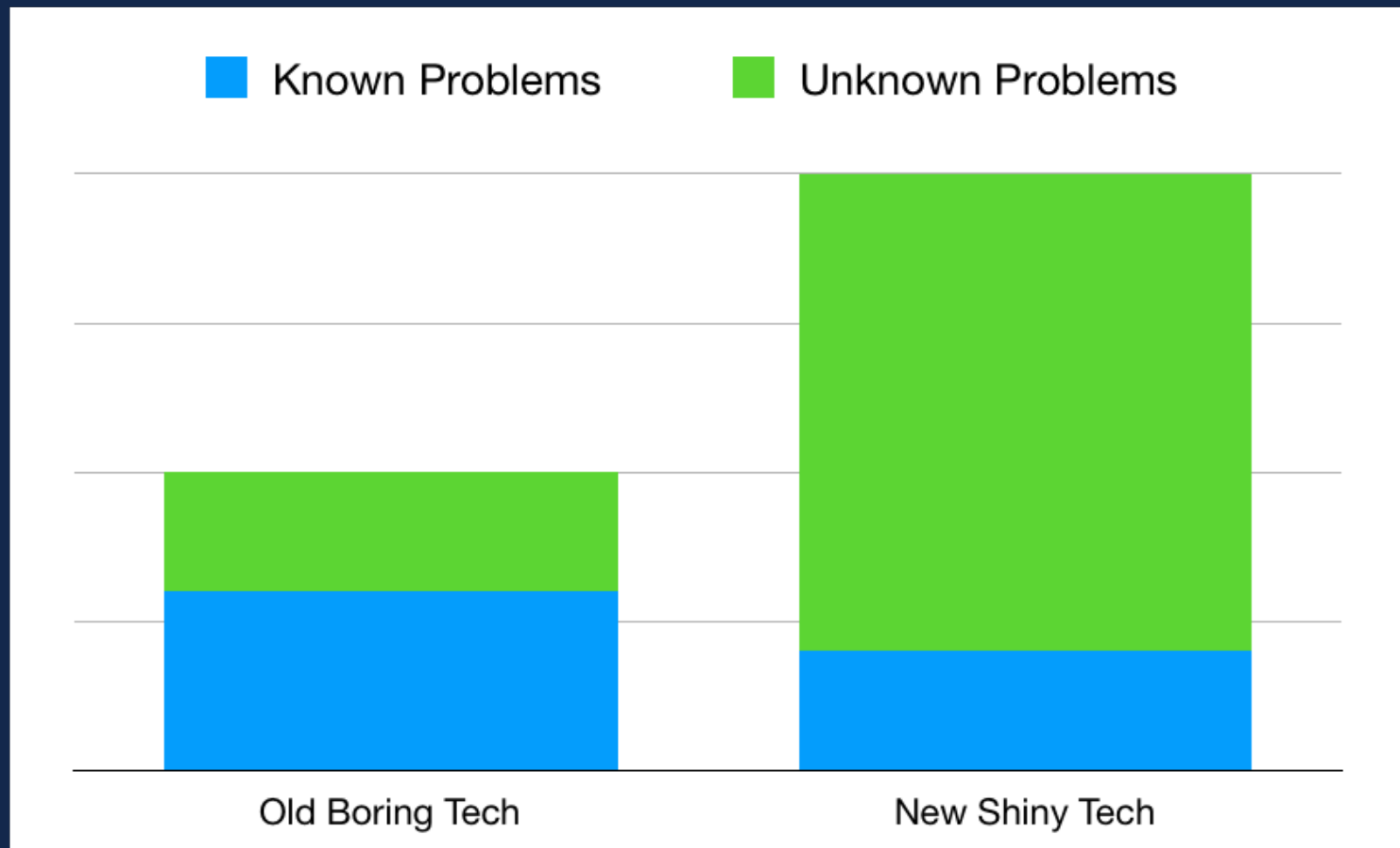
Tools

- We should certainly select good tools
- Rule: A good _____ does best with good tools, but good tools don't make good _____s.
- We should not be completely resistant to the evaluation of new tools after we mastered a tool though
- Tool analysis of course has to happen with Fact #5 and #6 firmly held
- Tool analysis also should be in light of specific need as opposed to generalized solution
- The newness or popularity of a tool is a far from perfect indicator of the value or efficacy of the tool

New & Old Tools/Tech Patterns?



Even though...

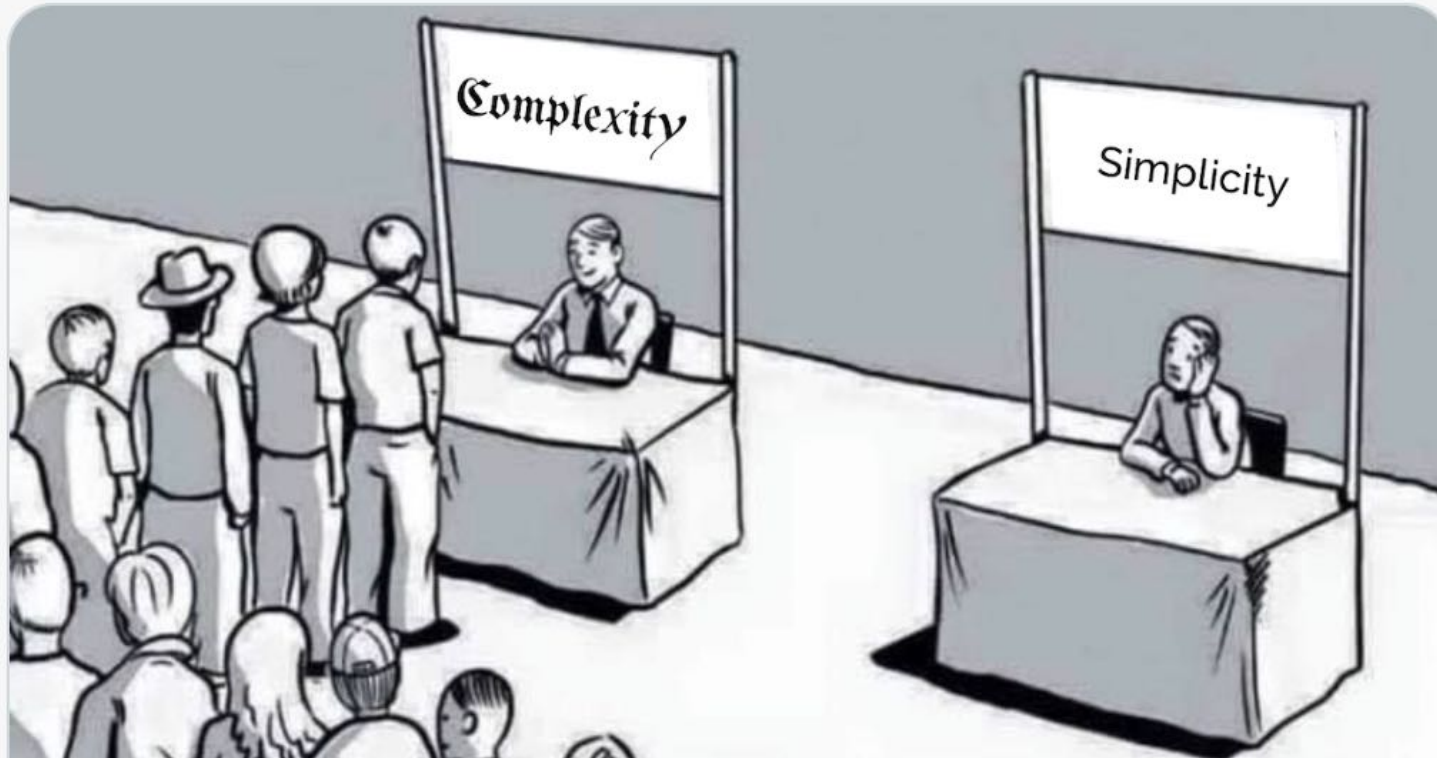


Gratification Problem?

- Seems almost as if we have some delay of gratification challenges here
 - Some influenced by those who would sell us stuff
 - Some influenced by our clear self-interest...get real we do need to stay relevant, so the crowds and their interests matter
 - Some pressure might be related at times to boredom or novelty
 - Some
- I really wonder if we have a bit of the “Instant Cake” fable which while not quite true is quite instructive to today’s CLI fetishing
 - <https://www.bonappetit.com/entertaining-style/pop-culture/article/cake-mix-history>

Now Apply Double Damage!
Not a true Scotsman Engineer

The Psychological Weakness of Simple



eugeneyan.com

Simplicity is An Advantage but Sadly Complexity Sells Better

Pushing back on the cult of complexity.

<https://eugeneyan.com/writing/simplicity/>

Simple Solutions Considered Harmful?



Bryan Liles @bryanl · Aug 5



Simple solutions are easier to implement and scale than complex solutions. However promotions often are rewarded to those who create complex solutions. We need to start acknowledging the power of simple. Don't accept complex. It's a trap.



See we often do have reasons, some psych and some organizational. Much of this is much broader than SE only and is talked about in various books like Subtract (by Leidy Klutz)

Applied Example: Magic Spoons, Tool Belts and Tool Boxes

- Given the programmer tendency to look for general solutions we see in industry an over emphasis on generalized tools or a search for the “one true tool”
 - Silver Bullets
 - LoTR Fixations
- Tools have trade-offs like anything else, best to have the correct tool for the job than a single tool for all jobs
 - Must acknowledge that too many tools leads to being good at nothing both because of lack of heavy use or tool compromise
- My pragmatic metaphor is the toolbox or tool belt then with some ‘go to’ tools within



Specific Example: Editors

- Compare to hammers - there is a range
 - Finishing, General/Claw, Framing, etc.
- Simple Text Editor
 - vi, emacs, Sublime, etc.
- Light Weight IDE
 - Coda
- Full IDE
 - Visual Studio, VSCode, IntelliJ (WebStorm), Eclipse
- Emphasizing
 - Mouse or Keyboard
 - Mode or no mode
 - Manual or Auto(complete)
 - Single Experience or Multi-Windowed/App Experienced

Careful - Whatever Becomes “Your *Trigger*” It Doesn’t Work Without The Skilled Performer



Gear Won't Solve the “Player Problem”

Back to the big one!

- You!
 - Your brain
 - Your focus
 - Your energy
 - Your habits
- A large mistake being made by many programmers is ignoring the biologically and emotionally imposed limits of your abilities

Secret to Software Success



+

```
for i in people.data.users:
    response = client.api.statuses.user_timeline.get(screen_name=i.screen_name)
    print 'Got', len(response.data), 'tweets from', i.screen_name
    if len(response.data) != 0:
        ldate = response.data[0]['created_at']
        ldate2 = datetime.strptime(ldate, '%a %b %d %H:%M:%S +0000 %Y')
        today = datetime.now()
        howlong = (today - ldate2).days
        if howlong < daywindow:
            print i.screen_name, 'has tweeted in the past', daywindow,
            totaltweets += len(response.data)
            for j in response.data:
                if j.entities.urls:
                    for k in j.entities.urls:
                        newurl = k['expanded_url']
                        urlset.add(newurl, j.user.screen_name)
        else:
            print i.screen_name, 'has not tweeted in the past', daywindow
```

=

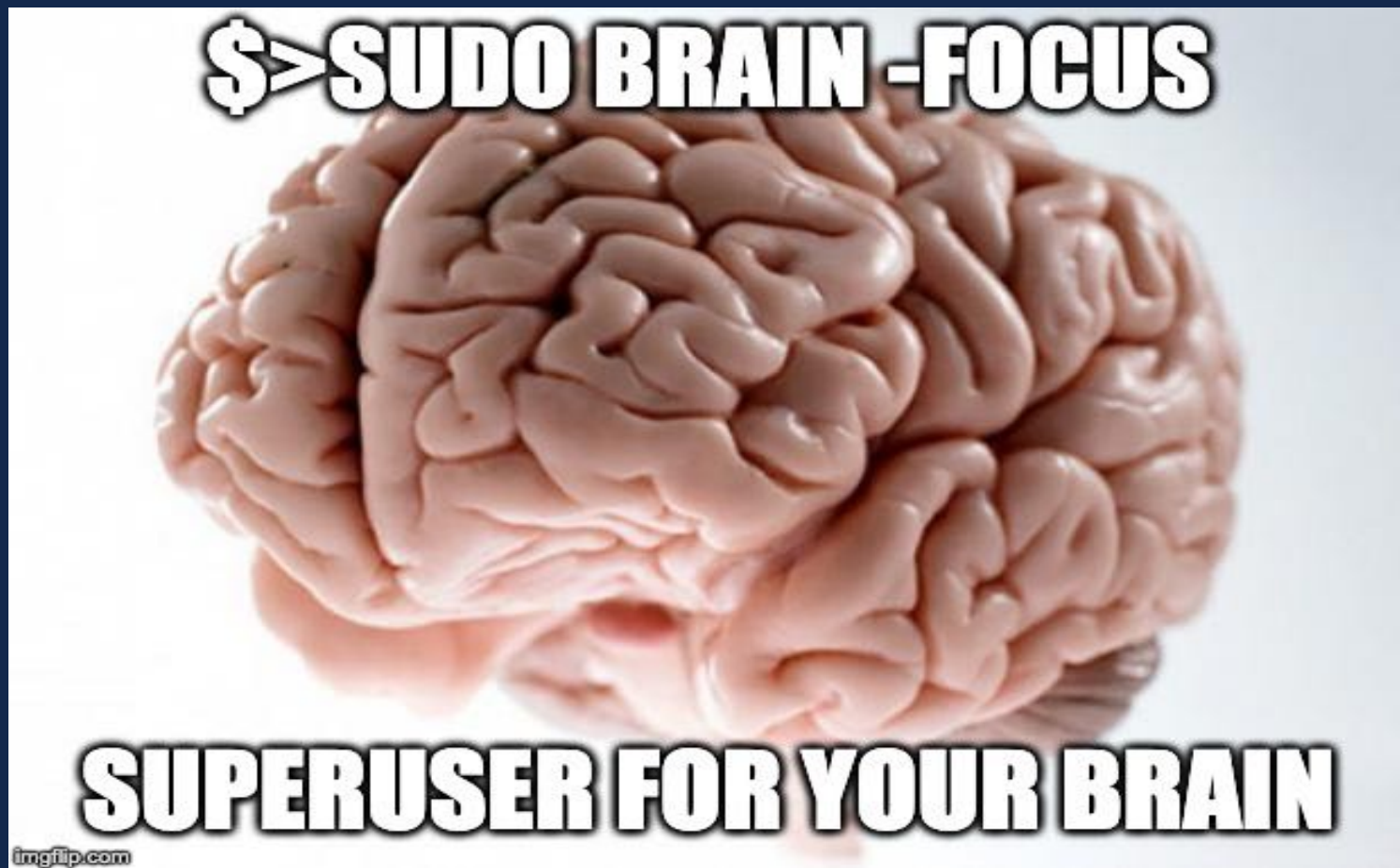


9-9-6 RIGHT?

Actually...

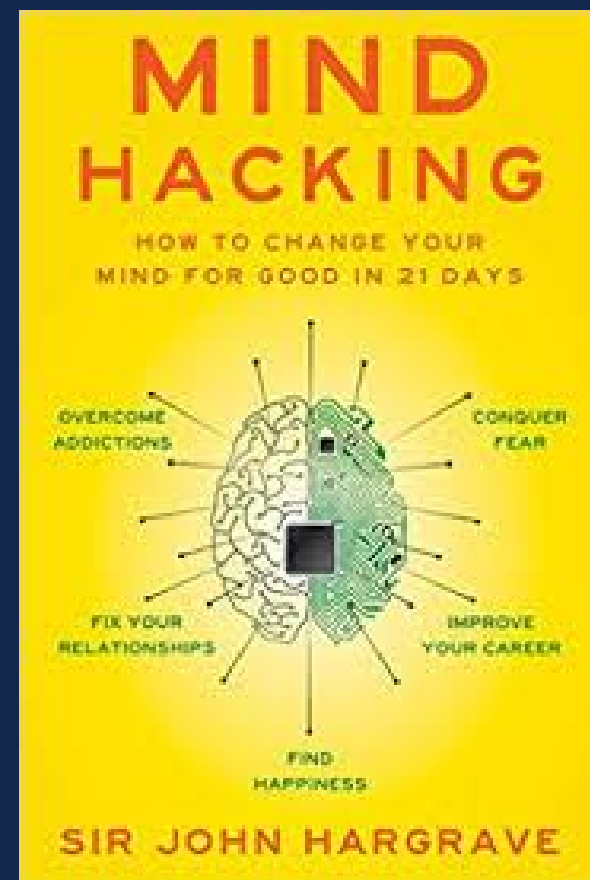
- This is not reasonable in many ways
 - Body and health would of course effect brain
 - We'd have to think about restfulness, moods, etc.
 - Pushing ourselves like that just isn't going to work, at least not over the long haul.
 - We'll not get into this now for time reasons, maybe later
- As we have limited time together so let's focus on the brain and what we can do to optimize it or be aware of things that will directly effect our thinking and code productivity

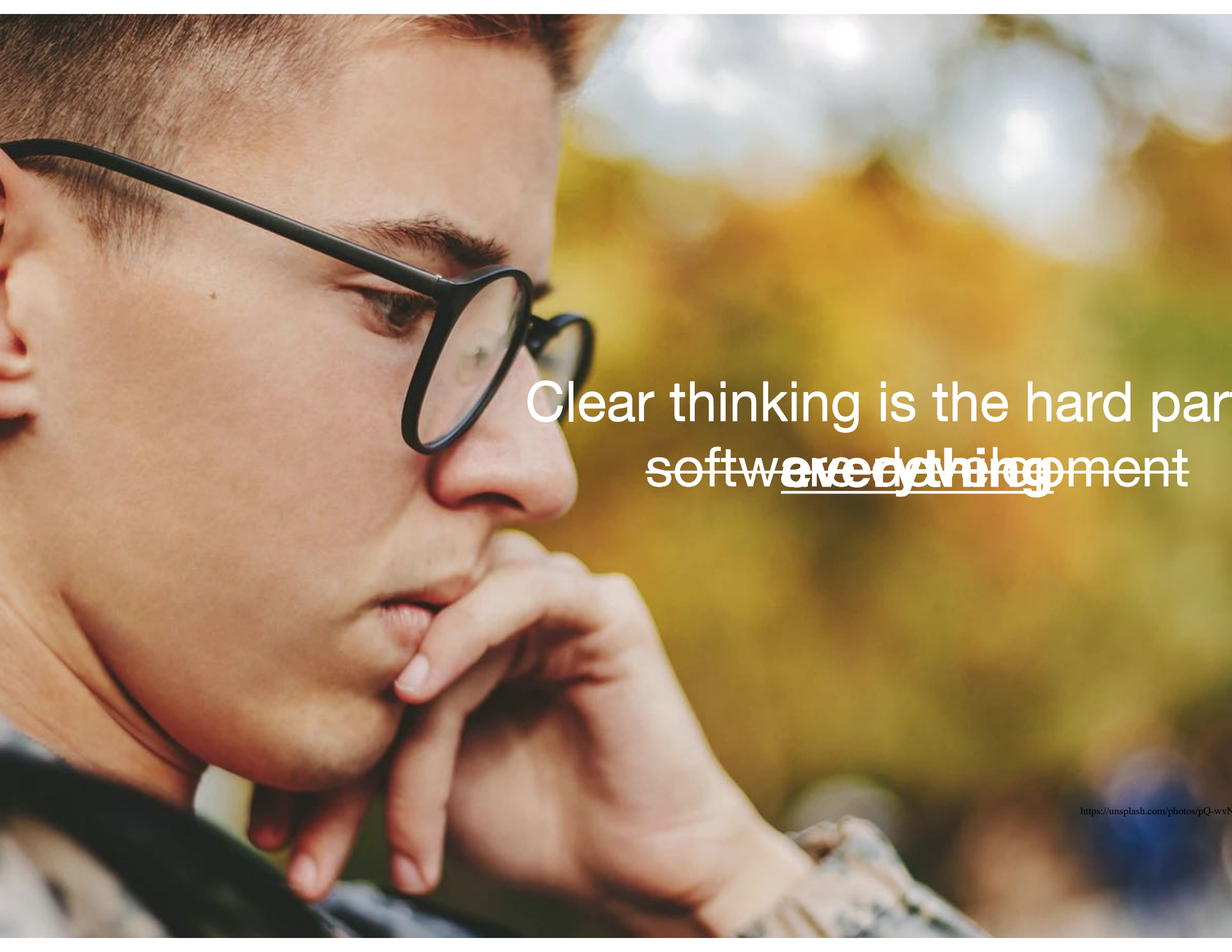
Do you have root access?



Should we hack it?

- The trick is becoming conscious of when you're in control of the mind (superuser mode) and when you're lost in the mind (user mode).
- Three Parts to the Hack
 - Analyzing
 - Imagining
 - Reprogramming
- Metacognition - think *about* our thinking
 - We CS types are often good at meta which is both good and quite bad at times



A close-up, profile view of a young man with short brown hair and black-rimmed glasses. He is resting his chin on his right hand, looking off to the side with a thoughtful expression. The background is a soft-focus outdoor scene with yellow and green foliage under a bright sky.

Clear thinking is the hard part
of software development everything

The Character of Mind

- At times, our brains are like misbehaving dogs
 - Positively though dogs can be trained!
- When are minds are like misbehaving dogs.
 - Technology toys are like squirrels in our front yards
 - Your mind craves information, that's what it eats.
 - Unfortunately, your mind has bulimia
- It takes attention to make attention, so we must invest attention to create even more of it.
- Not new at all.
 - William James - "...the skill of voluntary bringing back a wandering attention, over and over again, is the very root of judgement, character, and will."

Two Minds

- Top Down – Higher Level
 - Powerful and allows us to decide to focus on the midterm
- Bottom Up – Lower Level
 - Helps us miss getting hit by the bus when some yells "look out"
- The two streams cross too often and bottom-up today has been exploited by those who know how powerful it is.
 - Sorry to disappoint, especially for those good at top down, but bottom up rules the day ultimately
- To reclaim attention and focus concentration you must attack from both top and bottom

Two Top Minds

- Even at the top we are more than one thing – it isn't all logic.
 - Are we computers that act emotionally OR emotional beings who try to act like computers
 - As logical as you might think you are your biological and emotional system severely effects you
 - Things dubbed soft skills or characteristics end up being weakened to the point of danger
 - If we had to make a D&D character for ourselves what are our stats?
 - Intelligence? Wisdom? Constitution? Strength? Charisma? Etc.
 - Are we balanced characters?
- While we are not computers obviously interestingly, we driven by loops
 - Improve your loops improve yourself

METAL

My
Emotional
Thought
Action
Loop



Come on Prof...really

- Yeah really. Believe the ancient philosophers or believe modern sports if you like, it is the same advice
- Ask yourself a question, why do the elite athletes have “mental coaches”
 - Seattle Seahawks example
- Nearly all the elite people in sports and executional type things know this
 - Mental Simulation
 - Pro Golf Example – Reverse Movie Playback Visualization
 - The Media Knows This – How many times do you need to see it to realize this is real?
 - Example: The movie Rocky? Many Kung-Fu movies? Describe the “montage”
 - Ask Steve Jobs – watch some of his noted speeches, you can’t cherry pick Jobs he says this stuff very very strongly

Let's Actually Apply This Idea to Something We've Done

- Debugging!
- How much of debugging is really about attitude and discipline?
- Do any of you have a story to tell about debugging before I share a slide we'll likely revisit?

How much of this is technical?

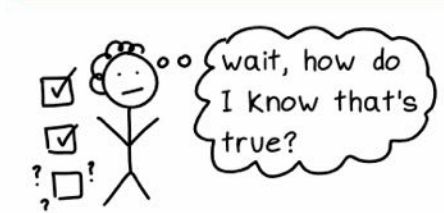
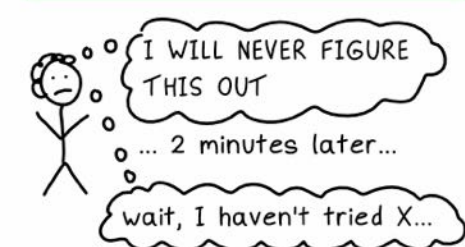
Other than #7 this is almost all attitude, process and reasoning!?!

Oh and my additions

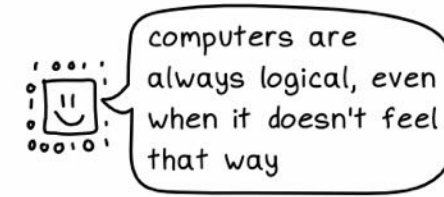
Reduce to last or a working point
Start from nothing and build up
Isolate the troubling area - 'pull it out and work on it separately'
Talk a walk, break, step back, etc.

a debugging manifesto

- 1 inspect, don't squash

- 2 check your assumptions

- 3 being stuck is temporary

- 4 don't go it alone

- 5 trust nobody and nothing

- 6 there's always a reason

- 7 build your toolkit

- 8 it can be an adventure


Yet Instead it is often “Straight to the GenAI!”
(Stack Overflow is so busted?)

Remember this one?



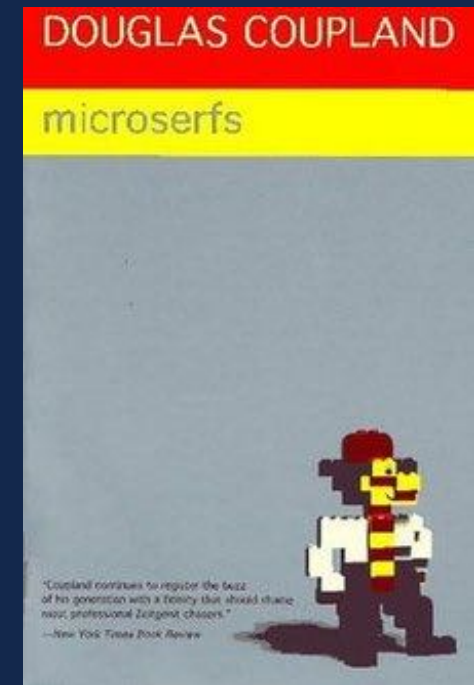
Interlude: Making This Real

*Are Soft Skills A Fast
Track to Senior?*

Soft/Special/Extra Skills = \$\$\$\$\$\$

- If \$ is what you want I strongly believe the faster route is working on these skills
- We do not want to be
 - Code Farmers, Meat Bots, Work Units, expanded “head count”, etc.

All this has happened before, and it will happen again...again...again...



<https://en.wikipedia.org/wiki/Microserfs>

Learn to play and understand this game and yourself if you want to win

Tech Skills Matter, Until They Don't?



Gergely Orosz
@GergelyOrosz



Early on in your software engineering career, getting promoted to the next level is mostly based on your work. After a few promos, many assume this will be the case going forward.

Oh no.

Getting promoted beyond this level & as a manager depends so much on your peers as well.

11:38 AM · Mar 7, 2022 · Buffer

The Path to Senior Engineer - Take 1

The Three Fundamental Traits

After eleven years as an engineering manager, I observed the skills that tech companies generally value the most. I noticed a pattern that generally falls into three different categories:

1. Technical expertise and craftsmanship

Understanding the fundamental concepts of computer science is the baseline to become a software engineer. Even though this is common knowledge, this science is vast and is continuously evolving.

2. Scope and autonomy

We do not live in a world where working alone and implementing specifications from UML diagrams is sufficient anymore. Those who know how to navigate complexity requiring minimum supervision are now extremely valuable professionals.

3. Communication and influence

Even though nobody expects you to be a skilled psychologist, we passed the era where programmers were introverts that spoke an unintelligible alien language. Knowing how to work with people and interact with non-technical partners is a valued skill in the market.

<https://betterprogramming.pub/what-does-it-take-to-become-a-senior-software-engineer-8accd950d98>

The Path to Senior Engineer - Take 2

	Expertise	S&A	Influence
Beginner	Learning	Feature & Supervised	Local
Seasoned	Proficient	Product & Tactical	Team
Advanced	Advisor	Domain & Strategy	X-functional/X-team

The Path to Senior Engineer - Take 3



Zain Rizvi  @ZainRzv · Mar 11



Junior engineer:

Take this tightly defined feature & build it

Mid-level engineer:

Take this vaguely defined feature & build it

Senior engineer:

Take this known problem & figure out how to solve it

Staff engineer:

Take this goal & find the problems we should be solving

 50

 853

 5,624



Junior to Senior Path - Take 4

Junior Engineer

Focus: Writing Software

Value code quality, “best practices”, adopt cutting-edge tech so lots of learning is required. Goal is elegant, performant, and maintainable software.

Senior Engineer

Focus: Building Systems

Understand the code isn't the point, though it might be part of the solution. Asks why questions, thinks of users, understanding environment, values measurement and focuses on finding a solution to a problem rather than writing a piece of software.

How?



Why?

True objectivity is not obtained until there is enough of a range of opposing experiences to provide a centering effect on the developer. There is no right way to code, only efficient ways based on situations and scenarios. The senior developer understands this. They accept that their code may not be the best and that there is space for improvement.

The Marks of a True Senior Developer...

Aphinya Dechalert

More Senior Signs

Senior developers often become effective code janitors, marking messes made by their peers and labeling the weak parts in the architecture. They are able to step back and see a much bigger picture with future contingencies while making choices based on the least expected negative impact. They are not bound to any one style of coding or paradigm — instead, focusing on the solution rather than the tool.

The Marks of a True Senior Developer...

Aphinya Dechalert

Sounds a lot like just simple experience (comes with time), humility and maturity

What I have Observed

- Poor or Average Software Engineers are just coders and find themselves troubled by tech shifts
- Good Software Engineers are at least somewhat Social Engineers and are less troubled by shifts and make predictable \$ gains over time.
- Great Software Engineers are almost always often polymaths who happen to code and often can name their price.
- If you want to be great, you'll ignore these observations at your peril, as so far 25 years+ they are holding other than random cases for me*
 - Tip: I avoid arguing with “lotto winner” style thinkers. If your friend's cousin did none of this and got rich as a coder, know that we can find someone who went to 7/11 and got millions in the lotto too. We can't all be special cases anyway!

Analogy

- Good footballers have good skills and fast reaction in the moment and in 1:1 situations, but what the need to be great is even more
- Great footballers have vision (heads up), communicate, are aware of others and how they relate and usually are thinking at least one step ahead (“what’s next?”)

Productivity Pursuits

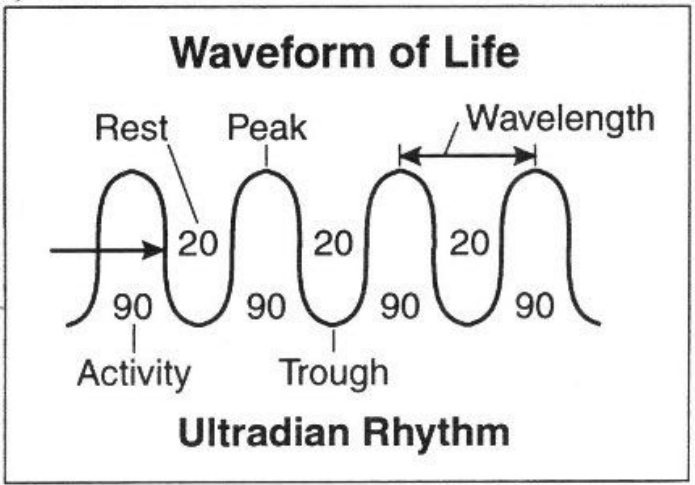
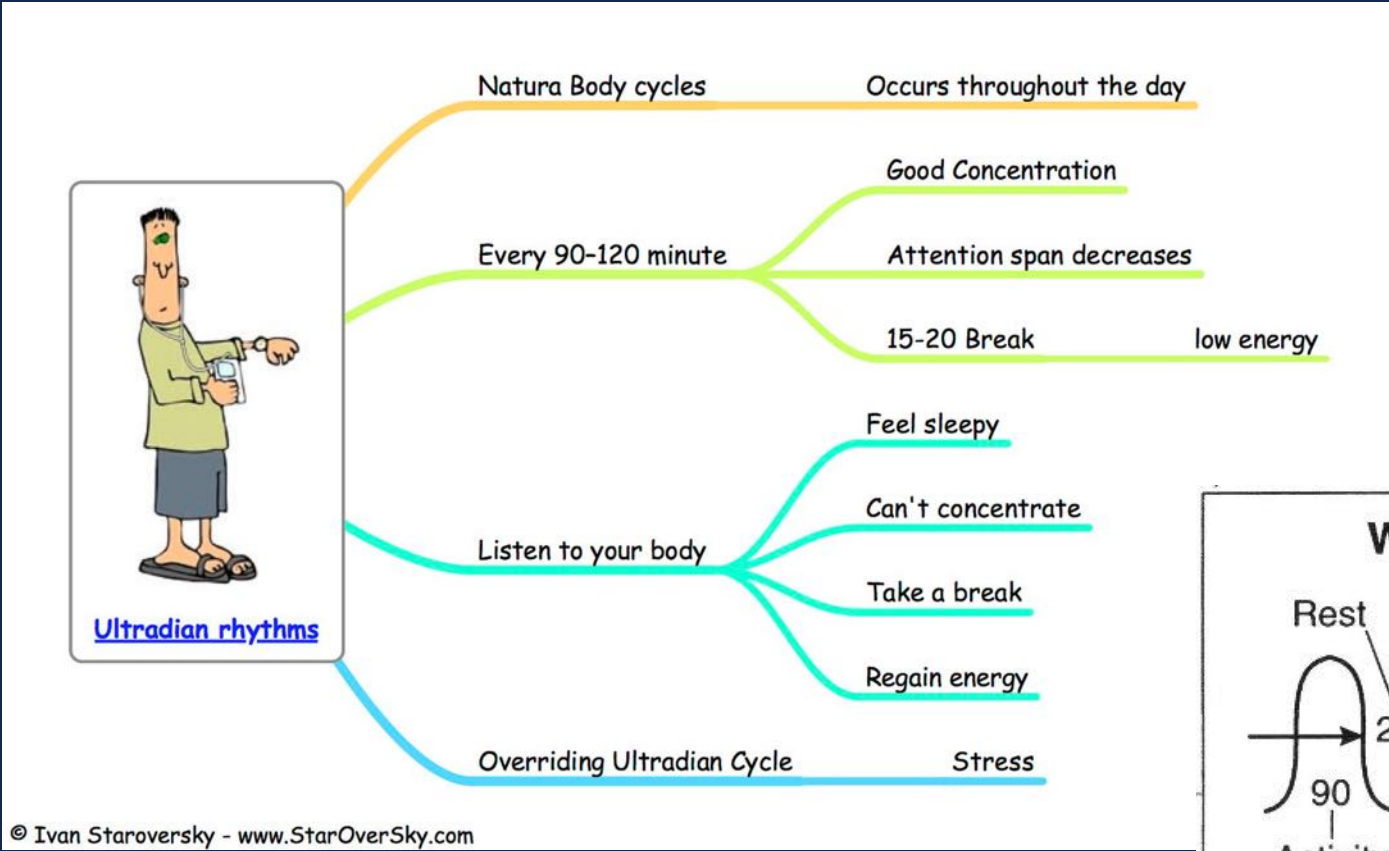
Productivity Myths

- Maximum productivity is a myth
 - Lots of “Feature Factories” exist though pushing this white collar assembly line approach to dev which believes this and “whips the ponies” to reach the goal or until the worker unit breaks down in some way
- Novice Programmers – believe in utopian workdays
- Experienced Software Engineers – accept real world days
- Real world agile does fundamentally agree with this in that in the estimation phase one is supposed to include a velocity factor (often 0.7) to temper work availability and estimates

Work Rhythms Part 1 - Time of Day

- Many people argue strongly for morning work regardless of discipline as the most effective approach for productivity increase
- A decent number on the other side cite individual energy patterns and push night work
- Few seem to argue for midday and afternoon work as it appears that human cycles and societal patterns don't seem to make this a great time for productivity
- My experience suggests it ultimately is somewhat the way a person is wired. However, rest seems to result in better work energy (thus the morning or the nap or exercise push) and interruptions tend to be a big deal (thus the night or early morning) and tiring out plus food comas are real so watch out for the afternoon. Ask Ben Franklin, Edison, Churchill, about naps, work, booze, rest, etc.

Respect Your Rhythms



Rhythm overrides are
sometimes needed

...but they have costs



Sleep

Secret Software Engineering Super Power



Hillel
@hillelogram



One of my most controversial software opinions is that your sleep quality and stress level matter far, far more than the languages you use or the practices you follow. Nothing else comes close: not type systems, not TDD, not formal methods, not ANYTHING.

Allow me to explain why.

2:10 PM · Apr 20, 2019 · Twitter Web Client

5.9K Retweets **802** Quote Tweets **13.7K** Likes



Naval
@naval



Forty hour workweeks are a relic of the Industrial Age. Knowledge workers function like athletes - train and sprint, then rest and reassess.

12:36 PM · Jun 10, 2017 · Twitter for iPhone

2,983 Retweets **265** Quote Tweets **6,964** Likes

Reality Check: You can't just sprint

Rest Matters



Gergely Orosz
@GergelyOrosz



The software engineering industry is so obsessed with measuring developer productivity... yet we're barely paying attention to measuring the impact of slack time / "not under pressure to ship" time on innovation, business impact, and retention.

9:46 PM · Feb 22, 2022 · Twitter Web App

208 Retweets **19** Quote Tweets **1,515** Likes

Last one...



Marcel Hagedoorn @marcelhagedoorn · Jan 2, 2021

True story

...

WHAT I THOUGHT WOULD MAKE ME PRODUCTIVE



WHAT ACTUALLY DOES

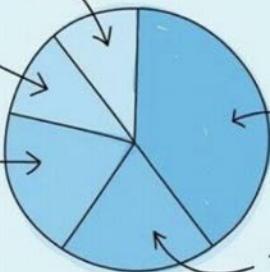
EXERCISE

HEALTHY EATING

SLEEP

HARD WORK

TIME OFF



@LIZANDMOLLIE

18

803

2,977

Work Rhythms Part 2 - Length of Work

- The problem with too much particularly all at once
- There are clear diminishing returns in focus and quality that have been measured
 - Again this is not at all specific to programming
- Idea: Employ the Pomodoro Technique
 - 25mins on, 5 mins off, work iterations if you like
- Roughly it appears particularly over time it is better to 2-4 hours focused attention (in flow hopefully) daily than 8+ hours of diminishing energy and distraction

Be Rhythmically Reasonable!

1hr a day!?!

Developers code less than one hour per day

Based on data from 250K+ developers in our global community, **developers code 52 minutes per day** — about 4 hours and 21 minutes during a normal workweek from Monday to Friday.¹

Code time is defined as time spent actively writing or editing code in an editor or IDE, which we use as an indicator of the amount of focused, uninterrupted time that developers have available to code during the workday.

Based on our estimates, developers spend an additional 41 minutes per day on other types of work in their editors, such as reading code, reviewing pull requests, and browsing documentation.

Our takeaway: Our findings suggest that developers frequently face constraints at work that prevent them from finding uninterrupted time to code.

<https://www.software.com/reports/code-time-report>

Careful here, the tool and type of person might really be skewing this!

Work Rhythms Part 3 – Variety and Type of Work

- It's clear that some types of work requires heavy concentration and other type of work doesn't
- Try to schedule your efforts for your energy and focus availability
- Cleaning code and making it meet a style guide can be a great mindless break, but why waste high energy hours doing that?
 - Easy to see people do this when they check off the easy items in an issue tracker right away thinking it builds momentum
 - Productivity research seems to suggest instead you would do better tackling harder things first when energy and attention is most plentiful
- Task switch is like “mental crop rotation” when done right and is distracted and inefficient thrashing when done wrong

Movie Time

Multitasking Demo

The Sequel with Science!



Work Rhythms Part 4 - Multitasking and Distraction

- Multitasking just doesn't work. Period. Stop.
 - Yes, we do it, but at our detriment for programmer productivity
- The Internet sings a constant siren song of better opportunities and information just over the horizon
 - There appears to be a generalized fear of being left behind, whether it be a social update or a new framework
- We can easily trick ourselves to seek out more motivating things often in the guise of being looped into what we are supposed to be doing
 - Examples: Product Hunt, Reddit, Hacker News, TechCrunch, etc.
 - We even see people with productivity-porn like habits always looking for the new GtD system, Todo list, Lifehack, etc.!

A man with dark hair and glasses is shown in profile, sitting at a wooden desk. He is wearing a blue button-down shirt. In front of him is a laptop displaying a dashboard with a line graph and a pie chart. He is holding a tablet in his left hand, which shows a social media or news feed, and a smartphone in his right hand, which displays a list of items. The background is slightly blurred, showing a chair and some papers on the desk.

R U Distracted?

Focus — a rare asset

Working on a single task- is a rare yet valuable asset.

Pay Attention!

[SUBSCRIBE](#)
4 weeks for only 99¢

[LOG IN](#)

Singer Bobby Brown's son found dead in Los Angeles home



Disney 'punishment' could blow up in DeSantis' face | Editorial



Pastor who shot, killed wife threatened to kill her during argument the day before,...



I-4 lanes shut down as Orange deputies help person 'in distress' on overpass ne...



Disr



[GUEST COMMENTARY](#)



Our attention span is shorter than a goldfish's. Here's what we can do about it | Commentary

By Jill Ebstein
InsideSources.com • Jul 06, 2021 at 5:30 am

 [Listen to this article](#)

There are so many crises that make the news, all of which are digested in bite-size morsels: political divisiveness, crime and policing, race relations, environmentalism, and the list goes on. What's not on the list but should be? Our short and ever-dwindling attention span. I'd happily take a bite-sized version of our concentration disorder.

Consider this: The average human attention span is now shorter than a goldfish's. A recent study found that the average human attention span has fallen from 12 seconds in 2000 to eight seconds today. It is reported that goldfish have a 9-second attention span.

LATEST

[ORLANDO MAGIC](#)
Magic's Markelle Fultz finished season healthy, comfortable with new teammates
8m



[NATIONAL & WORLD NEWS](#)
Grand Rapids police chief releases name of officer who shot Patrick Lyoya
13m



[NATIONAL & WORLD NEWS](#)


S E N D H E L P P L E A S E X



🔍 You have too many tabs open.



Jason Lengstorf @jlengstorf · Jul 18

...

The lie that we can get more work done by working on multiple things at once is so absurd, so incorrect, & so utterly destructive that I can't believe it's a near-universal practice.

I wrote a ~~rant~~ post on why no one should multitask. Ever. Seriously.



jason.af

Please stop multitasking. I'm begging you. Please.

The greatest trick we ever pulled on ourselves as knowledge workers was convincing ourselves we could juggle multiple projects with no ...

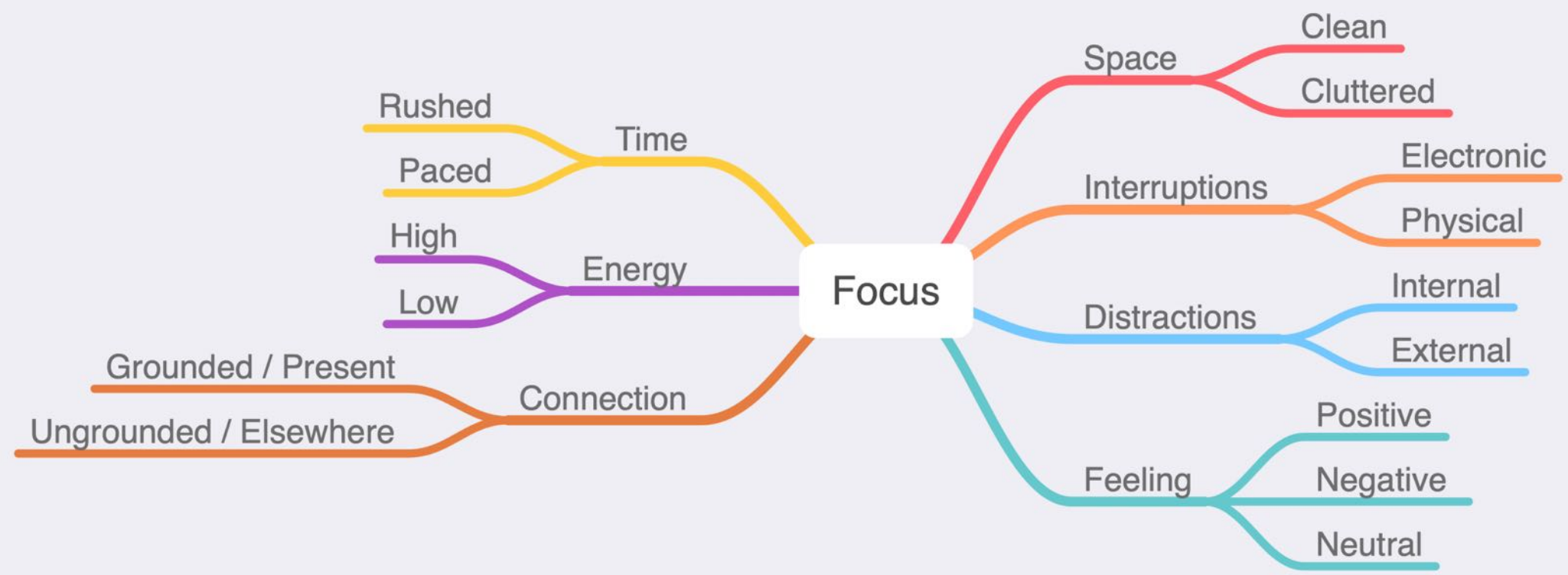


↻ 19

❤ 76

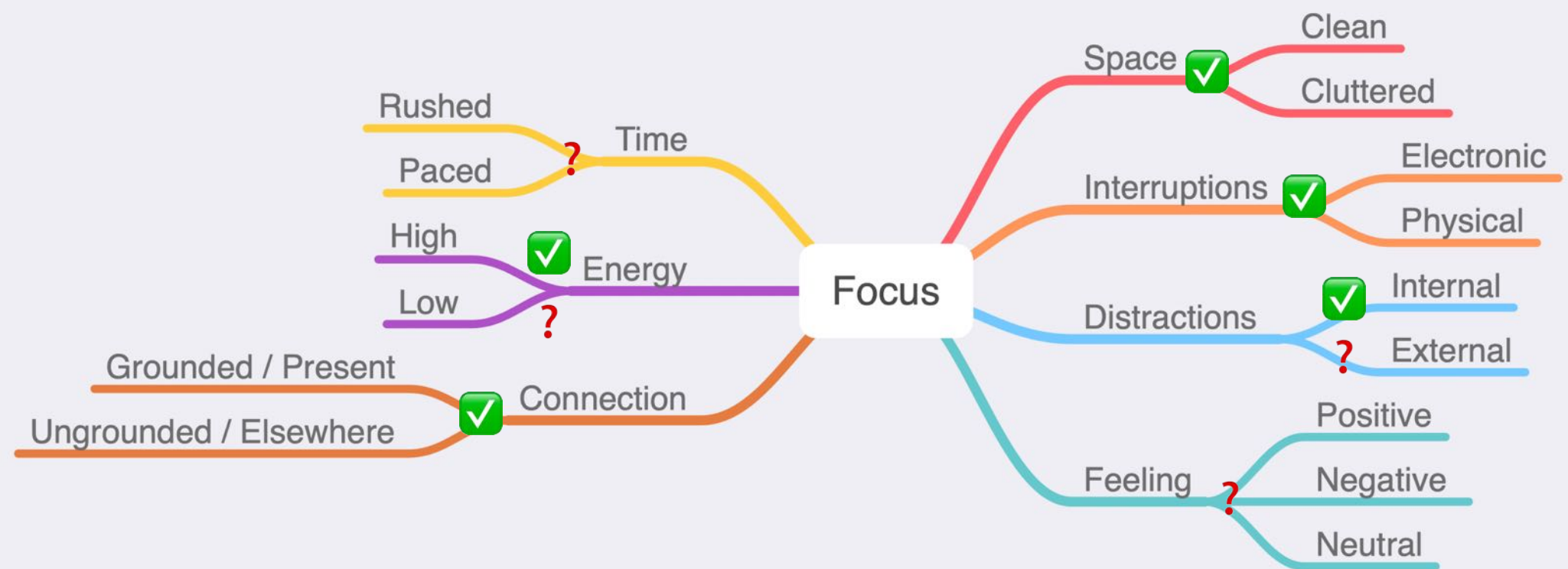


Focus Factors



🤔 Discussion: Which are controllable?

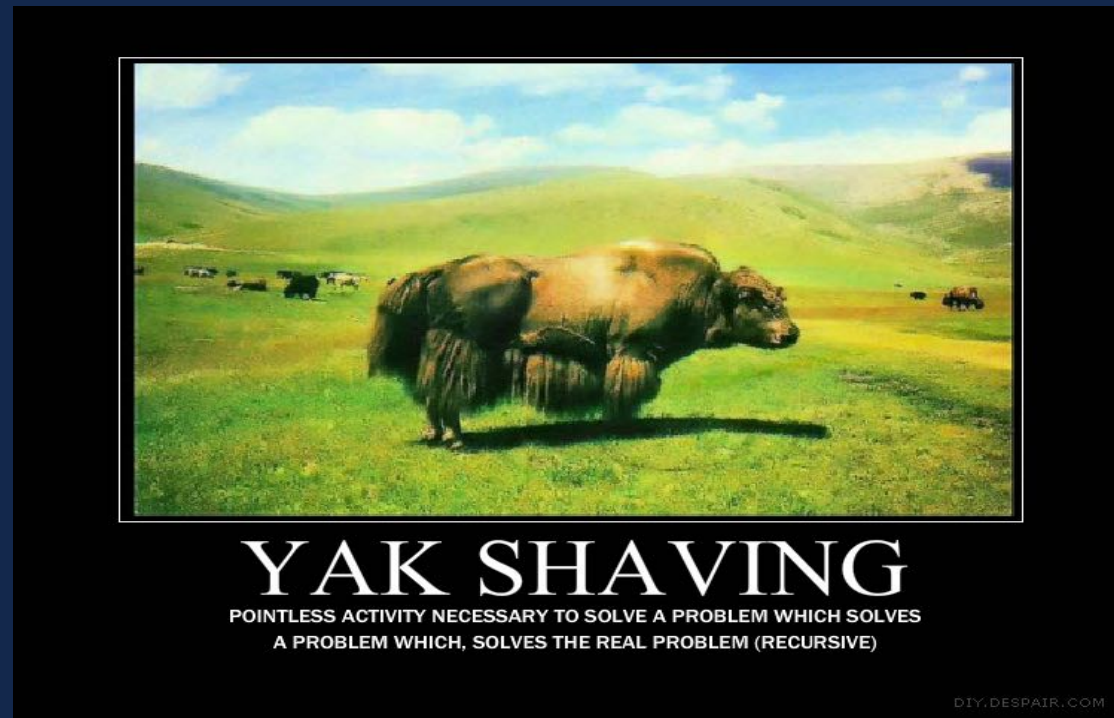
Focus Factors - My Answer



?

✗ ?

Yak Shaving != Productivity



[http://www.hanselman.com/blog/
YakShavingDefinedIIIGetThatDoneAsSoonAsIShoveThisYak.aspx](http://www.hanselman.com/blog/YakShavingDefinedIIIGetThatDoneAsSoonAsIShoveThisYak.aspx)

http://sethgodin.typepad.com/seths_blog/2005/03/dont_shave_that.html

Why do we need these?

**RescueTime**

[Home](#) [Features](#) [Pricing](#) [Customer Stories](#) [Blog](#) [Sign In](#) [Sign Up](#)

Find your ideal work-life balance.

With so many distractions and possibilities in your digital life, it's easy to get scattered.

RescueTime helps you understand your daily habits so you can focus and be more productive.

Get Started



How It Works

Anti-Social 

 Need Support?
Call 415-968-9172

 Email Us:
support@80pct.com

Target Your Digital Distractions

Anti-Social is the amazing app that makes it easy for you to target and block any distracting website so you can be more productive.

- ✓ Works with Windows and Macs, including El Capitan
- ✓ Block any site that wastes your time
- ✓ Keeps you honest, hard to turn off!
- ✓ Fully supported, 60-day money back guarantee

\$15

60 Day Money Back Guarantee

TRY NOW >

As seen in

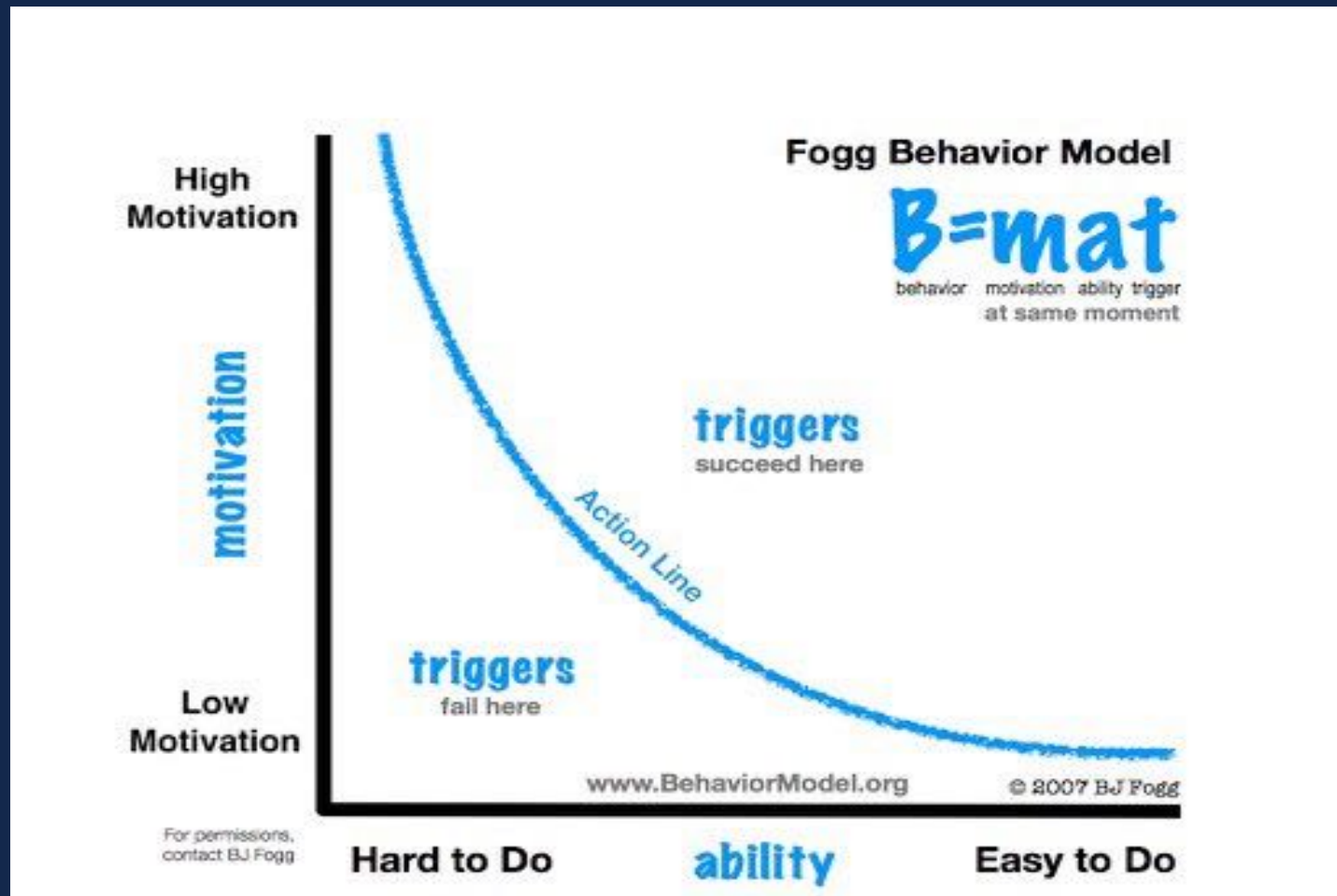
Features

Product Tour

Try Anti-Social

Press Coverage

Should We Use This Instead?

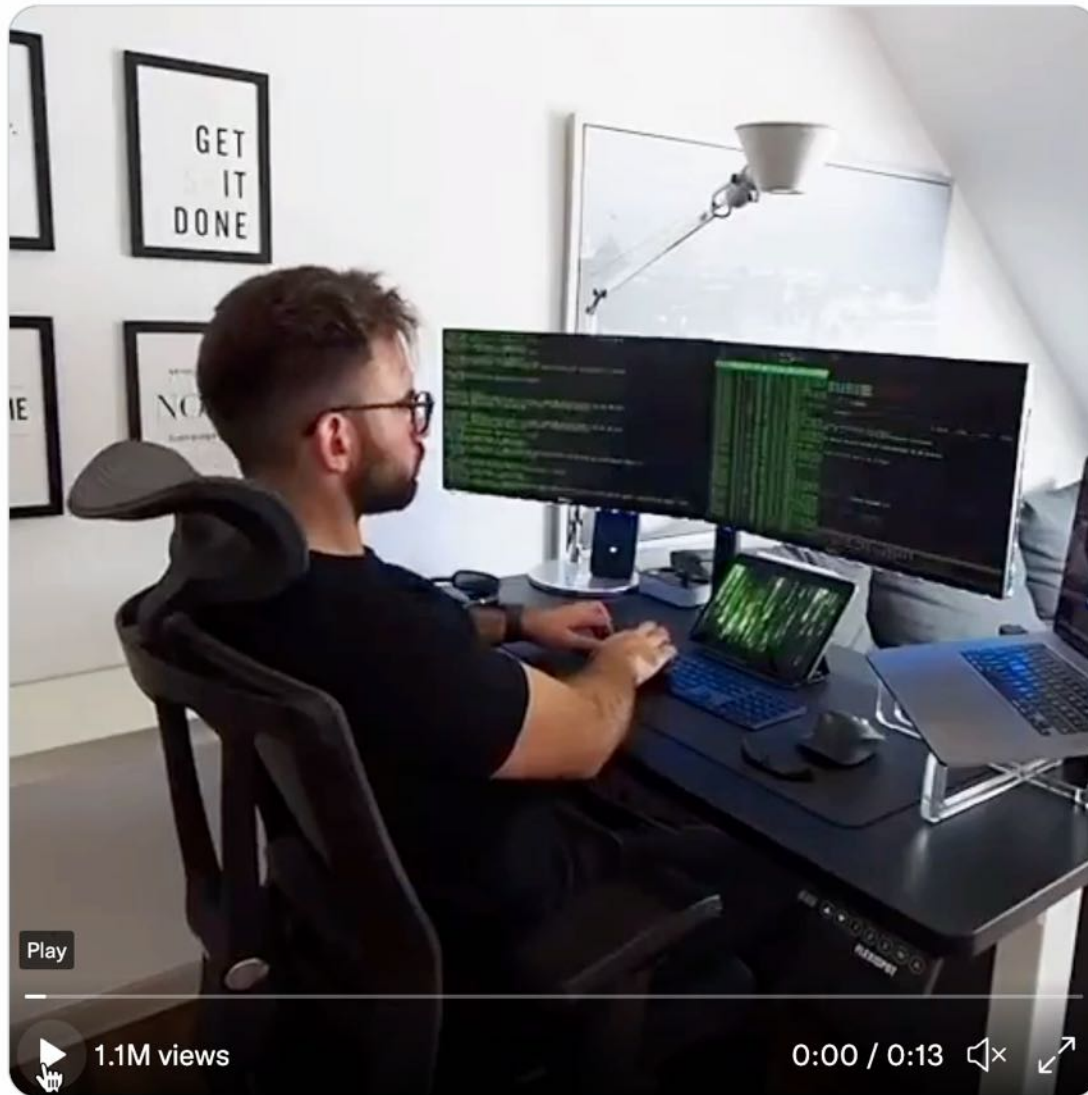


Work Rhythm Part 4 - Flow State

- Focused concentration where lots of other things melt away and you often are highly effective (and calm)
 - [https://en.wikipedia.org/wiki/Flow_\(psychology\)](https://en.wikipedia.org/wiki/Flow_(psychology))
 - <http://psygrammer.com/2011/02/10/the-flow-programming-in-ecstasy/>
- Getting into flow state is difficult somewhat like meditation and requires practice
 - Getting knocked out of it is easy with interruptions
 - Headphones, private offices over open floor plans
 - Distractions – chat, email, the Internet ☺
- Generally a few hours of flow is better than a day of bouncing around, even though you might feel you worked a lot you might have accomplished a little

Yet...Observe Reality Meeting Flow

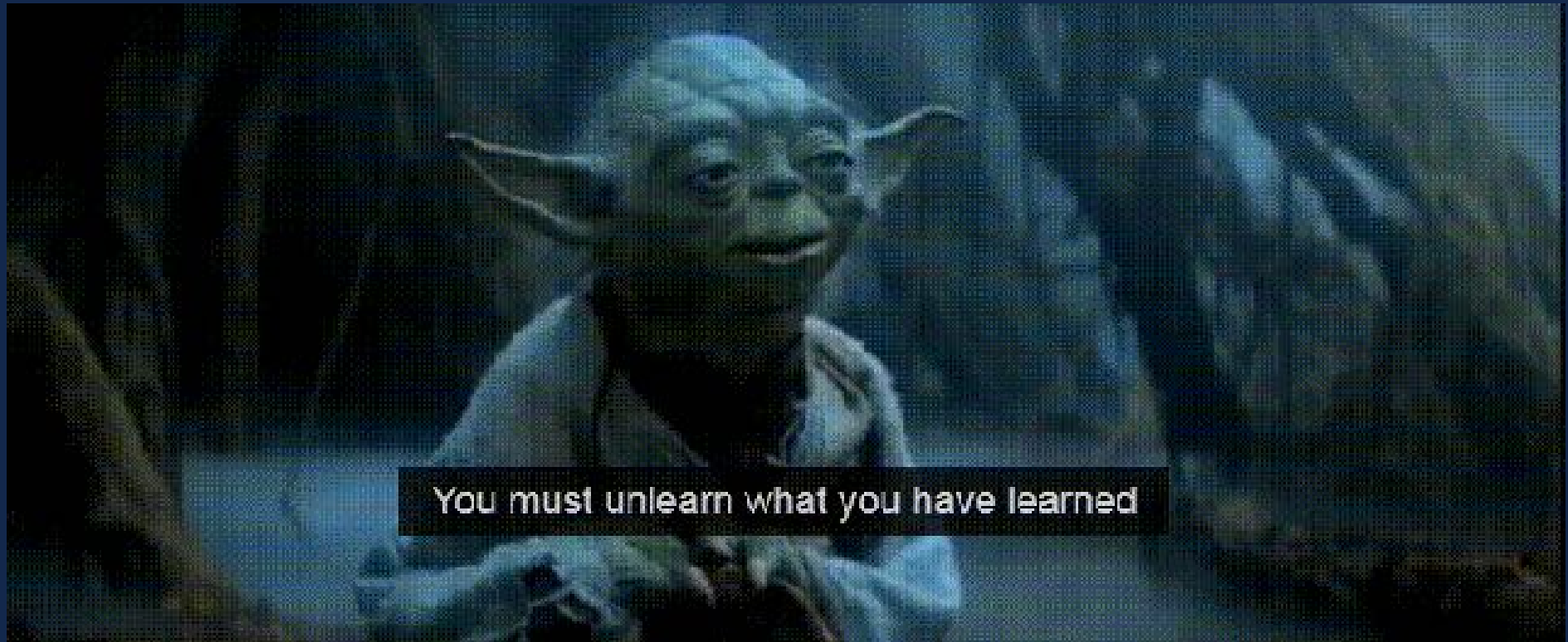
Software development: expectations vs. reality



Counter Point or Helpful Alternative - Beginners Mind

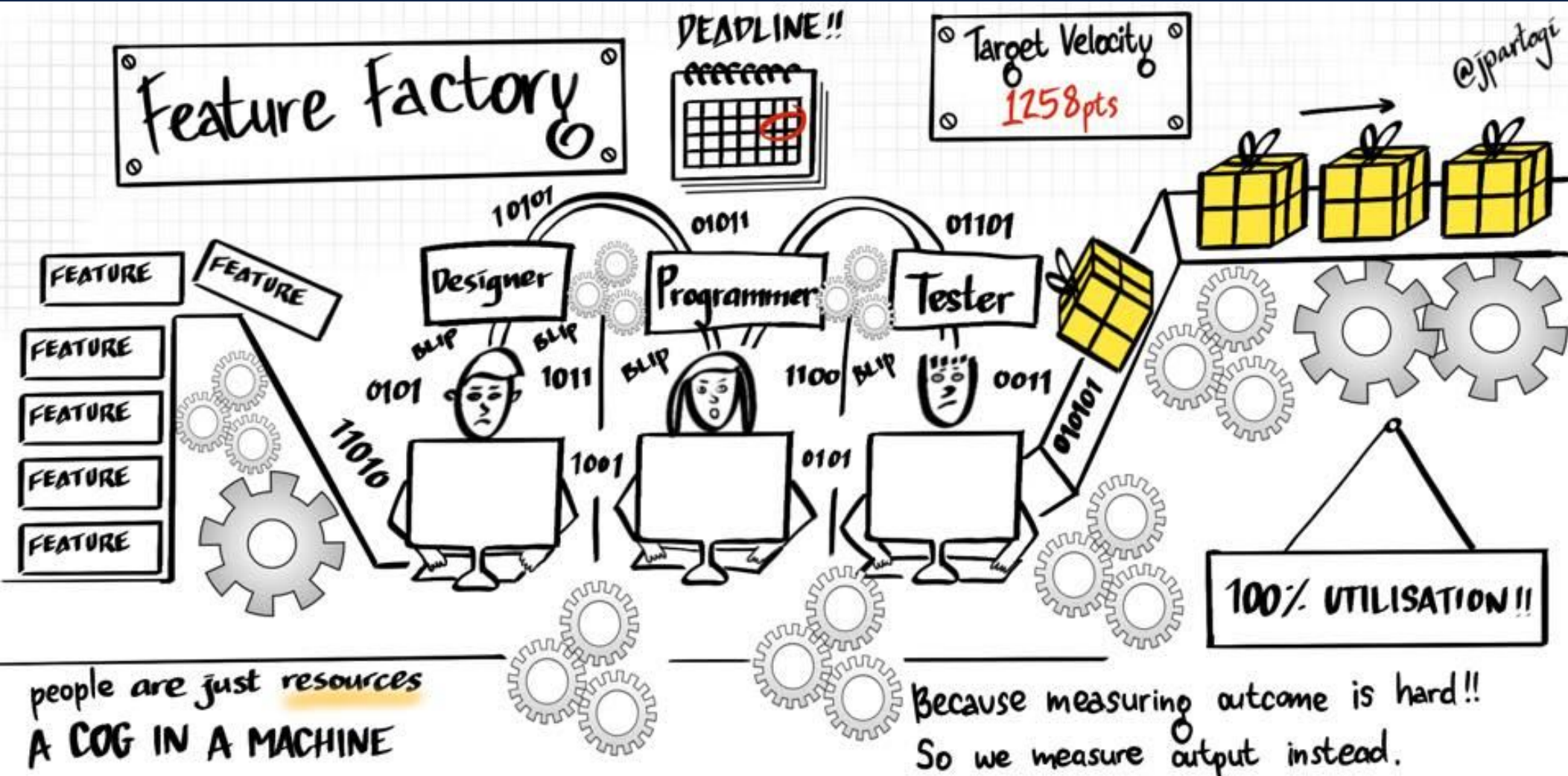
- Despite all this discussion of Flow online and beyond the Beginner's Mind [Shoshin (初心)] turns out especially in groups to be quite valuable especially as ability often built in flow state increases
- An interesting study about “Promiscuous Pairing” showed that measurable productivity increased over time on a team when swapping to less skilled developers as people got good at things. The return to novice allowed new insights and for data to be shared.
- The beginner's mind allows us to see the many possibilities while our expertise built on things that may have worked before can limit our view.

Oh now I get it...



Taking Process and Iteration a Bit Too Far

Maybe You End Up Working at a Feature Factory



<https://www.facebook.com/jpartogi/photos/a.1009723499131924/2076980699072860/>

<https://cutle.fish/blog/12-signs-youre-working-in-a-feature-factory>

<https://amplitude.com/blog/12-signs-youre-working-in-a-feature-factory-3-years-later>

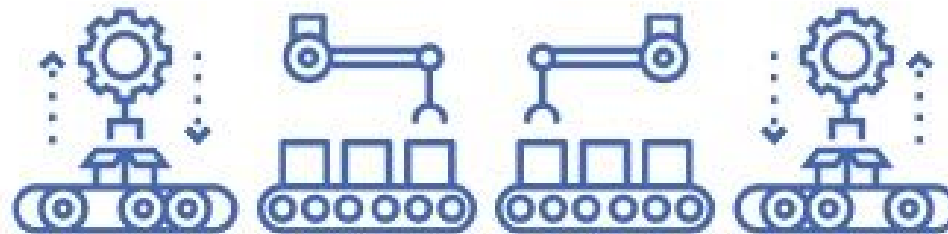
//CSE
110;

Feature Factories Effect SE

WHAT IS A FEATURE FACTORY?

Sitting in the factory, cranking out features, and sending them down the line

JON CUTLER



No connection from
delivery to value.



Success is binary
feature deliveries,
lack of failure.



Lack of product strategy,
understanding of
customer experience.



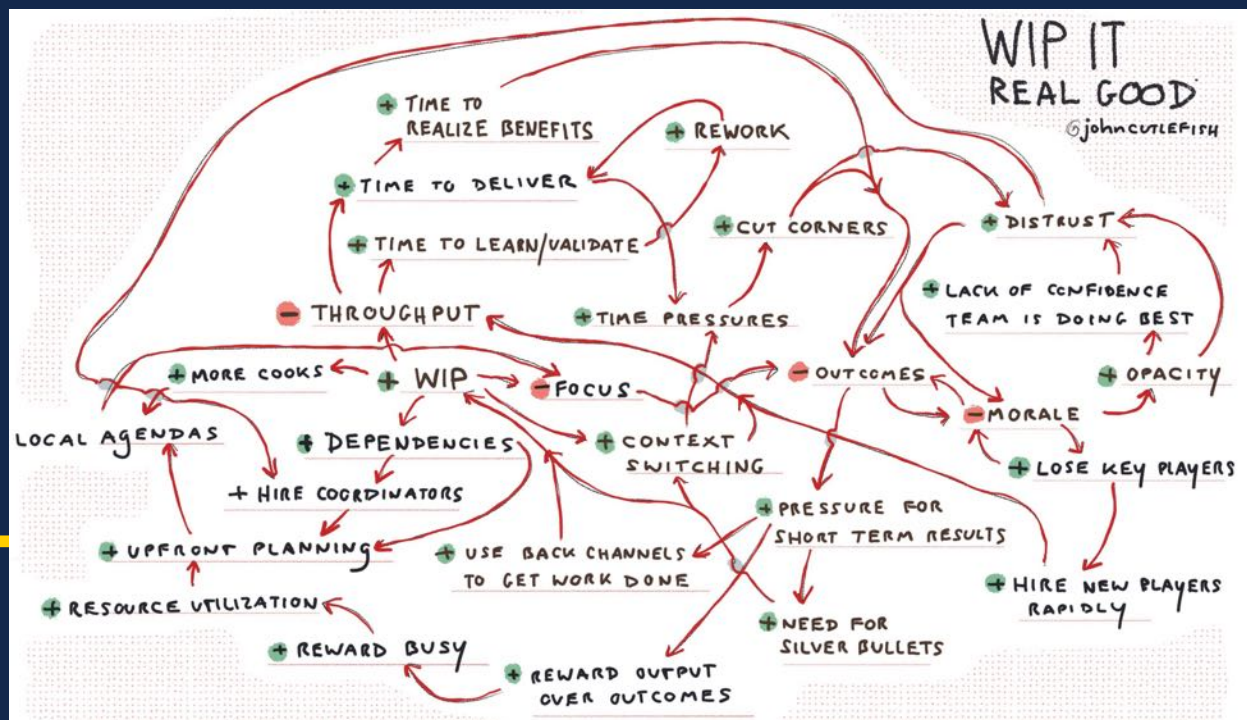
Large batches,
shiny objects.

© 2018 Michael R. K. W. W.

Which Leads Often to “Too Much WIP”



- WIP as Work In Progress
- Many “feature factories” run very high WIP and always aim to keep people busy, too much WIP leads to many challenges



And right back to ...

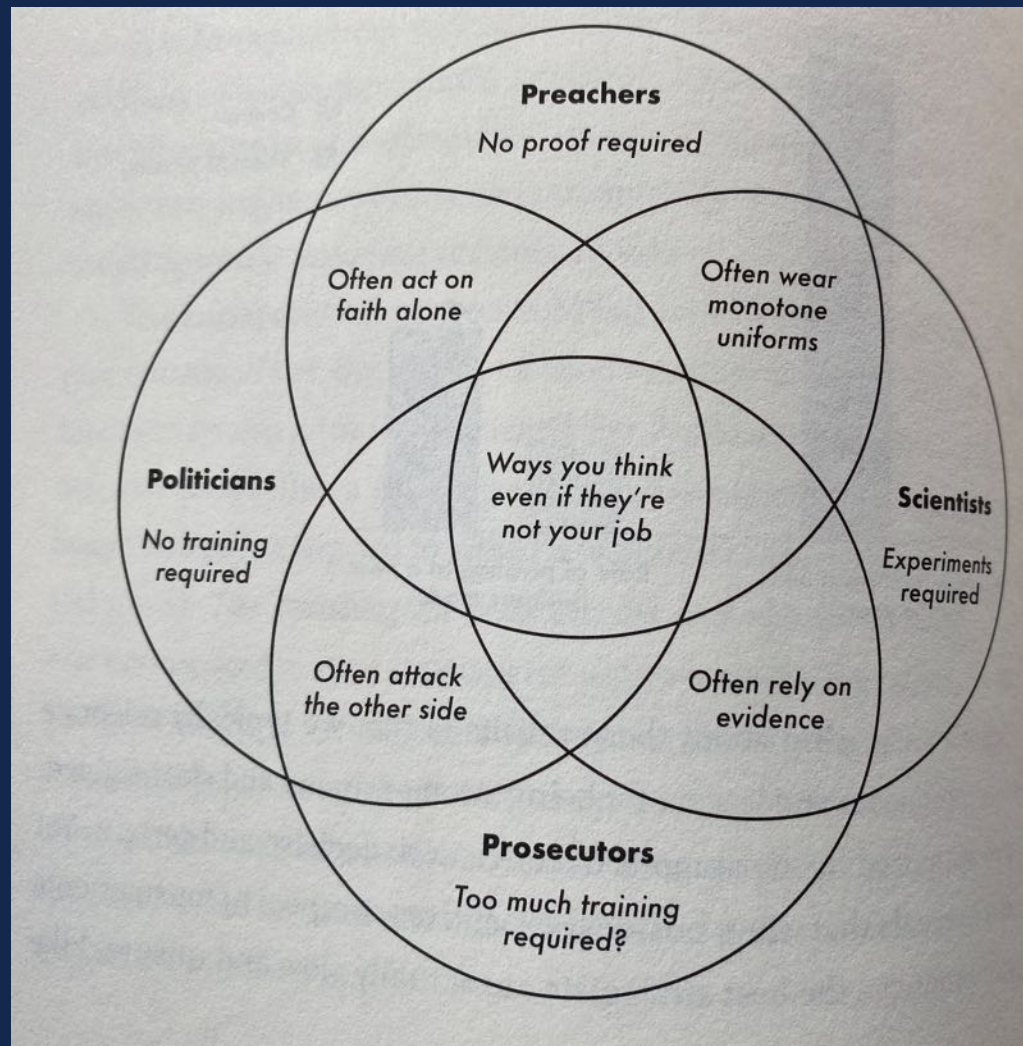
- Multitasking and overload madness
- Understand that we are part of systems and it is hard for us to effect change until we see our role in said systems
- Sometimes we are on that “crazy thing” so maybe get off or change the speed!



Developer Attitudes and Worries

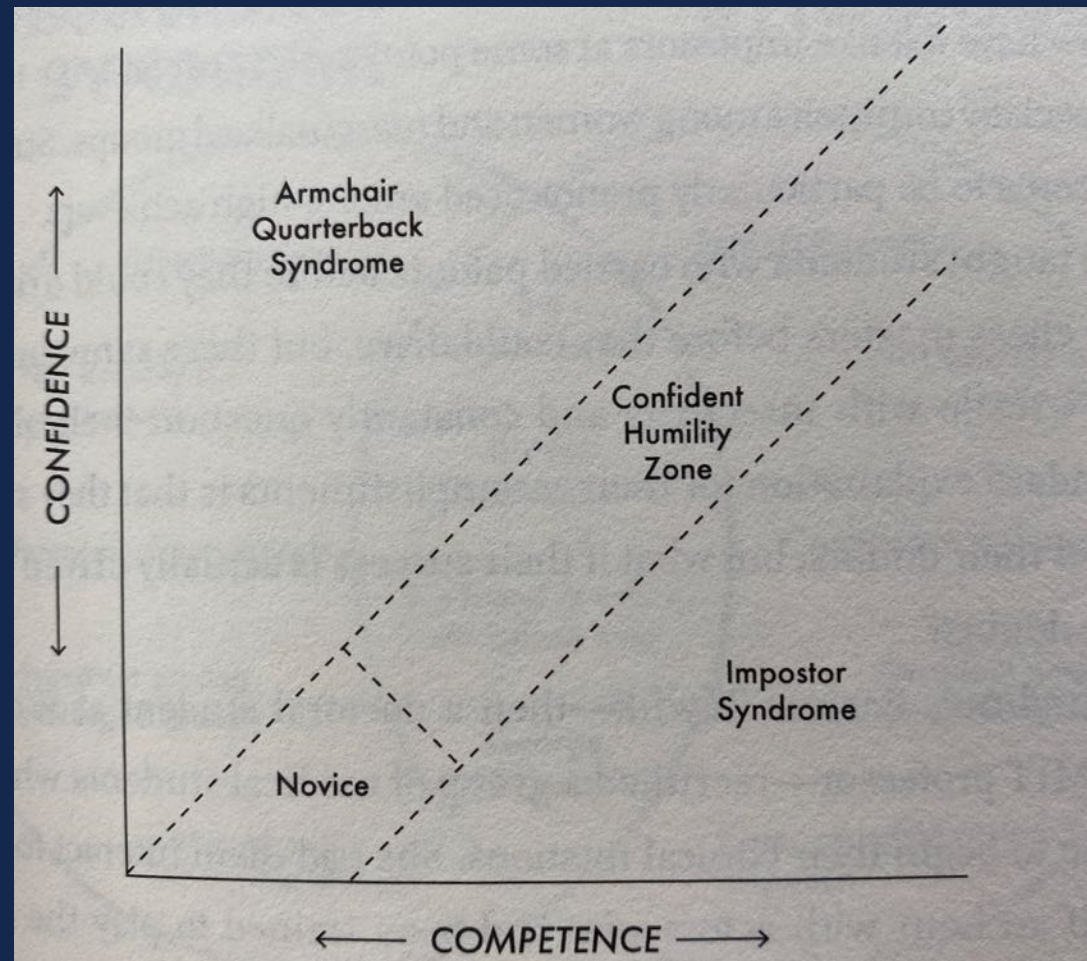
My Most Heard Topic from Students

How do we act?



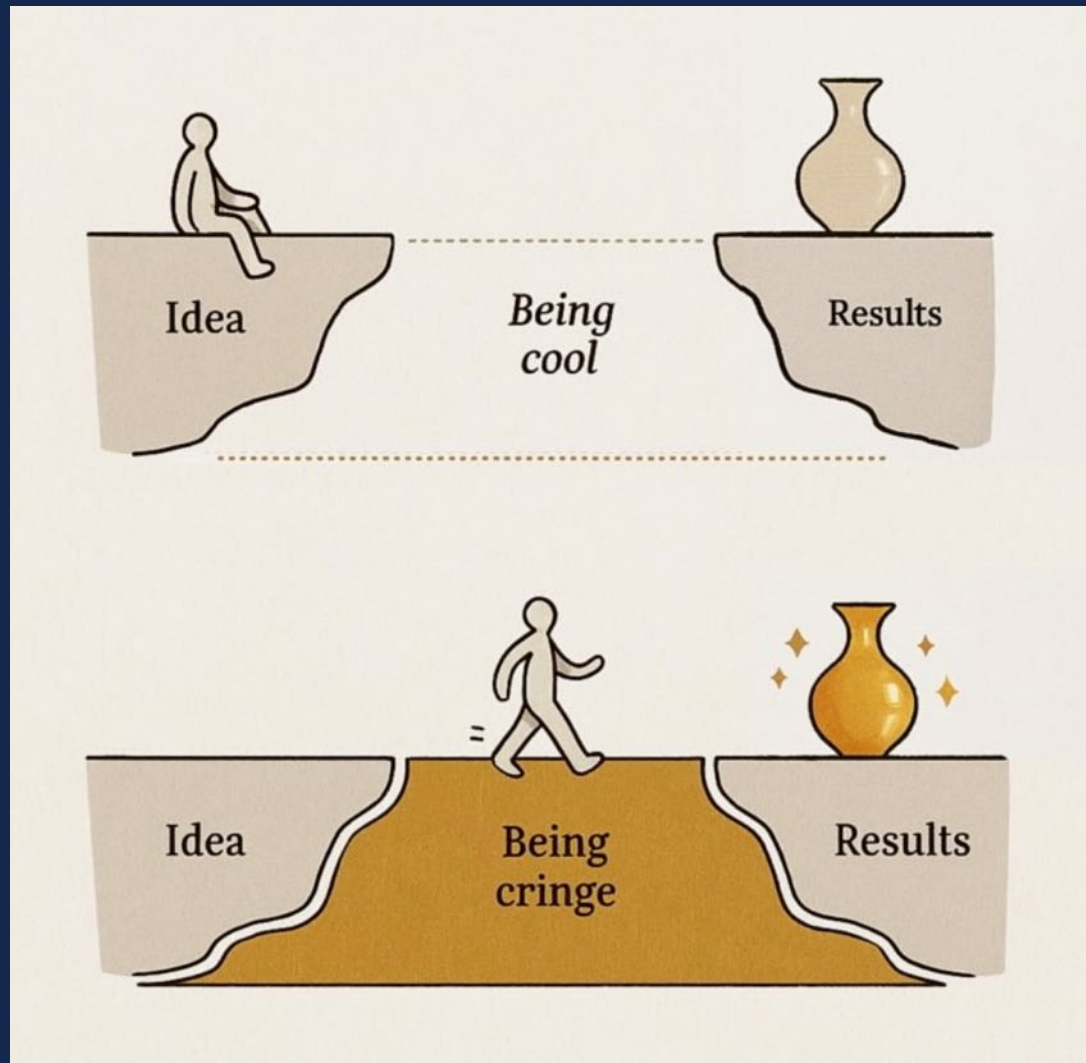
You may lean to an archetype, but really you are all of these at one time or another.

Confidence and Competence

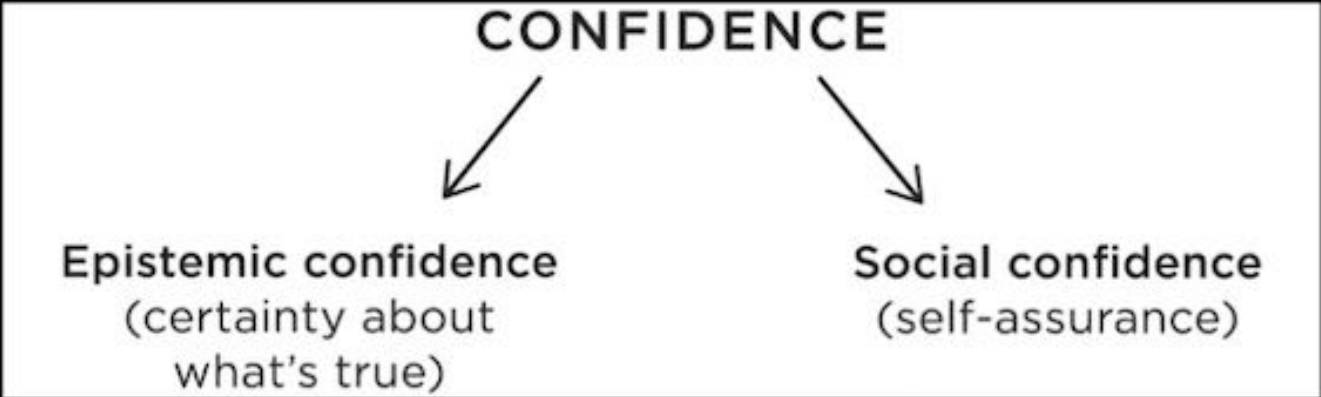


It can be difficult to find that middle channel!

We Gotta Allow It, If We Want to Grow



Careful Consideration of Confidence



	BEHAVIORAL CUE	OBSERVER-PERCEIVED COMPETENCE
Social confidence	Percent of time spoke	0.59**
	Confident and factual vocal tone	0.54**
	Provided information relevant to problem	0.51**
	Expanded posture	0.37**
	Calm and relaxed demeanor	0.34**
	Offered an answer later	0.24*
	Offered an answer first	0.21*
Epistemic confidence	Statements of certainty in estimate	0.21*
	Statements about ease or difficulty of task	0.18
	Statements about one's own competence	0.09

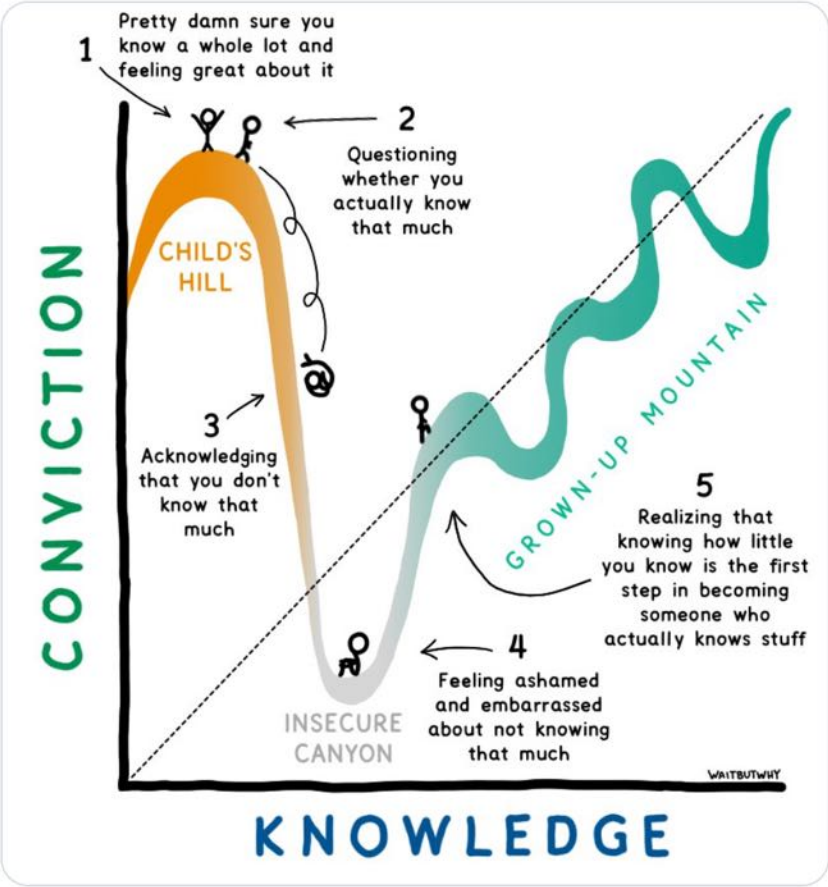
TABLE ADAPTED FROM C. ANDERSON ET AL. (2012), TABLE 2. PAGE 10.

* = RESULTS SIGNIFICANT AT $p < 0.05$; ** = RESULTS SIGNIFICANT AT $p < 0.01$

Remember Our Old Friend Dunning Krueger



Tim Urban @waitbutwhy · Mar 6
Seems relevant these days



The diagram illustrates the Dunning-Kruger effect as a journey across a landscape. The vertical axis is labeled 'CONVICTION' and the horizontal axis is labeled 'KNOWLEDGE'. A dashed diagonal line represents the path of increasing knowledge and conviction. The journey is divided into five numbered stages:

- 1** Pretty damn sure you know a whole lot and feeling great about it. (Top of the first hill, labeled 'CHILD'S HILL')
- 2** Questioning whether you actually know that much. (Down the slope of 'CHILD'S HILL')
- 3** Acknowledging that you don't know that much. (Bottom of the valley, labeled 'INSECURE CANYON')
- 4** Feeling ashamed and embarrassed about not knowing that much. (Bottom of the valley, labeled 'INSECURE CANYON')
- 5** Realizing that knowing how little you know is the first step in becoming someone who actually knows stuff. (Up the slope of the second hill, labeled 'GROWN-UP MOUNTAIN')

The diagram is signed 'WAITBUTWHY' at the bottom right.

99

1,896

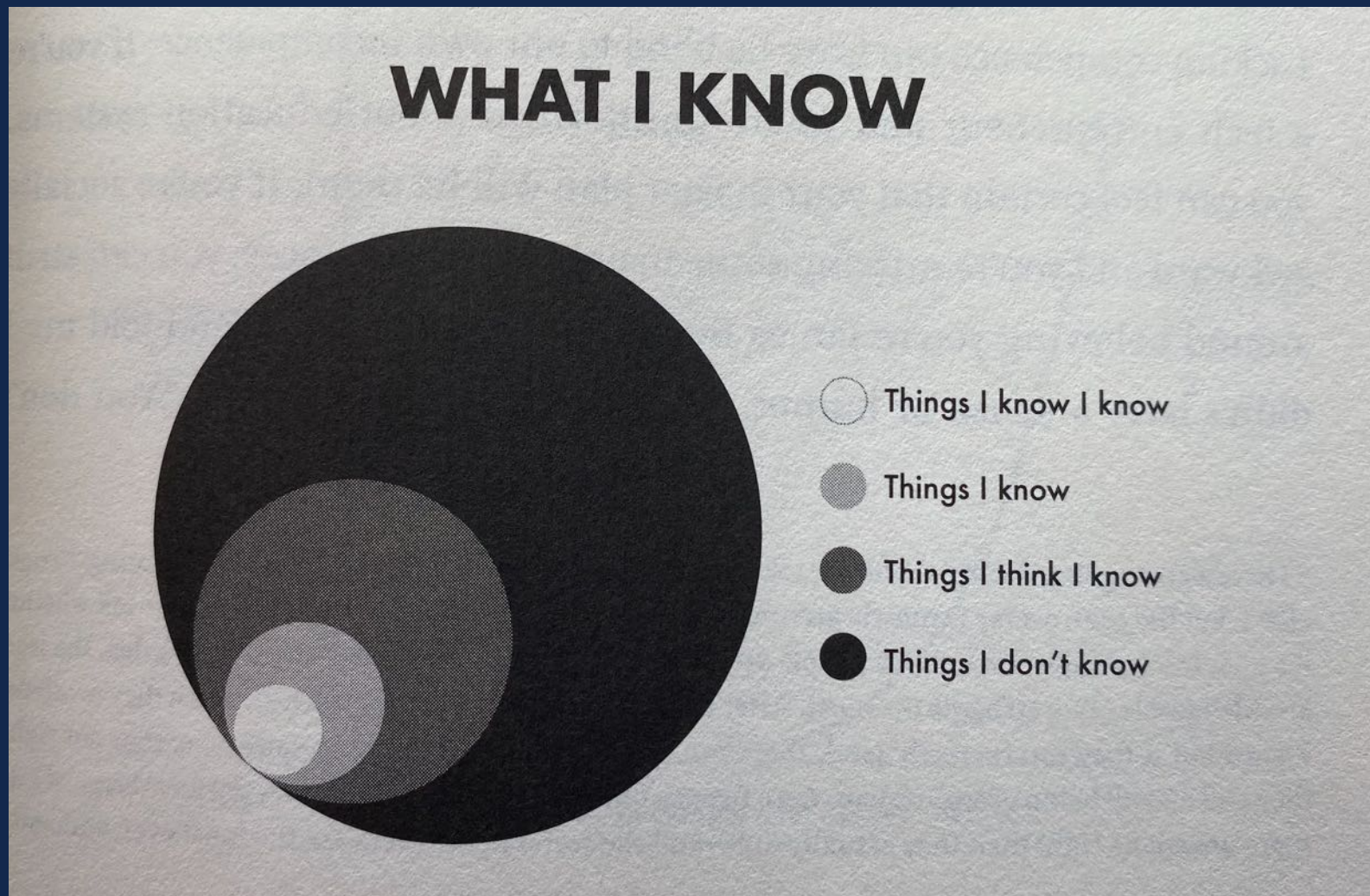
10K

Knowing All The Things

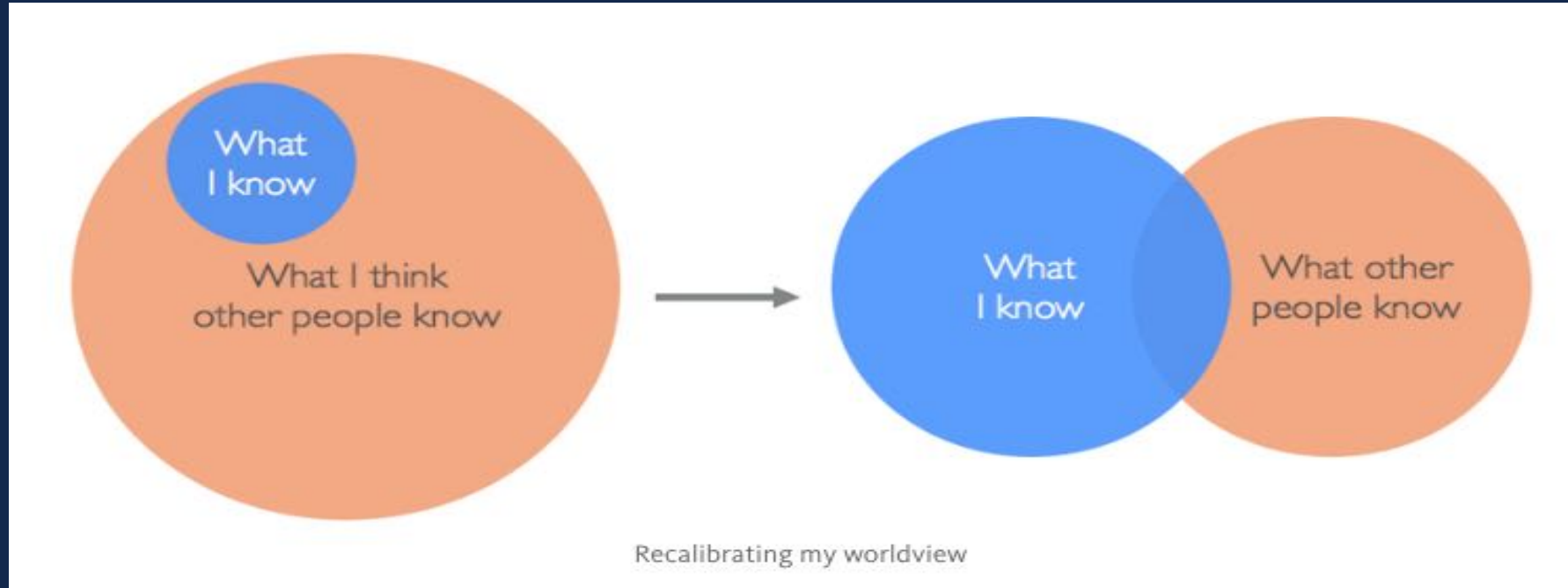
Danger of Knowing All The Things?

- Many developers tie their self-worth up in the knowing of things - in our case technical things
- Obviously it is a good idea to know important things that help us make good decisions and accomplish our tasks as SWEs
- However, doing all the things isn't really reasonable and is in fact unhealthy

Variation of the Pie Chart of Doom



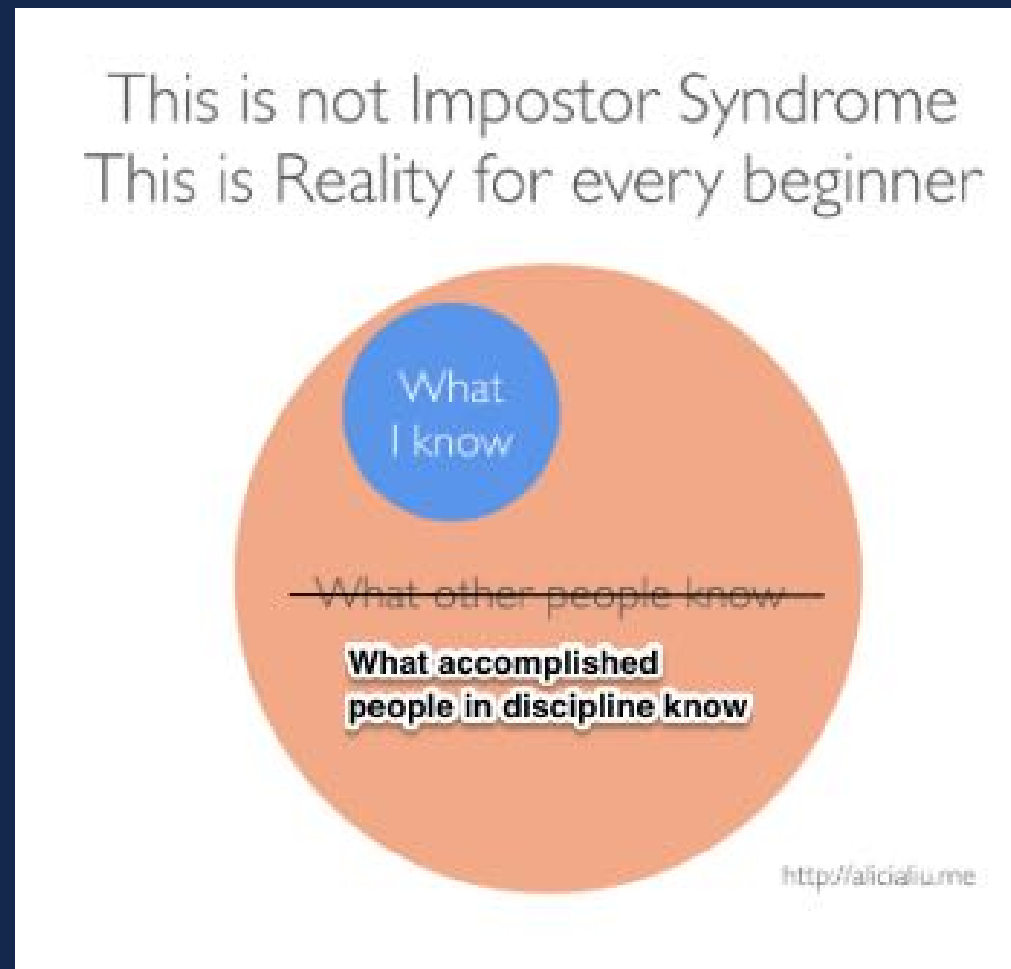
Impostor Syndrome



<https://medium.com/@aliciatweet/overcoming-impostor-syndrome-bdae04e46ec5#.jl5rmjs2u>

[https://www.reddit.com/r/cscareerquestions/comments/1tbmmm/
how_do_you_deal_with_impostor_syndrome_as_a/](https://www.reddit.com/r/cscareerquestions/comments/1tbmmm/how_do_you_deal_with_impostor_syndrome_as_a/)

Healthy View of Being Novice

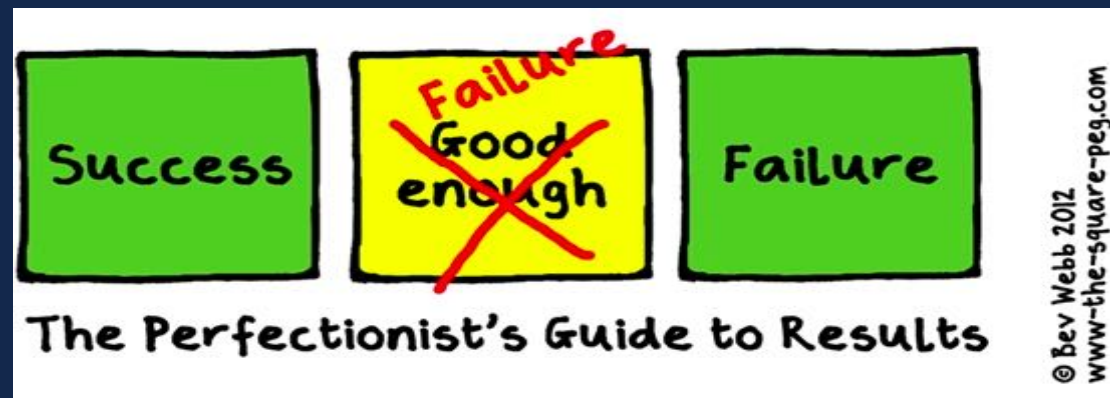


It's Ok to Not Know Stuff



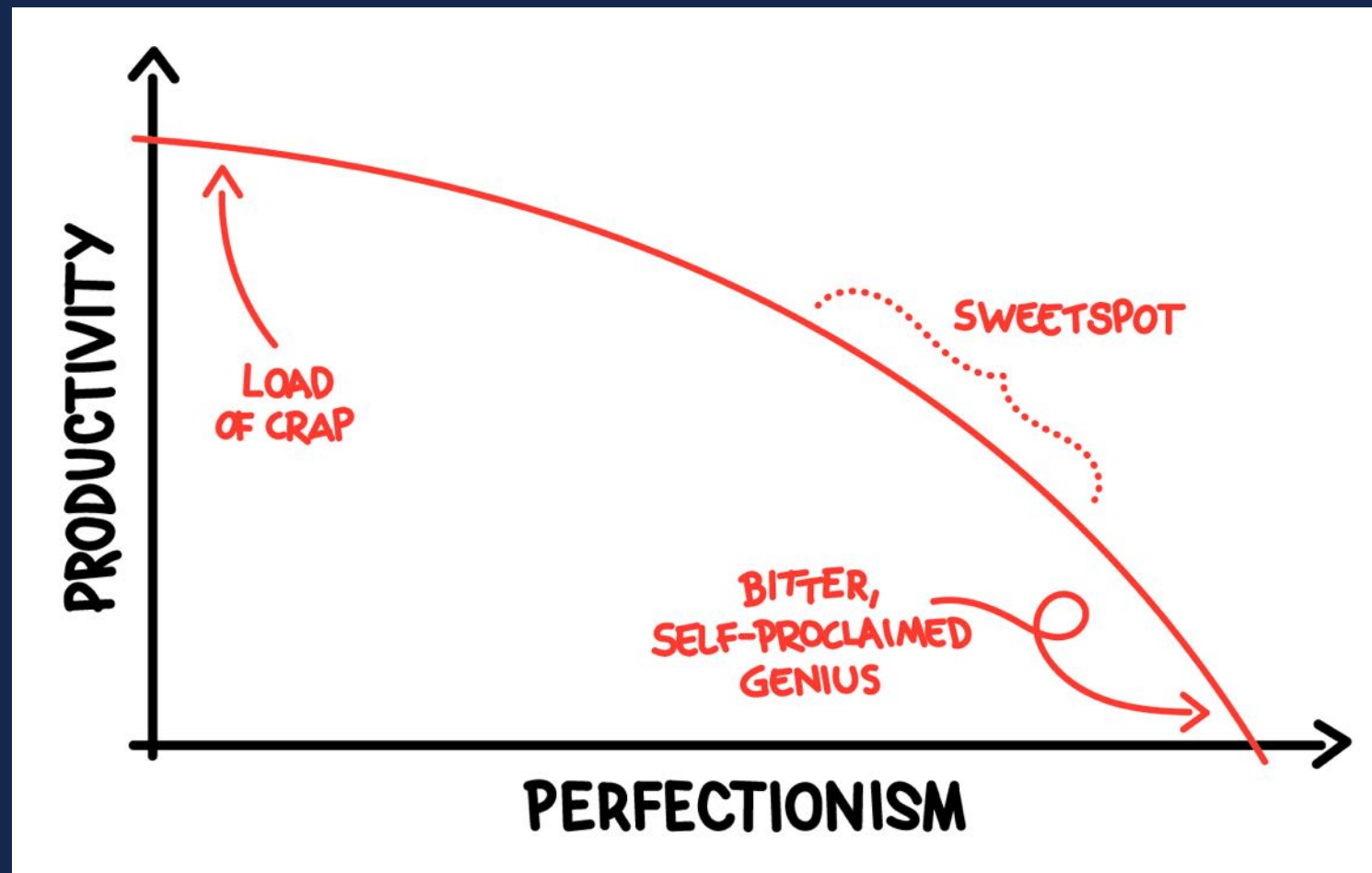
📺 Interviewers often employ an “will they admit ignorance” test so be careful!

Perfectionism



- [https://en.wikipedia.org/wiki/Perfectionism_\(psychology\)](https://en.wikipedia.org/wiki/Perfectionism_(psychology))
 - This is not actually a positive trait; some tendencies are but in general it is very counter productive
- Perfectionism really seems anti-Agile as agile embraces continuous improvement
 - In my experience perfectionists don't ship software often and when they do it isn't perfect anyway
- Procrastination is related to perfectionism

Perfectionism and Productivity



Programmer Personalities

The 5 types of programmers

July 18th, 2010 -- Desktop Development, Web Development (Opinion) 272 Comments »

In my code journeys and programming adventures I've encountered many strange foes, and even stranger allies. I've identified at least five different kinds of code warriors, some make for wonderful comrades in arms, while others seem to foil my every plan.

However they all have their place in the pantheon of software development. Without a healthy mix of these different programming styles you'll probably find your projects either take too long to complete, are not stable enough or are too perfect for humans to look upon.

The duct tape programmer

The code may not be pretty, but damnit, it works!

This guy is the foundation of your company. When something goes wrong he will fix it fast and in a way that won't break again. Of course he doesn't care about how it looks, ease of use, or any of those other *trivial* concerns, but he will make it happen, without a bunch of talk or time-wasting nonsense. The best way to use this person is to point at a problem and walk away.

The OCD perfectionist programmer

You want to do what to my code?

This guy doesn't care about your deadlines or budgets, those are insignificant when compared to the art form that is programming. When you do finally receive the finished product you will have no option but submit to the stunning glory and radiant beauty of perfectly formatted, no, perfectly beautiful code, that is so efficient that anything you would want to do to it would do nothing but defame a masterpiece. He is the only one qualified to work on his code.

The anti-programming programmer

I'm a programmer, damnit. I don't write code.

His world has one simple truth; writing code is bad. If you have to write something then you're doing it wrong. Someone else has already done the work so just use their code. He will tell you how much faster this development practice is, even though he takes as long or longer than the other programmers. But when you get the project it will only be 20 lines of actual code and will be very easy to read. It may not be very fast, efficient, or forward-compatible, but it will be done with the least effort required.

The half-assed programmer

What do you want? It works doesn't it?

The guy who couldn't care less about quality, that's someone else's job. He accomplishes the tasks that he's asked to do, quickly. You may not like his work, the other programmers hate it, but management and the clients love it. As much pain as he will cause you in the future, he is single-handedly keeping your deadlines so you can't scoff at it (no matter how much you want to).

The theoretical programmer

Well, that's a possibility, but in practice this might be a better alternative.

This guy is more interested in the options than what should be done. He will spend 80% of his time staring blankly at his computer thinking up ways to accomplish a task, 15% of his time complaining about unreasonable deadlines, 4% of his time refining the options, and 1% of his time writing code. When you receive the final work it will always be accompanied by the phrase "if I had more time I could have done this the right way".

<http://stevenbenner.com/2010/07/the-5-types-of-programmers/>

Depth of CS or Breadth Beyond CS?



Ken Kocienda @kocienda · Aug 25



Computing is a field where it helps to know a lot about things outside of computing.



6



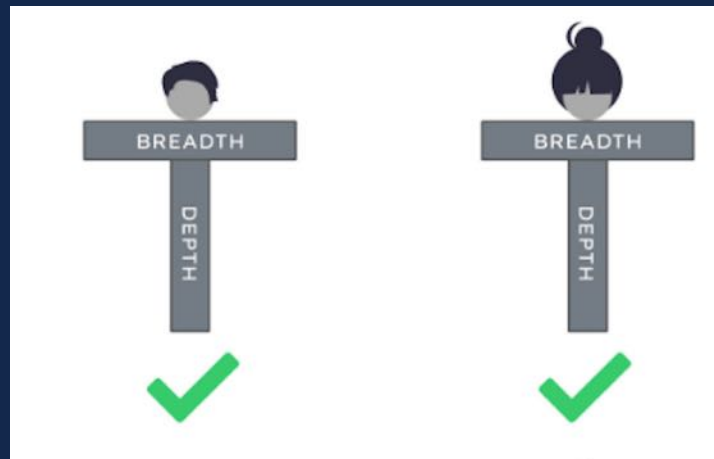
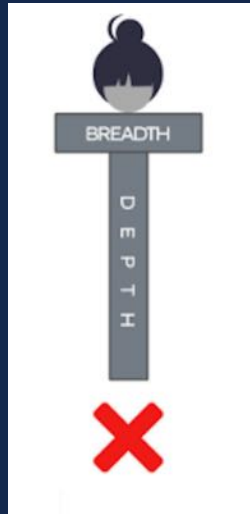
25



182



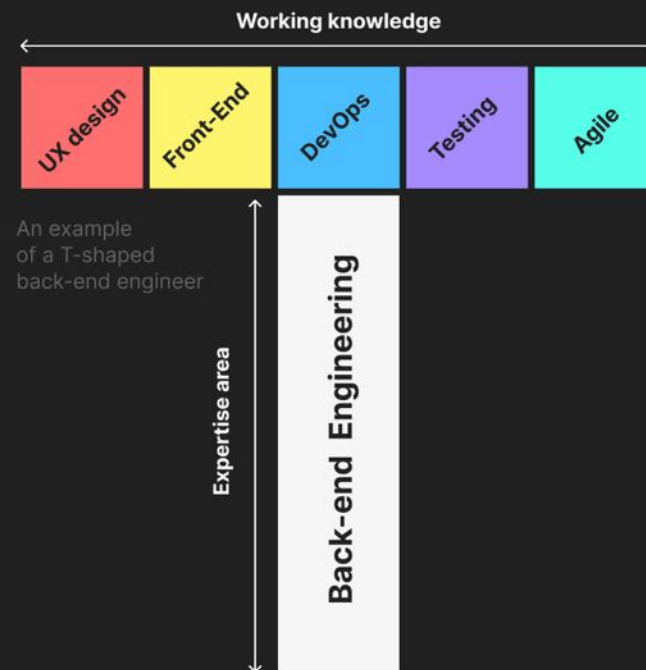
T-Shaped People



More Applied T

T-Shaped Software Engineers

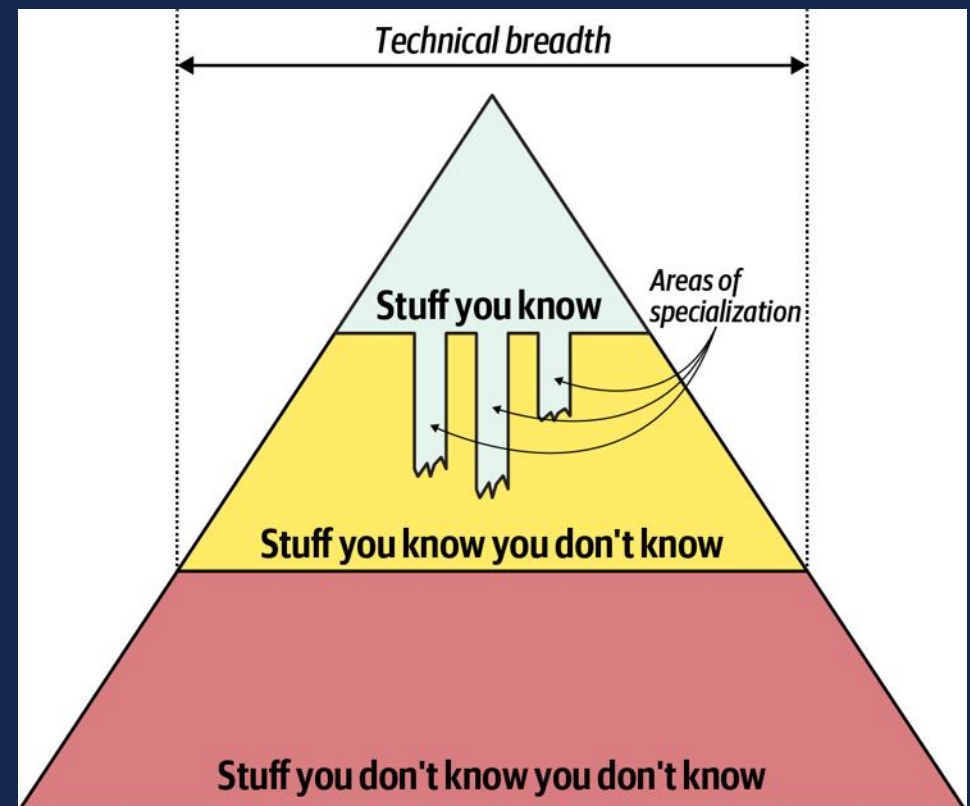
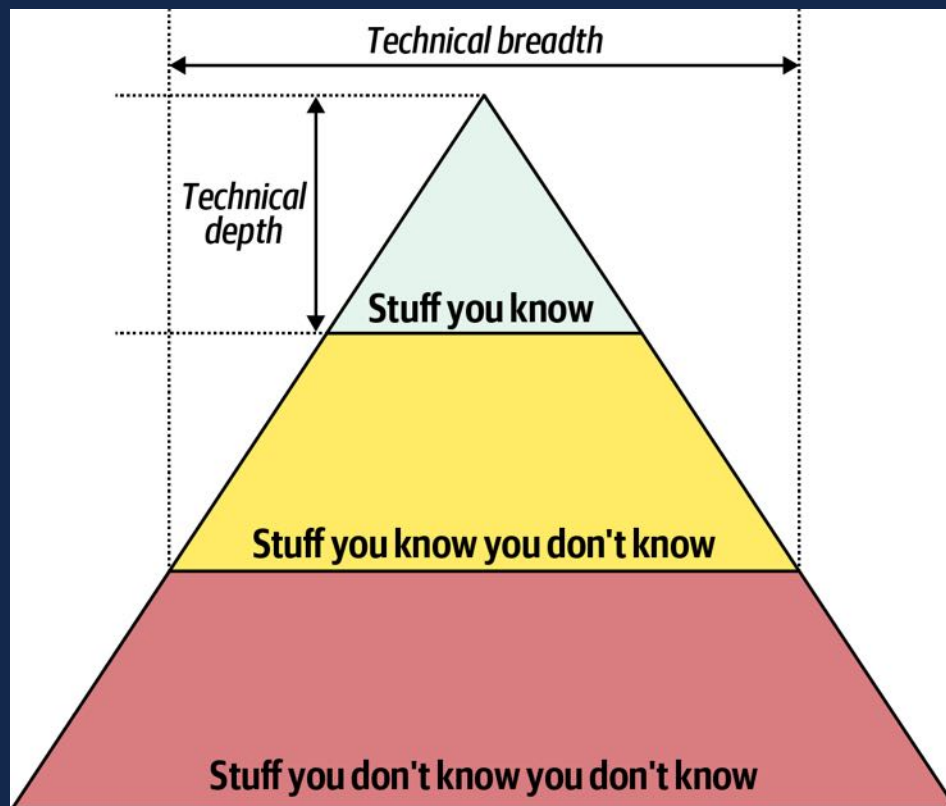
T-shaped engineers have deep knowledge of one area and a working knowledge of other areas their team needs, enabling them to deliver more value.



@pvukovic

2022

Same Thing - Different Way



The Need for Balanced T

- Too much generalization can lead to a “jack of all trades master of none” and things which probably would be easy become harder
 - If your shape seek careers in R&D, visioning or even management!
- Too much specialization creates blind-spots though in some disciplines the required time truly requires this degree of specialization
 - If this is your shape seek out jobs that are compartmentalized, but heed the warning potentials for stagnation or insecurity
- It is hard to promote low breadth, studies have suggested that high performing break-thru scientific teams had breadth & diversity. The break-thus happened because of liminal and lateral thinking
 - The class interaction assignment suggest you are more of this on a whole than you might believe

Overload and Fatigue

Mental Costs of Decisions

- Optimal decisions and will power have been studied to come from a finite source
 - This source is related to rest and glucose availability
- Generally this idea is related to what is termed decision fatigue
https://en.wikipedia.org/wiki/Decision_fatigue
- General Effect

“...if your willpower is depleted, you’re more likely to make a decision that’s good in the short-term, but bad in the long-term. Once your willpower is depleted, you become less likely to thoroughly consider the complex tradeoffs some decisions require.”

<http://greatnotbig.com/2011/08/decision-fatigue/>
- Trouble: Coding is filled with decisions!

Decision Fatigue and the Modern Programmer

- Programmers are people this stuff effects us
 - Maybe more than the average person since we work so heavily with decisions
 - <http://effectivesoftwaredesign.com/2011/08/23/how-decision-fatigue-affects-the-efficacy-of-programmers/>
- Example: Copy-Paste coding
 - What's easier thinking out an abstraction or copying paste some working code?
 - Answer: The later
 - Downside: Your code bloated! Short-term gain but long-term consequence of software rot, technical debt or fragility

Wait what!? Why?



```
1  if (me.initAfterRender) {  
2    if (me.initIf && eval(me.initIf)) {  
3      me.widget.on('afterrender', me.init, me);  
4    } else {  
5      me.widget.on('afterrender', me.init, me);  
6    }  
7  } else if (me.initOn) {  
8    // TODO  
9  }
```

Tech Fatigue

- Computing and the web space in particular suffers from a proliferation of technologies
 - <http://www.sitepoint.com/drowning-in-tools-web-development-industry/>
 - <https://medium.com/@ericclemmons/javascript-fatigue-48d4011b6fc4#.tvzld2bjg>
 - TodoMVC example
 - Languages that compile to JS
- Generalized false assumption: 2.0 better than 1.0 - (See Consumer Devs Later)
 - If so, why do the majority of movie sequels suck so bad!?
- Combine this with impostor syndrome (if I don't learn X I am not worthy) and perfectionism (better tool or approach just over the horizon) and you can spend more time evaluating than doing (remember Yak Shaving?)

Solution Anxiety and the Rise of Identity and Brand Based Developers

Constantly Fight Solution Anxiety

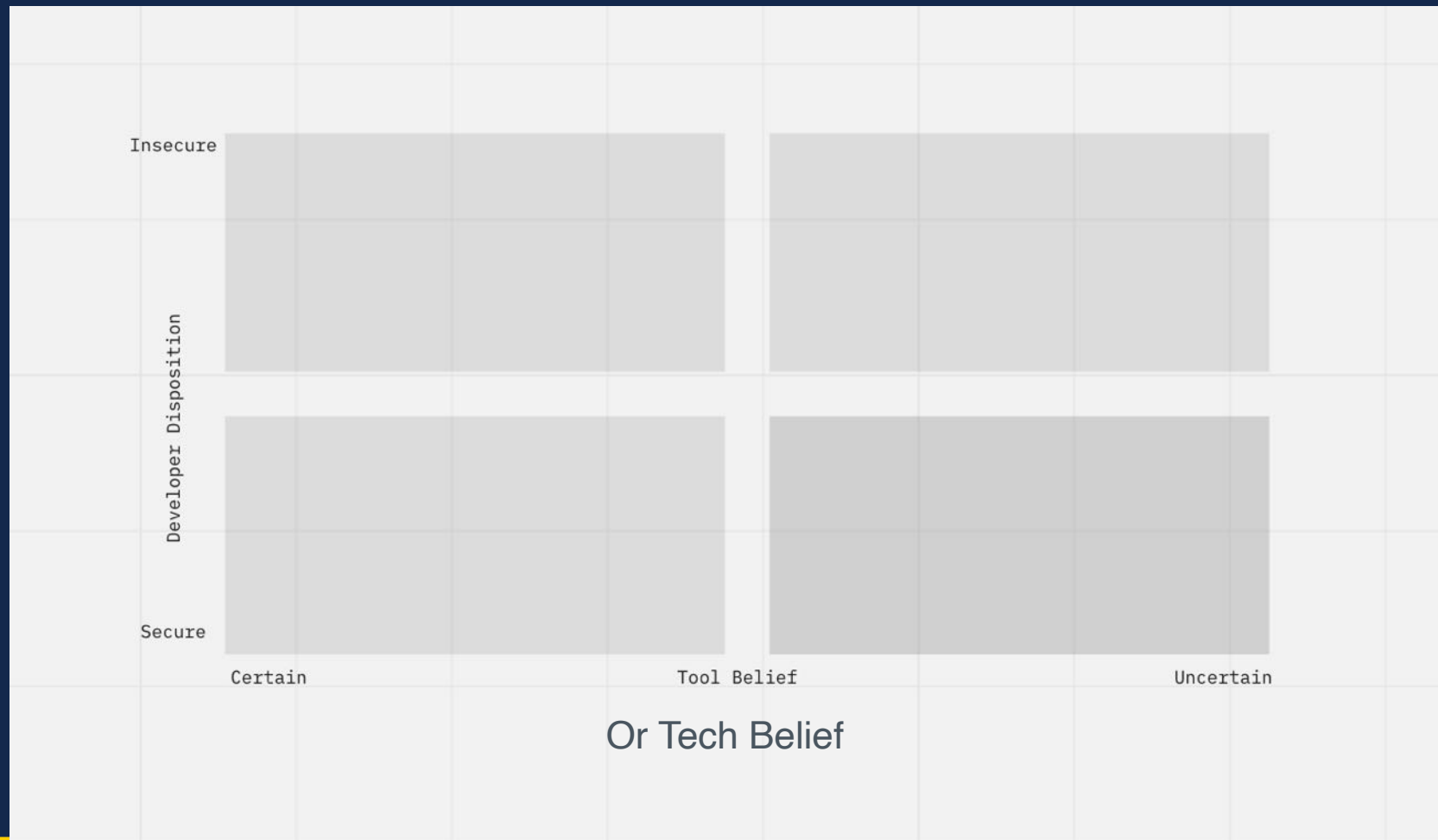
- Worrying about the future and trying to plan for every possible contingency is in some ways a form of anxiety
- Obviously we should be prudent and not under-plan, but a “what about”, “what if” taken to an extreme borders on actual anxiety activities as opposed to go engineering
- Stay present and try to focus on real risks and real problems you are currently facing or about to face
- Get too anxious about all the things expect productivity to plummet and you might end up in a state I often dub “spinning”. (Spinning around with lots of vigor but not really making progress and getting a bit dizzy and disoriented to where things actually stand)

The Rise of Brand Developers

- Today many developers are brand or influencer effected developers who associate with their tech too deeply and are easily triggered if Reddit, Twitter, etc. is any guide at all
- It is easy to demo this and I have been doing it for well over 10 years as I have watched the trend magnify well beyond where it used to be the extreme minority of our trade.

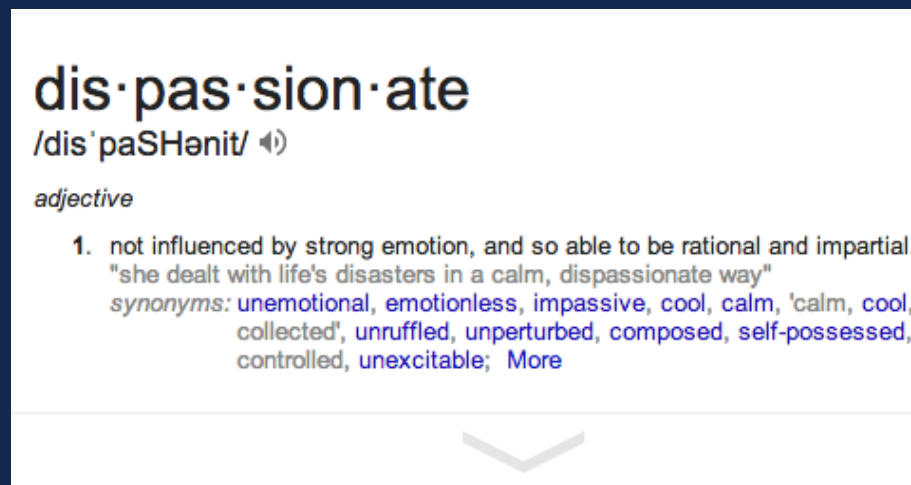
Trigger Warning: The thought experiment I'll now say is not unique to 2021 and what I'll say is meant to break any spell you might be under to see the suffocating "dev consumer" mindset which is terribly unhelpful for making pragmatic engineering decisions

Where do we fall?



Pro Tip: Employ Dispassionate Decision Making About Tech As Much As Possible

- YOU != YOUR CODE, YOU != YOUR FRAMEWORKS, YOU != YOUR IDEs, etc. etc. etc.
- If this is how you feel ask WHY you feel this way know you probably should aim to be more dispassionate



The Power of Process and Habits

Process and Consistency for the Win

- You can't control outcomes, but you can control the process.
 - Your destination is actually never fixed, but the way you walk the journey can be.
- Earned dopamine is more satisfying than all other forms of it. It is the delayed gratification, the lack of it that gives it the richness.

Long-term consistency beats short-term intensity and only requires a 3 step process

1. Show up.
2. Do your thing.
3. Do that thing some minimum amount of time.

Process Wins Over Time - Take 2

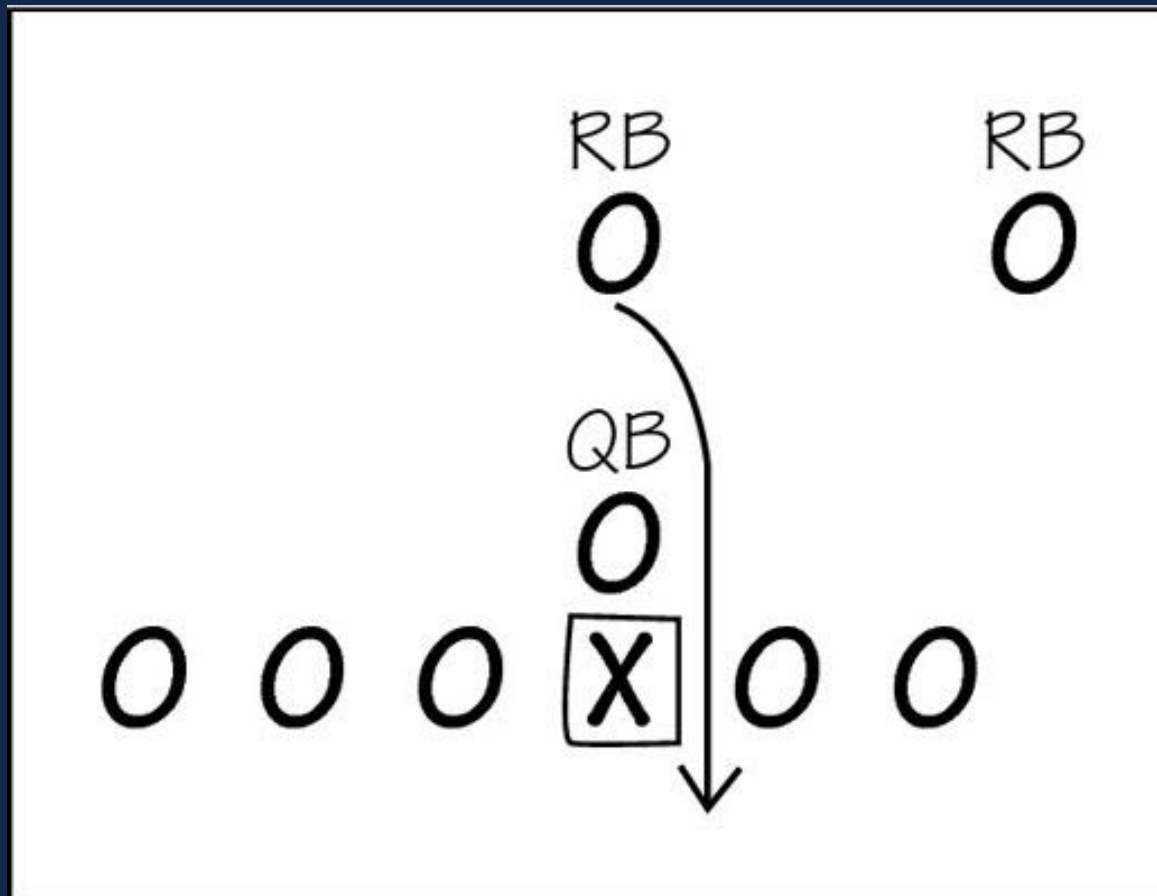
Being process-oriented, not product-driven, is the most important and difficult skill for a **software engineer** to develop.

Being process-oriented means:

- 1 seeking to understand a **problem** before chasing after solutions;
- 2 not force-fitting solutions to old problems onto new problems;
- 3 removing yourself from prideful investment in your projects and being slow to fall in love with your ideas;
- 4 making **investigations** and decisions holistically (that address several aspects of a **problem** at once) rather than sequentially (that finalize one aspect of a solution before investigating the next);
- 5 making **decisions** conditionally—that is, with the awareness that they may or may not work out as you continue toward a final solution;
- 6 knowing when to change and when to stick with previous decisions;
- 7 accepting as normal the anxiety that comes from not knowing what to do;
- 8 working fluidly between concept-scale and detail-scale to see how each informs the other;
- 9 always asking “What if . . . ?” regardless of how satisfied you are with your solution.

Lots of thinking and attitude in here!

Solution: Run 1 Yard Plays Consistently!?!

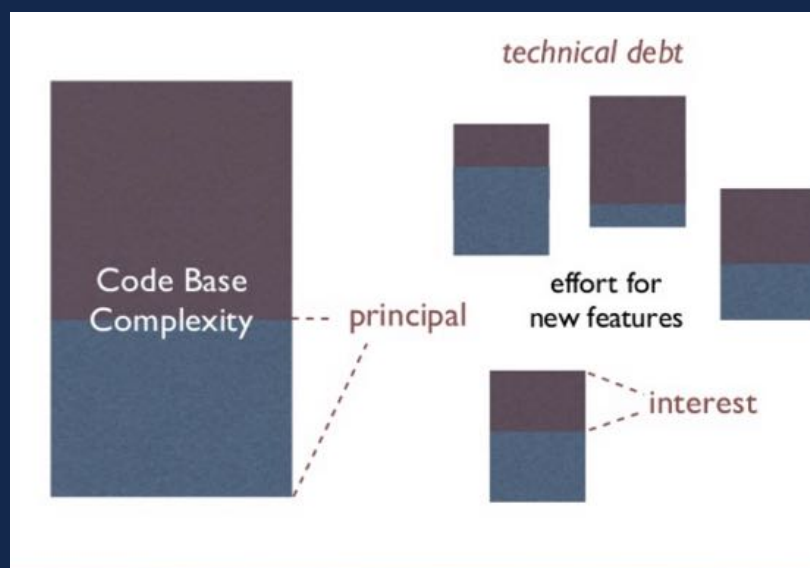


Good Habits

- Habit - an acquired behavior pattern regularly followed that becomes almost involuntary
- Habits are the kind of the things that can lead to some forms of 'multitasking'
 - The muscle memory of editing and reformatting for example
- How long to develop a habit?
 - 21 days, 6 months, a long time
- Hard to stick to habits
 - Try 'Micro Habits' – 1-yard plays is what I call this
 - Code cleaning, relate to job site cleaning and technical debt
- Keystone Habits can lead to much much more

Habit Example

- Cleaning up the work site I mean code base is a really good habit to have
 - If you don't clean up, generally bad things happen (eventually)
 - “Sweeping” your code can be the boring part but it is vitamins that avoids your eventual need for aspirin



Habits of The “Good” Programmer Trending Again

Summary

- Write clean code
- Test
- Script everything
- Learn your tools by heart
- Optimize for flow

<http://www.slideshare.net/andersjanmyr/habits-of-a-responsible-programmer>

Small Habitual Changes = Big Eventual Changes

We are, in a very real sense, the sum total of our habits, and nothing more; hence by changing our habits, once we understand which ones we should change and in what directions, and understand our limitations in changing ourselves, then we are on the path along which we want to go.

The Art of Doing Science and Engine...

Richard W. Hamming and Bret Victor



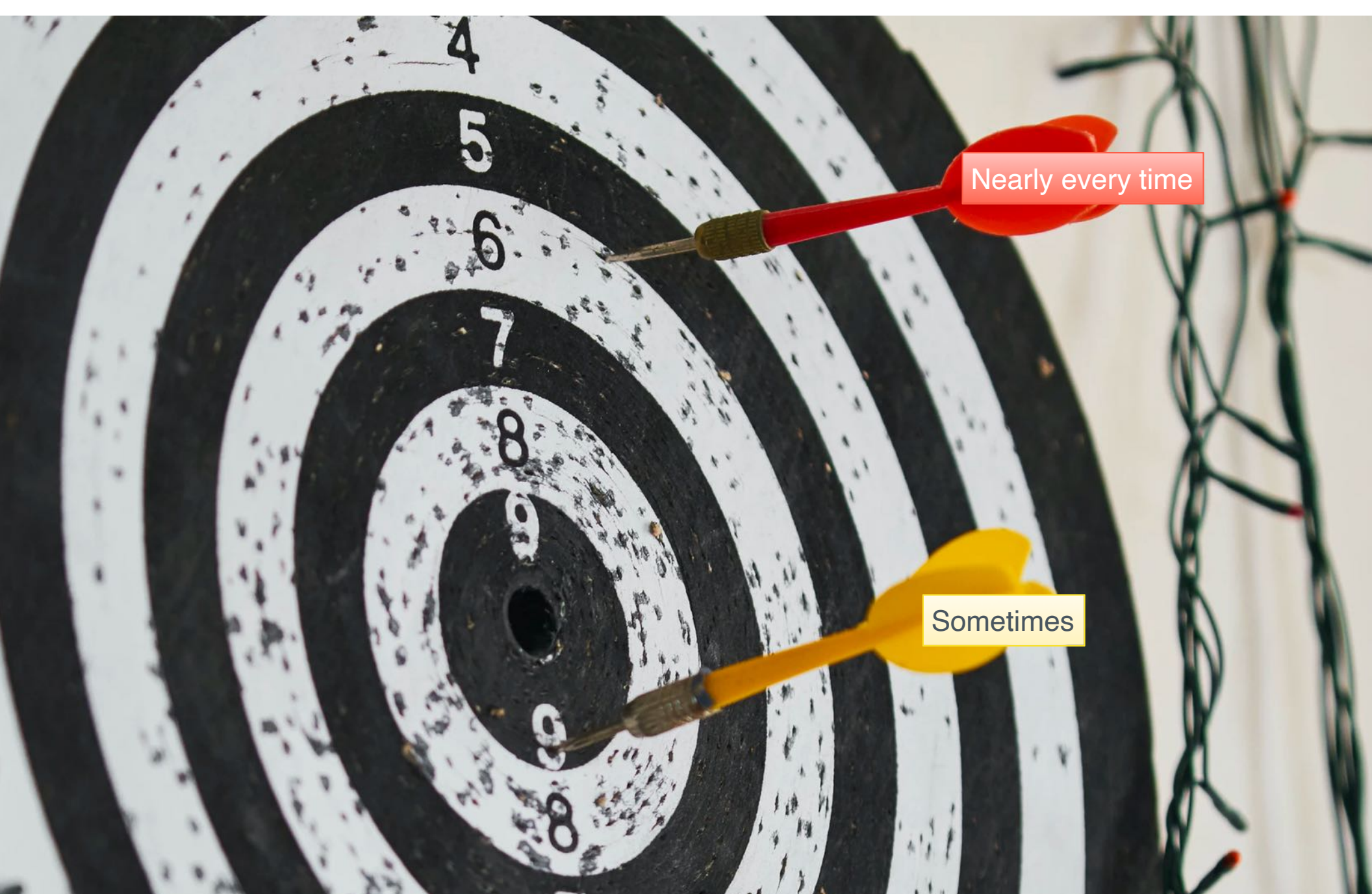


Software Engineer Kata Rules

Focus
+ Discipline
Success

Example: How do we actually write our code?

- Are we _____ ?
 - Structured
 - Methodical
 - Predictable
 - Measurable
 - Proactive
 - With ample time and pace
- Or are we
 - Creative or disorganized (depending on situation)
 - Unsystematic
 - Unpredictable
 - Unmeasured other than in service of our grade or boss
 - Reactive
 - Under time pressure and rushed



Aim for consistency not for perfection

If we are honest about all this

- We probably aren't engineering anywhere near the definition of the word ... yet we are absolutely doing interesting and useful things !
- Bluntly we are practicing our dev efforts more as a *craft* and that might be ok actually.
- This is probably appropriate before we “industrialize” in my opinion and a needed step
- Prof Thought: “I strongly believe part of the failure of software engineering is the lack of craftsman culture mastery from the individual practitioner up to the industry as a whole.”

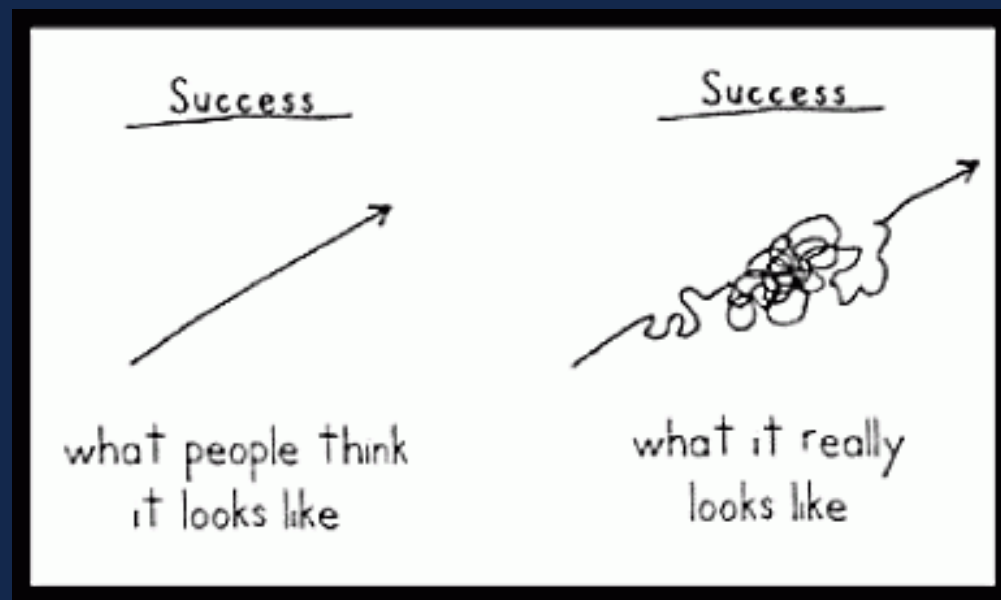
Accept Fails

Living with Imperfection

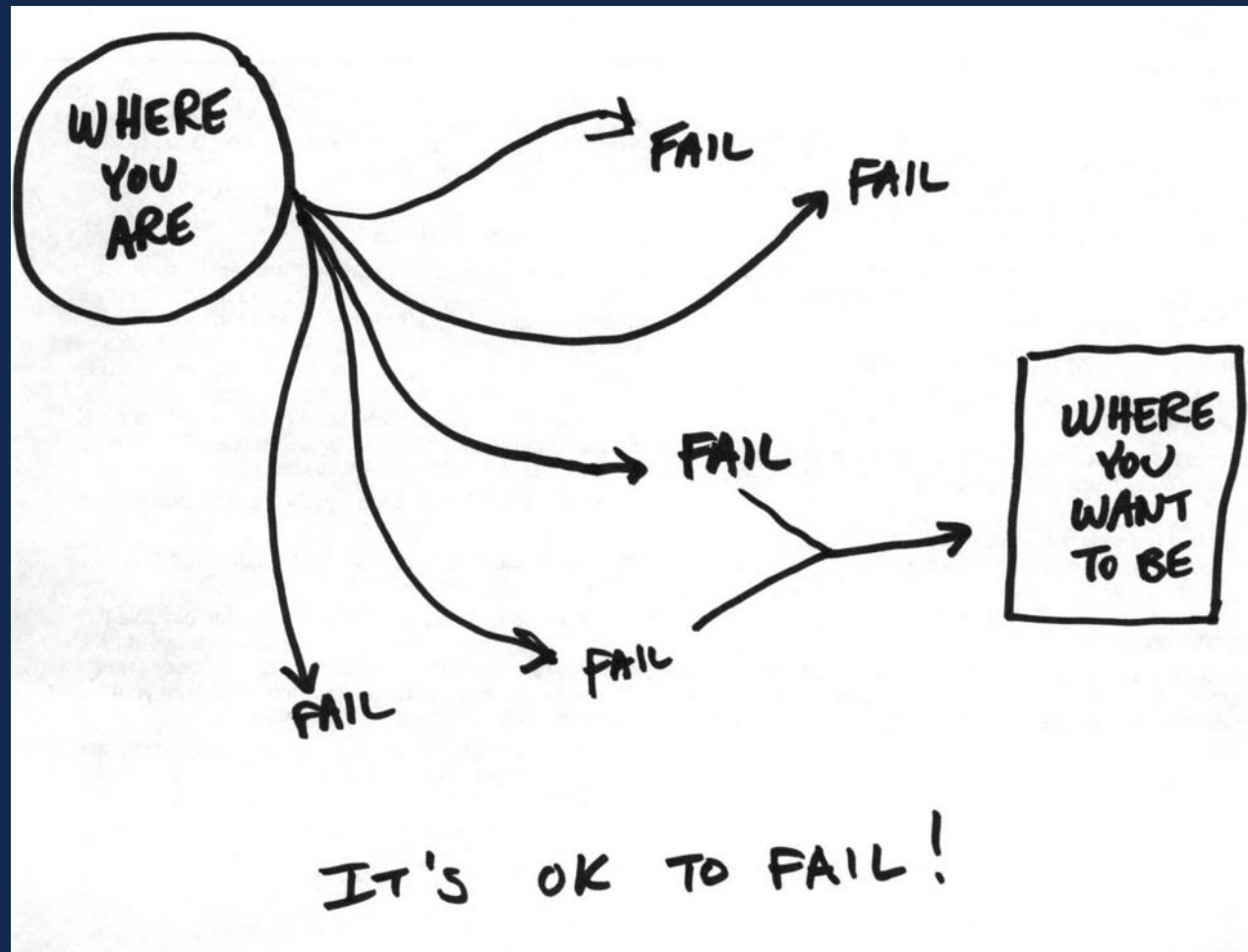
- Fact: Some days just don't go well and you won't get into flow at all and we will make serious mistakes!
- Very productive good days shouldn't be interpreted as what should always happen and in fact may only be good relatively or be enabled because of many "less good" days
- Ultimately you will fail
 - You will ...
 - write some boneheaded code
 - push broken code to master
 - overwrite working code
 - get very little done
 - Add your own
- Be very careful relating an unproductive day, bad code review, poor decision, screwed up algorithm, crash, etc. with your own self worth.

Embrace Failure

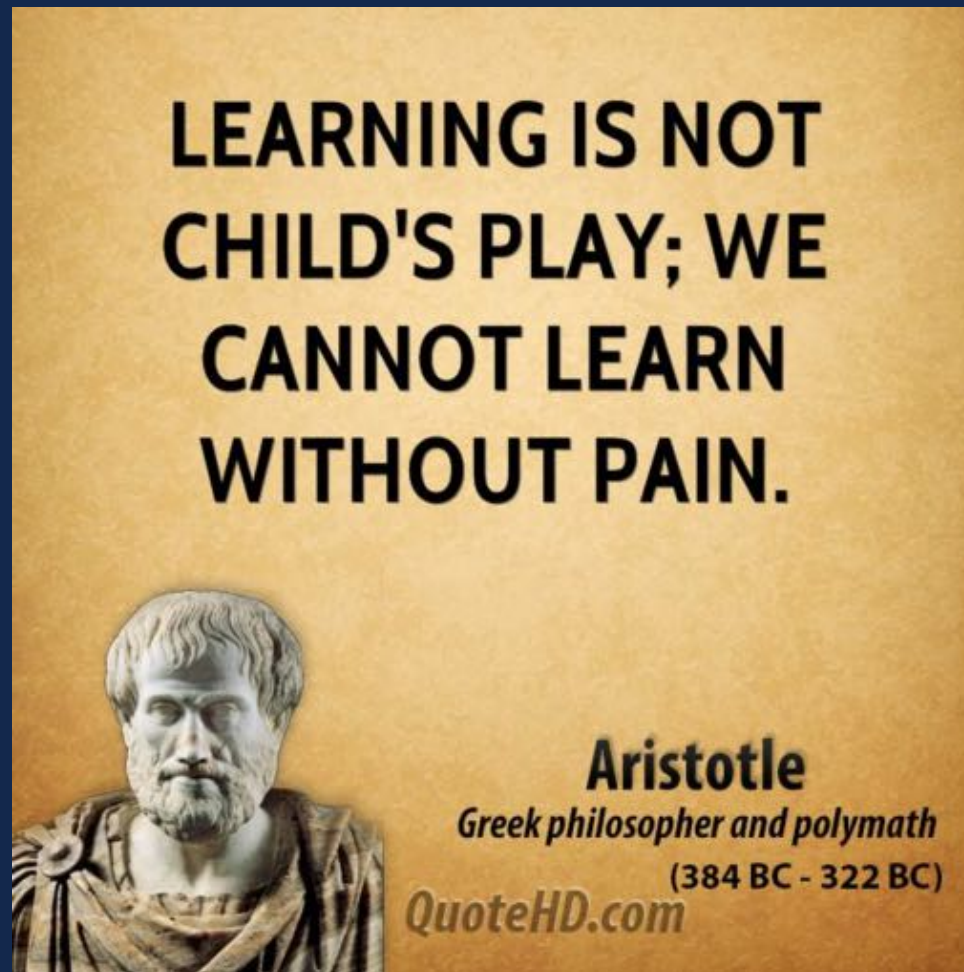
- Generally getting to a successful outcome takes many failed outcomes.
 - Well known quotes from Edison and others
- This often goes against the romantic idea of things



Seeing the path to success from the place you failed



The pain of learning and failure is certainly not a new idea



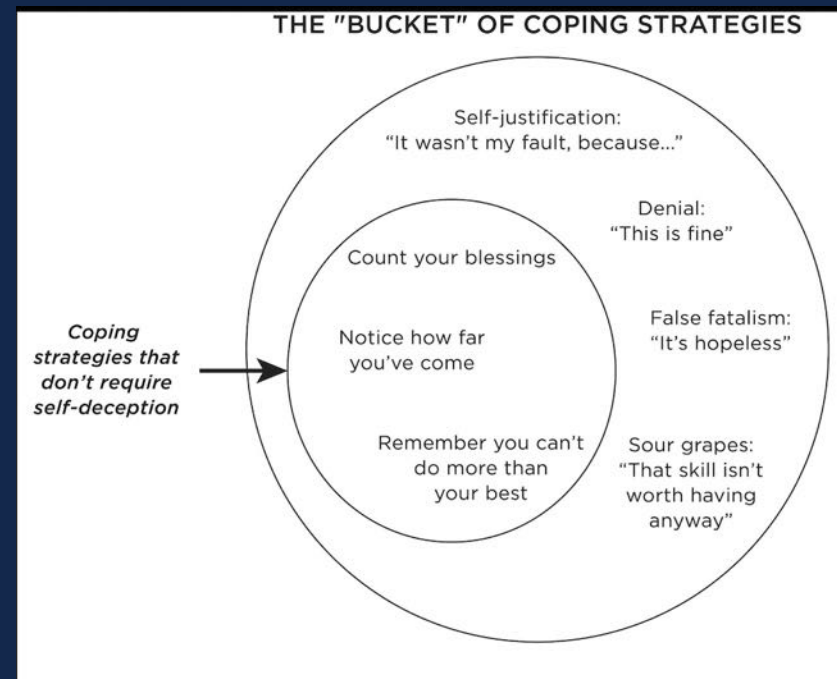
SUCCESS IS A LOUSY TEACHER.



**IT SEDUCES SMART PEOPLE INTO
THINKING THEY CAN'T LOSE.**

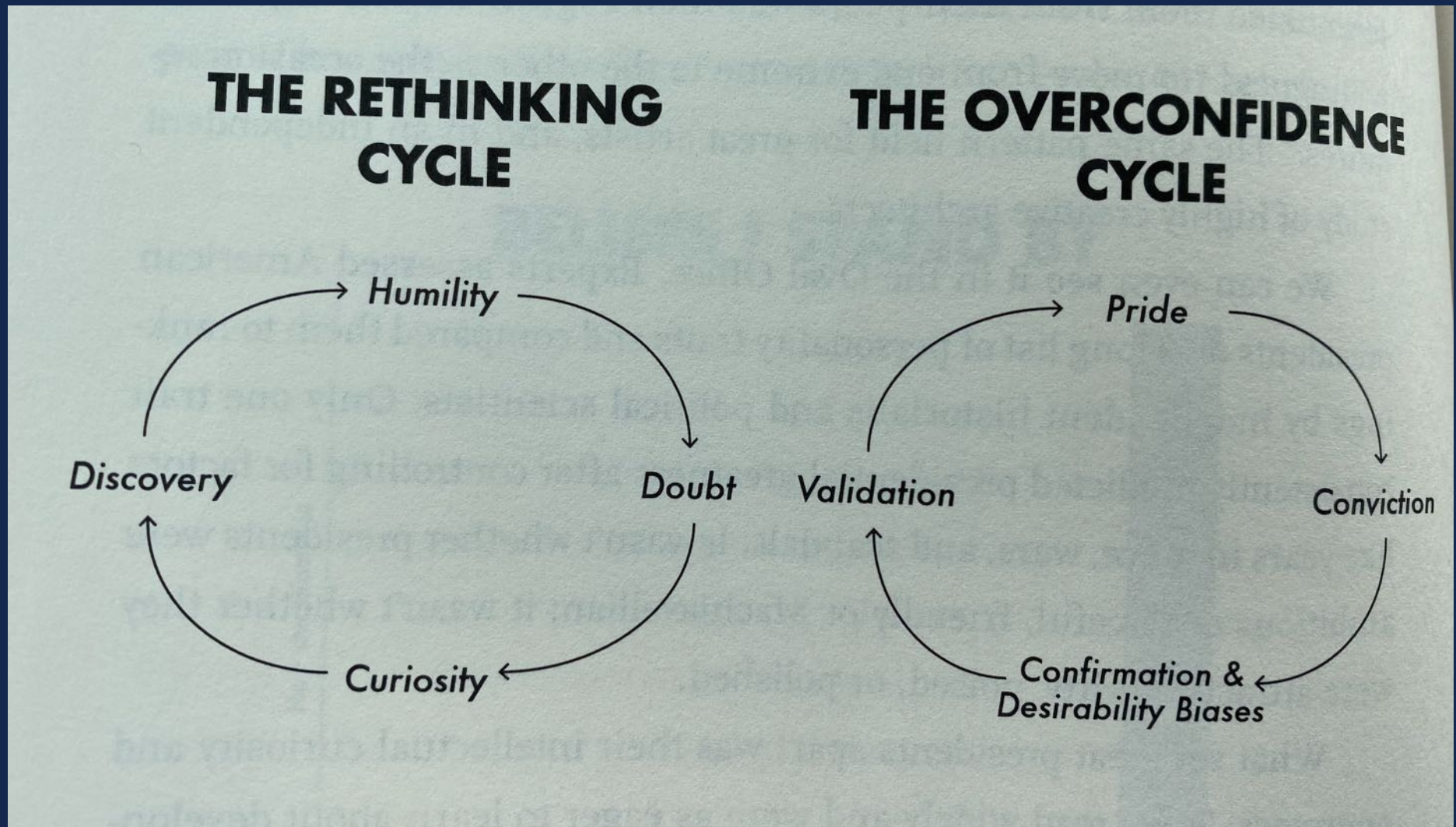
Dealing with “Failure” and Adversity

- Things won't always go well, we may fail or encounter very challenging moments. The key question is how we deal with this as people
- Developers are like any other people and we will have lots of fail, the question is how do we cope with these “fails”?
 - Health vs. Unhealthy Coping Strategies

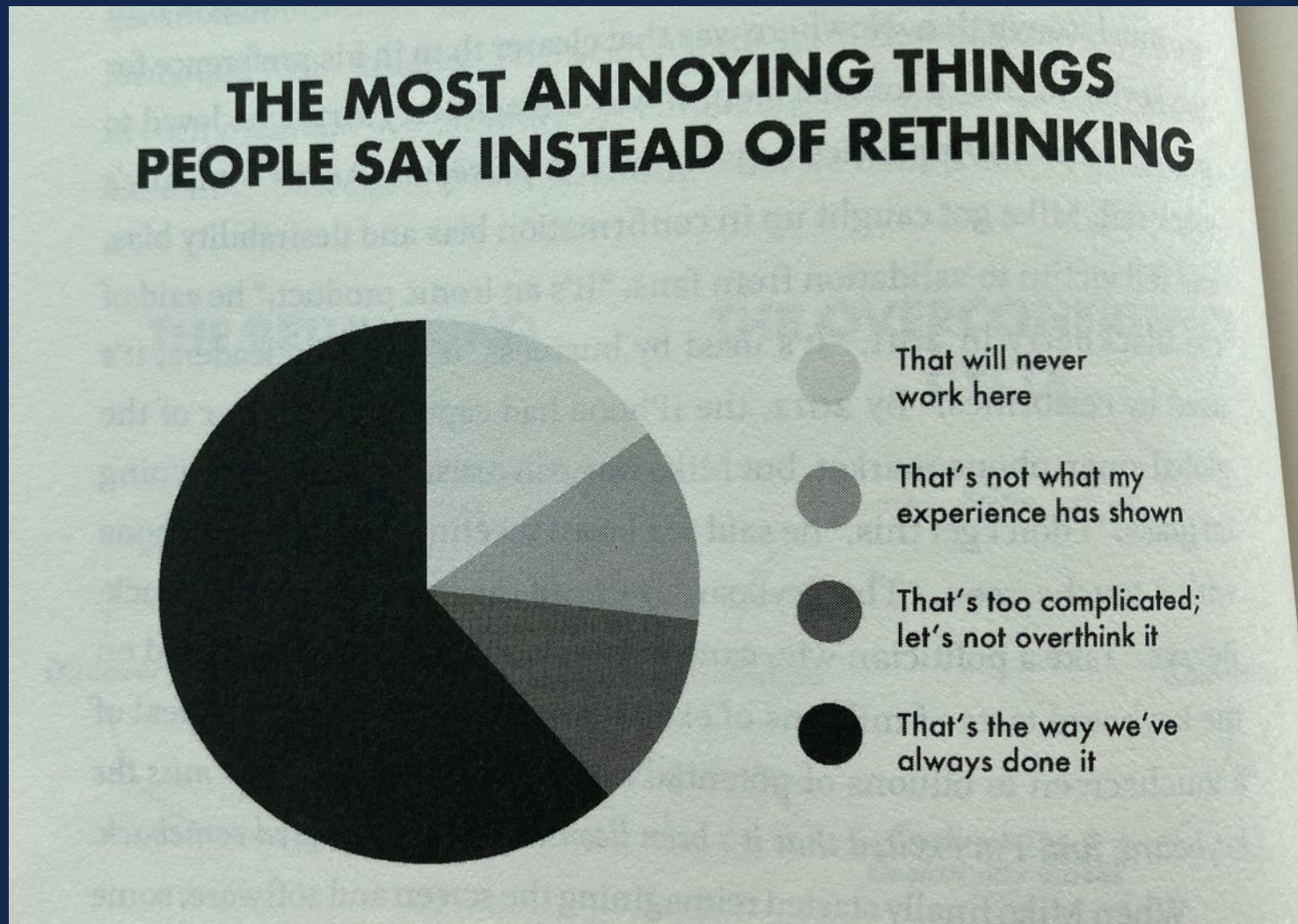


Rethinking our approach can really help us when we “fail” at something

Allow Opportunities to Rethink Things



Rhetorical Methods for Dodging Rethinking

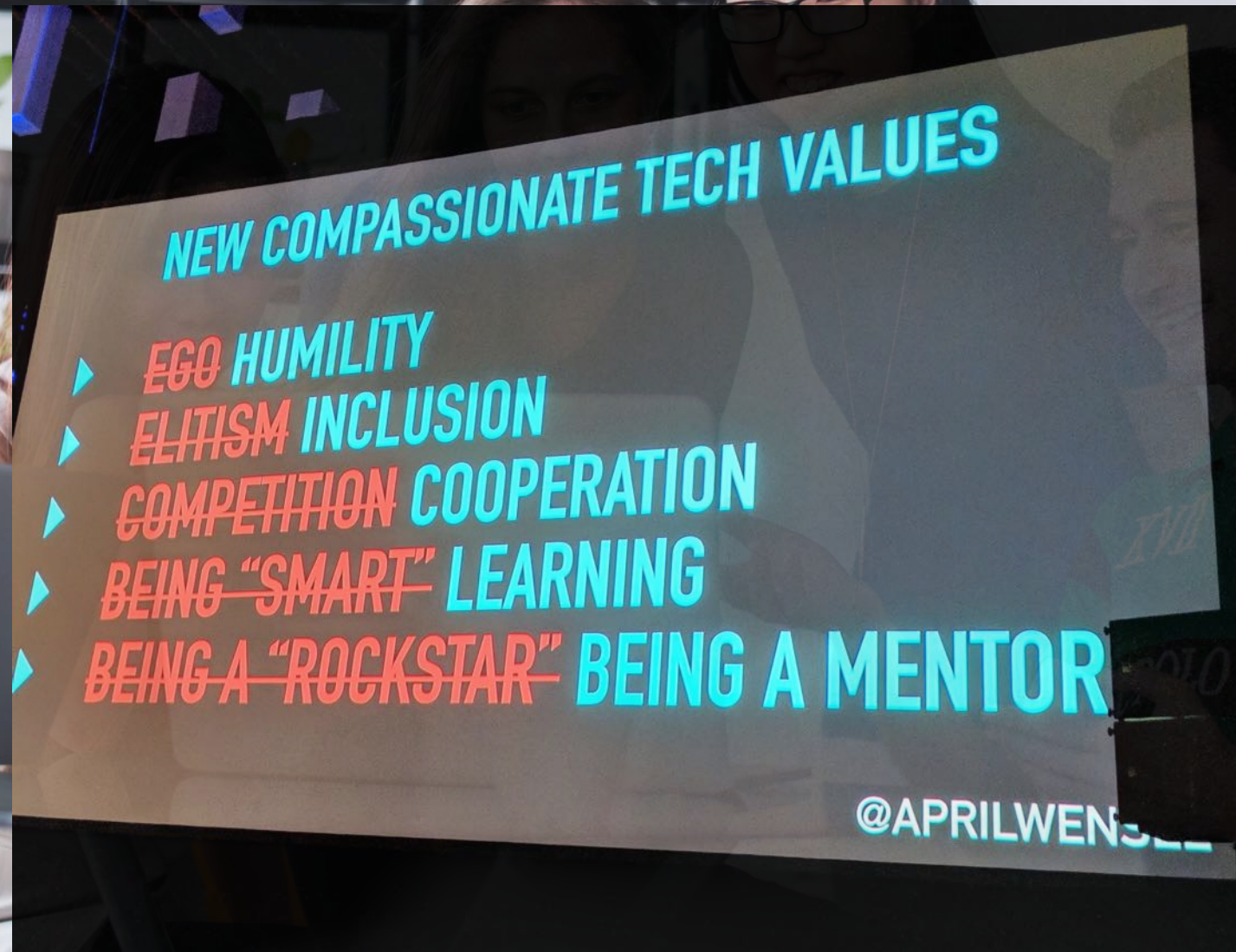


Aim to Be Broadly Aware

Accept Your Role

- What we do really does matter in our software driven world
- Your decisions have impact often much larger than you think.
 - Example: Semantic HTML and Exclusion
 - Example: Exclusion via poor execution (demo time!)
- Ignorance of our decision impact is not a valid excuse, it is our job to think about this kind of stuff
- Solution: Start first with your values and live those into what you build not build stuff and try to overlay ethics later!

SE Values Example



Admit it's a long process

Train for a Marathon, Not Sprints

- Most self-aware successful people tend to acknowledge the role of perseverance, timing and luck in their eventual success
- Writing meaningful code takes quite a LONG time
 - If your code becomes legacy then you win...it generally means it is useful!
- Careful mistaking that you prove the reverse with the speed code effort using the proof of idea by PoC aim of MVP development, hackathons, market effects, etc.
- Code can be like wine, it can age well, but only under the right conditions and processes, take UNIX as an example. Conversely code can be like milk and spoil quickly, especially if exposed to unstable dependencies & environments

Try Not to Panic or Rush, Be Realistically Patient Instead

- Don't Panic - There is too much to know - there always will be (pie chart here), You always be faced with the unknown.
- Be Patient - it takes time to get good, you are innately born with the ability you have to get the ability with work, practice and experience.
- Be Realistic - There is only so much time and money you are working with. Stop Math solving with some perfect solution and start solving as good as you need to for what you are doing

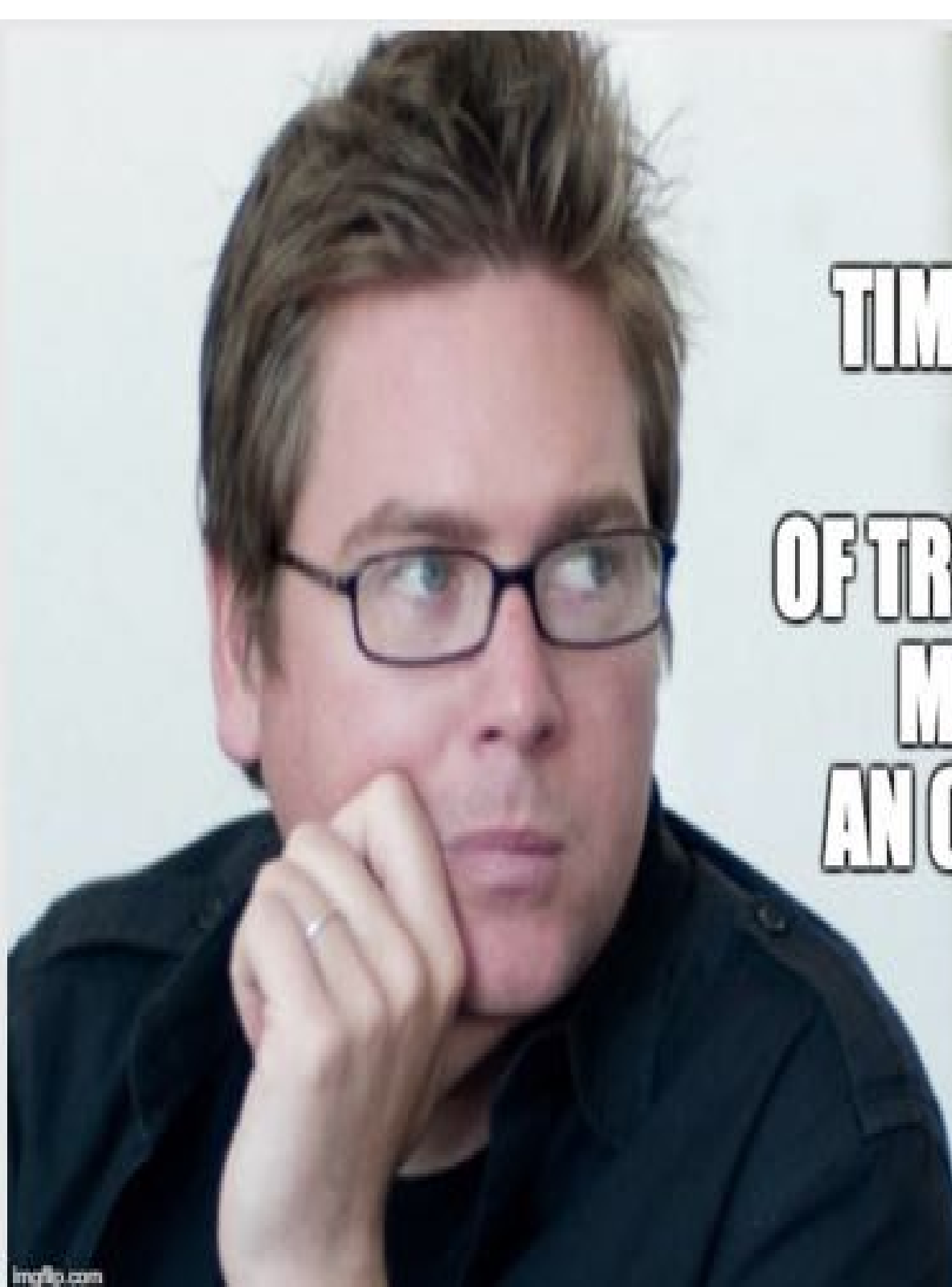


The belief that quality and discipline increase speed is a courageous belief because it will constantly be challenged by powerful but naive folks who are in a hurry.

Clean Agile

Robert C. Martin



A man with brown hair and glasses, wearing a dark shirt, is shown from the chest up. He has a thoughtful expression, with his hand resting on his chin. The background is a plain, light-colored wall.

**TIMING, PERSEVERANCE,
AND TEN YEARS
OF TRYING WILL EVENTUALLY
MAKE YOU LOOK LIKE
AN OVERNIGHT SUCCESS.**

-BIZ STONE

Wind Down

Wind Down – A Graphic Review?

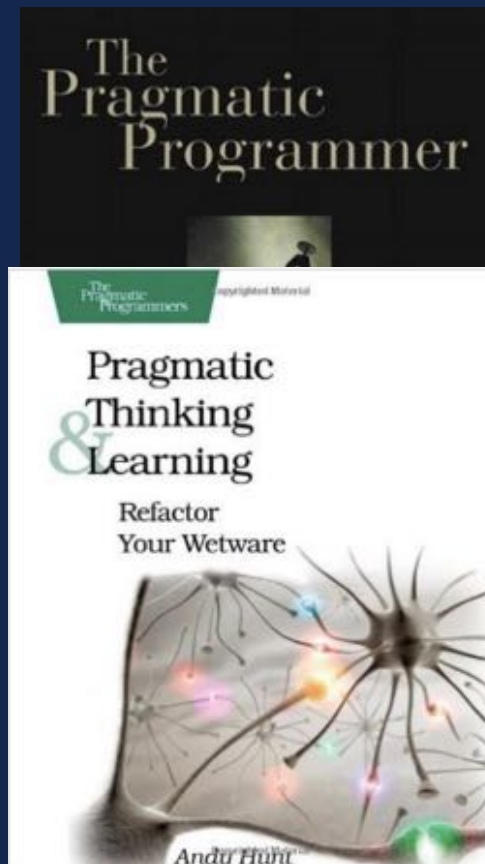
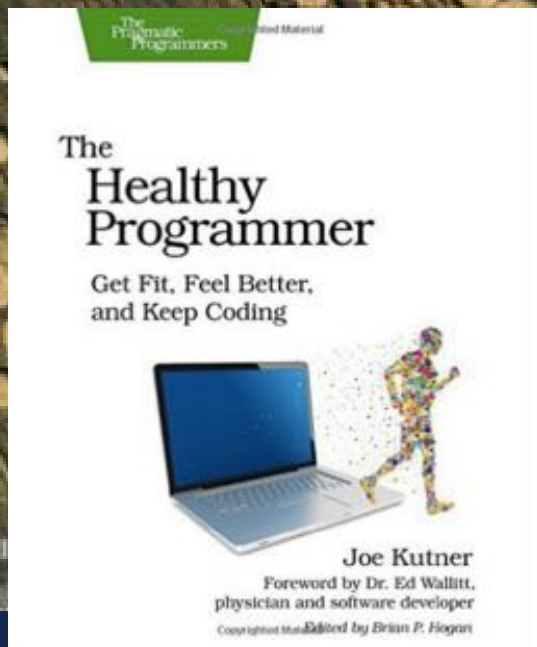
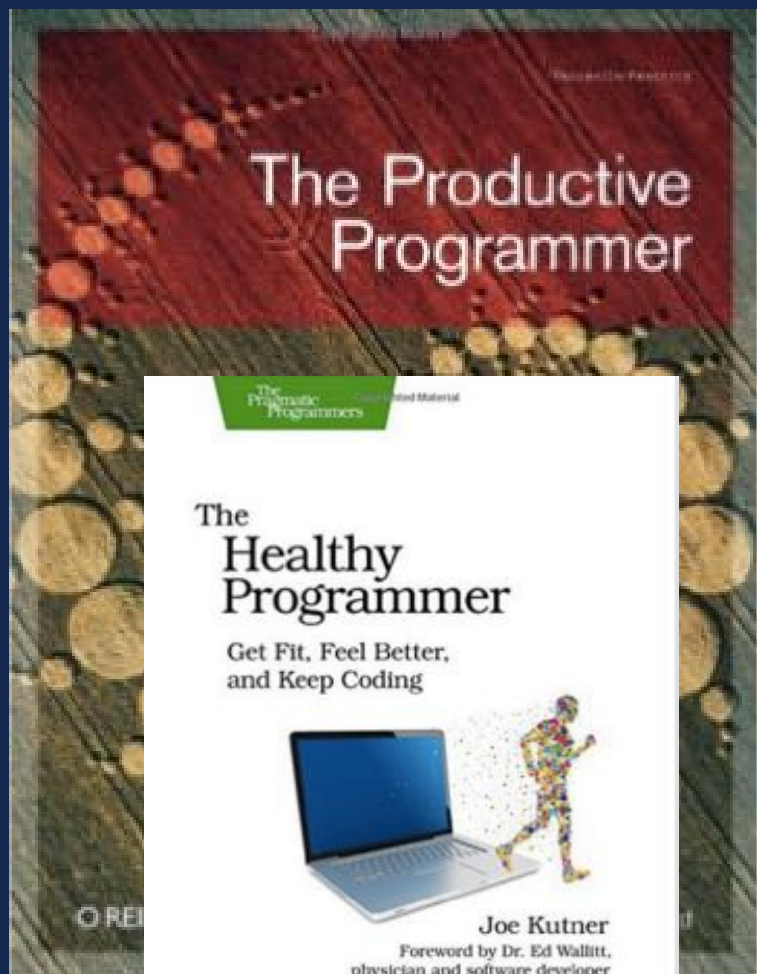


Seriously Though ... Stop, Drop and Apply

- Clearly, we can get quite off track and end up deep in self-help tactics and introspection
 - Maybe not a bad thing unless it leads to analysis paralysis
 - However, none of this stuff is good for us as aspiring Software Engineers unless we find some applied use for it.
- For our mixed online and in-person quarter I think this deck will be more important than it usually is so. Whatever you can do to improve yourself will probably pay off in ways well beyond your team or this course.

BTW if you do find your tools or techniques are useful, we especially want you to share with the whole class

Finally, There's a Reason These Exist



Nothing in this deck or these book should be weird, if it is ask why it might help you see your thoughts and biases

Coming Up in CSE 110

- Group Formation
- Group and Approach Lectures
- Group Intros
 - Team Building
 - Tech Warmup
 - Reveal Videos